

Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
ДИПЛОМНОГО ПРОЕКТА НА ТЕМУ:**

Председатель ГЭК _____ к.т.н., доцент _____ В.К. Дюбков
 должность, ученая степень, ученое звание _____ дата _____ подпись _____ И. О. Ф.

Минск 2026

Лист задания. Не вставлять. Заглушка просто

Оглавление

Реферат	6
Abstract	7
Введение.....	8
1 Постановка задачи и аналитический обзор аналогичных решений	9
1.1 Обзор аналогов мобильного приложения	9
1.1.1 Аналог «Pill reminder»	9
1.1.2 Аналог «Medisafe».....	13
1.1.3 Аналог «MyTherapy».....	19
1.2 Постановка задач мобильного приложения.....	24
1.2.1 Реализация системы авторизации и аутентификации	24
1.2.2 Создание расписания приема лекарств.....	24
1.2.3 Разработка системы уведомлений о приеме лекарств.....	25
1.2.4 Просмотр справочной информации о лекарствах	25
1.2.5 Учет остатков лекарственных средств (аптечка).....	26
1.2.6 Визуализация статистики и формирование отчетов.....	26
1.2.7 Разработка интуитивно понятного интерфейса	27
1.3 Выводы по разделу	27
2 Проектирование мобильного приложения	28
2.1 Обзор средств разработки.....	28
2.1.1 Платформа Android	28
2.1.2 Язык программирования Dart	29
2.1.3 Среда разработки.....	29
2.1.4 Фреймворк Flutter.....	30
2.1.5 Управление состоянием (Provider).....	30
2.1.6 Облачная база данных Cloud Firestore	31
2.1.7 Аутентификация Firebase Authentication	31
2.1.8 Система локальных уведомлений (flutter_local_notifications)	32
2.1.9 Получение справочной информации о лекарствах.....	33
2.2 Архитектура программного продукта	33
2.3 Описание работы мобильного приложения.....	35
2.3.1 Описание диаграммы использования.....	37
2.4 Проектирование базы данных мобильного приложения	38
2.5 Проектирование классов приложения.....	43
2.5.1 Основные классы сущности	43
2.5.2 Контракты доступа к данным	43
2.5.3 Классы управления состоянием.....	44
2.5.4 Классы для работы с уведомлениями	44

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата	Оглавление			
Разраб.		Осадчий П.А.						
Пров.		Тимонович Г.Л.						
Н. контр.		Подрез А.А.						
Утв.		Блинова Е.А.			БГТУ 1-98 01 03, 2026			
					Лит.	Лист	Листов	
					У	1	3	

2.5.5	Взаимодействие классов в системе	45
2.6	Проектирование основных алгоритмов	46
2.6.1	Алгоритм работы первого запуска	46
2.6.2	Алгоритм создания расписания приема.....	46
2.6.3	Алгоритм планирования уведомлений	47
2.6.4	Алгоритм получение справочной информации о лекарстве	47
2.6.5	Алгоритм учета остатков при отметке приема	48
2.6.5	Алгоритм формирования статистики и экспорта PDF	48
2.7	Выводы по разделу	49
3	Реализация мобильного приложения	50
3.1	Разработка клиентской части	50
3.1.1	Реализация регистрации и авторизации пользователя	52
3.1.2	Маршрутизация и навигация	53
3.2	Реализация доступа к данным	55
3.2.1	Класс FirebaseService для работы с Firestore	56
3.3	Реализация взаимодействия между экранами	57
3.4	Реализация работы уведомлений	58
3.5	Реализация виджета рабочего стола	59
3.6	Реализация статистики и экспорта отчетов	61
3.7	Вывод по разделу	62
4	Тестирование мобильного приложения	63
4.1	Тестирование авторизации и аутентификации.....	63
4.2	Тестирование добавления расписания	64
4.3	Выводы по разделу	66
5	Руководство пользователя.....	67
5.1	Регистрация пользователя	67
5.2	Авторизация пользователя	68
5.3	Главная страница.....	69
5.4	Страница всех лекарств	71
5.5	Страница аптечки	73
5.6	Страница статистики.....	76
5.7	Виджет	77
5.8	Вывод по разделу.....	78
6	Технико-экономическое обоснование проекта	79
6.1	Общая характеристика разрабатываемого программного средства	79
6.2	Исходные данные для проведения расчетов	79
6.3	Методика обоснования цены.....	80
6.4	Определение объема программного средства	81
6.5	Расчет основной заработной платы	82
6.6	Расчет основной заработной платы	83
6.7	Расчет отчислений на социальные цели.....	84
6.8	Расчет расходов на специальное оборудование и платных услуг.....	84
6.9	Расчет расходов на материалы	85
6.10	Расчет прочих прямых затрат.....	85
6.11	Расчет общепроизводственных и общехозяйственных расходов	85

6.12 Расчет суммы расходов на разработку программного средства 86

6.13 Расчет расходов на сопровождение и адаптацию 86

6.14 Расчет полной себестоимости программного средства..... 87

6.15 Определение цены, оценка эффективности..... 87

6.16 Выводы по разделу 89

7 Анализ информационной безопасности приложения 90

7.1 Процедуры идентификации и разграничения доступа..... 90

7.2 Обеспечение безопасности при обмене данными 91

7.3 Конфиденциальность 92

7.4 Защита системы уведомлений 93

7.5 Резервное копирование и синхронизация данных..... 93

7.6 Выводы по разделу..... 94

Заключение 95

Список использованных источников 96

ПРИЛОЖЕНИЕ А Диаграмма развертывания..... 97

ПРИЛОЖЕНИЕ Б Диаграмма вариантов использования 98

ПРИЛОЖЕНИЕ В Логическая схема базы данных..... 99

ПРИЛОЖЕНИЕ Г Блок-схема алгоритма создания расписания приема 100

ПРИЛОЖЕНИЕ Д Таблица экономических показателей..... 101

ПРИЛОЖЕНИЕ Е Скриншот работы программы 102

Реферат

Пояснительная записка дипломного проекта содержит 102 страницы, 46 иллюстраций, 12 таблиц, 11 литературных источников, 6 приложений, 13 листингов.

FLUTTER, DART, FIREBASE, PROVIDER, ANDROID STUDIO

Целью дипломного проектирования является разработка мобильного приложения для отслеживания приема лекарств.

Для достижения цели необходимо выполнить следующие задачи:

- провести анализ аналогичных решений;
- сформировать требования к программному средству,
- спроектировать и разработать программное средство,
- протестировать разработанное приложение;
- разработать руководство пользователя;
- провести технико-экономическое обоснование проекта.

Пояснительная записка дипломного проекта состоит из реферата, оглавления, шести глав, заключения и списка использованных источников.

В первом разделе описаны аналоги данного программного средства, произведен обзор теоретического материала, выполнен патентный поиск по теме данного дипломного проекта, а также описан набор используемых технологий.

Во втором разделе описан процесс проектирования программные средства, описаны объекты базы данных и алгоритм составления расписания.

В третьем разделе приведено описание процесса разработки серверной и клиентской части веб-приложения. Приводятся листинги исходного кода и описания архитектурных решений.

В четвертом разделе проводится тестирование программного средства.

В пятом разделе приведено руководство пользователя.

В шестом разделе приводится расчет экономических показателей и себестоимости программного средства.

В заключении подводятся итоги выполнения дипломного проекта, обсуждаются достигнутые результаты и перспективы развития веб-приложения.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			Реферат	Лит.	Лист	Листов
Пров.		Тимонович Г.Л.				У	1	1
						БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.						
Утв.		Блинова Е.А.						

Abstract

The explanatory note of the degree project contains 102 pages, 46 illustrations, 12 tables, 11 literary sources, 6 appendices, 13 listings.

FLUTTER, DART, FIREBASE, PROVIDER, ANDROID STUDIO

The purpose of the work is to develop a mobile application for tracking medication intake, designed to help patients follow their prescription schedules accurately through timely reminders and dose logging.

To achieve the goal, you must complete the following tasks:

- to analyze similar solutions;
- create requirements for the software,
- design and develop a software tool,
- test the developed application;
- develop a user 's guide.

To conduct a feasibility study of the project. The explanatory note includes an abstract, table of contents, six chapters, a conclusion, and a list of sources.

The first section describes analogs of this software tool, reviews the theoretical material, patent search on the topic of this diploma project, and also describes the set of technologies used.

The second section describes the software design process, describes the database objects and the scheduling algorithm.

The third section describes the development process for the server and client parts of the web application. Provides source code listings and descriptions of architectural solutions.

The fourth section conducts testing of the software.

The fifth section provides a user guide.

The sixth section provides the calculation of the economic parameters and the cost of the software tool, including key cost components and estimation methods.

The conclusion summarizes the results of the diploma project, discusses the achieved results, and outlines the prospects for the development of the web application for online education in the field of programming.

					ДП 00.00.ПЗ								
		ФИО	Подпись	Дата									
Разраб.		Осадчий П.А.			Abstract			Лит.		Лист		Листов	
Пров.		Тимонович Г.Л.								1	1		
								БГТУ 1-98 01 03, 2026					
Н. контр.		Подрез А.А.											
Утв.		Блинова Е.А.											

Введение

Современные смартфоны стали неотъемлемой частью повседневной жизни. Они всегда под рукой, что делает их удобным инструментом для организации различных аспектов здоровья. Мобильные приложения, помогающие контролировать прием лекарств, позволяют пользователю не запоминать сложные расписания, а получать своевременные напоминания, фиксировать факт приема и видеть историю своих действий. Это повышает ответственность пациента за свое лечение и улучшает взаимодействие с врачом, который может получить достоверные данные о соблюдении назначений.

На рынке существует немало подобных приложений, однако многие из них либо перегружены лишними функциями, либо имеют сложный интерфейс, либо требуют платной подписки для доступа к базовым возможностям. Некоторые решения ориентированы только на определенные страны или языки, другие не предоставляют удобного способа поделиться отчетом с врачом. В связи с этим актуальной является задача создания собственного приложения, которое сочетало бы в себе простоту использования, необходимый минимальный функционал и возможность безопасного хранения данных.

Цель дипломного проектирования – разработать мобильное приложение для отслеживания приема лекарств с функциями напоминаний, журнала приемов и истории. Это повысит приверженность лечению, снизит риск пропуска доз и облегчит контроль врача.

Для реализации поставленной цели необходимо выполнить ряд задач:

- проанализировать мобильные приложения и программные решения для напоминаний и контроля приема лекарств;
- сформировать функциональные требования к мобильному приложению;
- спроектировать архитектуру приложения;
- реализовать мобильное приложение в соответствии с техническими требованиями к проекту;
- провести тестирование мобильного приложения;
- разработать руководство пользователя;
- провести технико-экономическое обоснование проекта.

В процессе разработки планируется использовать фреймворк Flutter и язык Dart. Для управления состоянием приложения выбран паттерн Provider. В качестве облачного решения используются сервисы Firebase: Authentication для регистрации и входа, Cloud Firestore для хранения данных. Для отправки напоминаний используется библиотека flutter_local_notifications.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			Введение	Лит.	Лист	Листов
Пров.		Тимонович Г.Л.				У	1	1
						БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.						
Утв.		Блинова Е.А.						

1 Постановка задачи и аналитический обзор аналогичных решений

Перед созданием собственного мобильного приложения важно проанализировать уже существующие на рынке аналоги. Это позволит понять, какие решения популярны у пользователей, в чем их преимущества и недостатки, а также какие ошибки стоит избегать при разработке. Кроме того, изучение конкурентных продуктов помогает сформулировать требования к функционалу и интерфейсу, опираясь на реальный пользовательский опыт.

В данном разделе будет выполнен обзор существующих мобильных приложений для контроля приема лекарств, после чего сформулированы основные задачи, которые предстоит решить в рамках проекта. Результаты анализа послужат основой для формирования технического задания, учитывающего современные тенденции в области цифрового здравоохранения и необходимость повышения приверженности пациентов к лечению.

1.1 Обзор аналогов мобильного приложения

1.1.1 Аналог «Pill reminder»

Pill reminder [1] – одно из самых простых приложений для напоминания о приеме лекарств. Программа предлагает пользователям базовые функции: создание списка лекарств, установка времени приема, получение уведомлений. Интерфейс отображен на рисунке 1.1 и выполнен в минималистичном стиле, что делает его удобным для широкого круга пользователей.

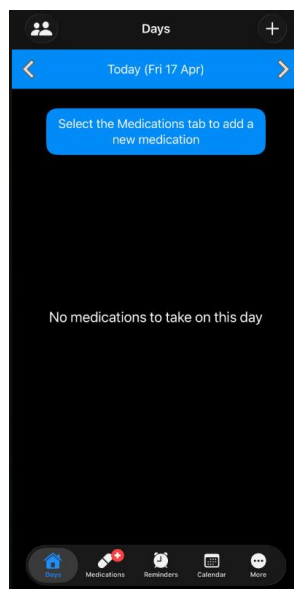


Рисунок 1.1 – Интерфейс приложения «Pill reminder»

					ДП 01.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			1 Постановка задачи и аналитический обзор аналогичных решений	Лит.	Лист	Листов
Пров.		Тимонович Г.Л.				У	1	19
						БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.						
Утв.		Блинова Е.А.						

Процесс добавления нового расписания в Pill reminder представляет собой последовательность действий, последовательно начинающихся с создания лекарства. При нажатии на кнопку Medications в нижнем меню пользователь переходит на страницу медикаментов. Интерфейс основной страницы просмотра всех медикаментов представлен на рисунке 1.2.

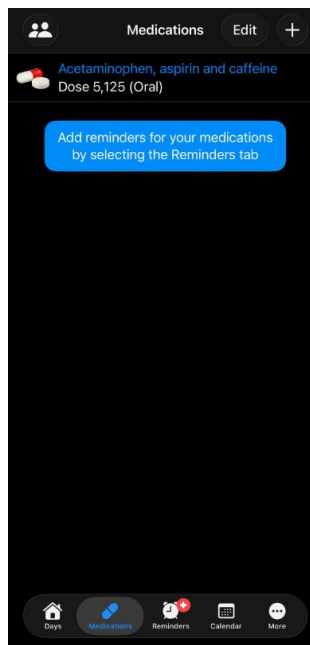


Рисунок 1.2 – Экран просмотра медикаментов

На странице медикаментов можно нажать на «+», что откроет страницу добавления нового лекарства. При добавлении лекарства нужно указать название (полную информацию указывать необязательно) и нажать «Сохранить» в правом верхнем углу. Интерфейс страницы добавления представлен на рисунке 1.3.

Рисунок 1.3 – Экран добавления медикаментов

После создания медикамента приложение подсказывает, что нужно создать расписание. Для этого нужно перейти во вкладку Reminders в которой нужно также добавить расписание. Нажатием на + в верхней части экрана переходим на страницу добавления расписания, которая представлена на рисунке 1.4.

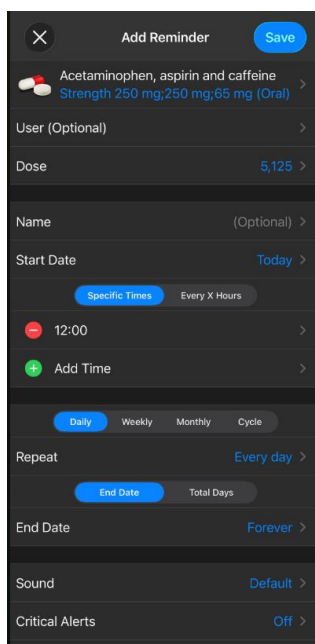


Рисунок 1.4 – Экран создания расписания

На ней также не обязательно указывать все, потому что множество значений стоит по умолчанию, нужно лишь указать само лекарство, которое мы создавали до этого, и выбрать время. После выполненных действий можно увидеть созданное расписание на странице расписаний, которая представлена на рисунке 1.5.

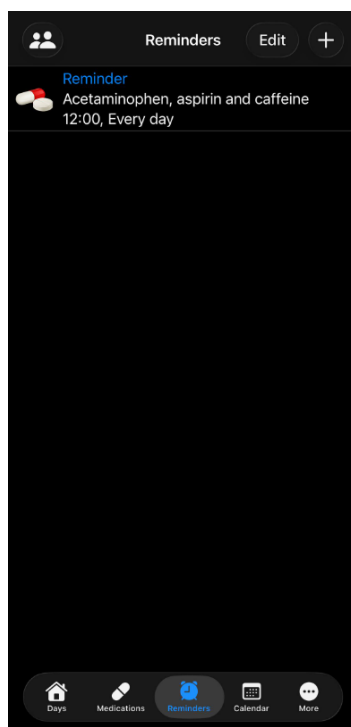


Рисунок 1.5 – Экран просмотра всех расписаний

После всех проделанных действий наше расписание окажется рабочим и будет показано на главной странице. На главной странице сверху указан день, который можно поменять, ниже находятся расписания, установленные для этого дня, а также checkbox который устанавливает прием лекарства. Главную страницу с нашим расписанием можно увидеть на рисунке 1.6.



Рисунок 1.6 – Главный экран

Также наше расписание отображается и в календаре, который можно найти, если нажать в нижнем меню на соответствующую кнопку. В календаре при нажатии на определенную дату нам выведутся все расписания на эту дату, которые можно также отметить, как выполненные. Удобный экран с календарем показан на рисунке 1.7.

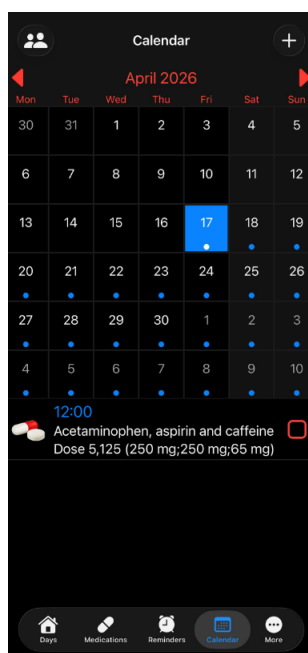


Рисунок 1.7 – Экран календаря

На последней вкладке под названием More находятся различные дополнительные настройки, такие как настройки уведомлений, настройки аккаунтов, полезные советы и подробная информация о приложении. Экран с дополнительной информацией показан на рисунке 1.8.

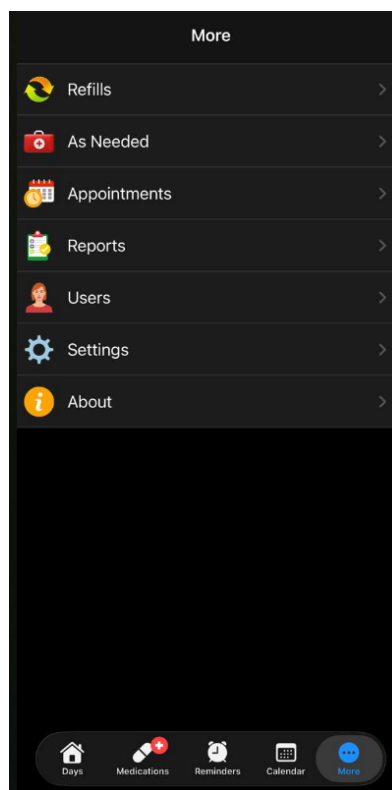


Рисунок 1.8 – Экран дополнительной информации

Анализируя функциональность Pill Reminder, можно выделить как положительные, так и отрицательные стороны. К достоинствам следует отнести простой и понятный интерфейс, который позволяет быстро добавлять лекарства и настраивать напоминания. Приложение поддерживает различные режимы приема: ежедневно, через день, по определенным дням недели или с заданным интервалом. Встроенная система учета остатков помогает пользователю вовремя заметить, что запас препарата заканчивается. Напоминания срабатывают в заданное время, а возможность отметить прием прямо из уведомления делает использование приложения максимально удобным. Кроме того, Pill Reminder предоставляет базовую статистику соблюдения лечения в виде календаря и графиков. В целом, Pill Reminder представляет собой удобное решение для тех, кому нужен простой инструмент для контроля приема лекарств без избыточной функциональности.

1.1.2 Аналог «Medisafe»

Medisafe [2] – это одно из наиболее популярных мобильных приложений для отслеживания приема лекарств, которое подходит как для пациентов с простыми схемами лечения, так и для тех, кто вынужден принимать множество препаратов одновременно. Приложение позволяет создавать гибкие гра-

фики приема с учетом сложных циклов терапии, предоставляет персонализированные напоминания и дает возможность генерировать отчеты о соблюдении лечения для передачи врачу. Кроме того, Medisafe имеет собственную базу лекарств с подсказками при вводе, что ускоряет добавление популярных препаратов. Также в приложении реализован учет остатков медикаментов и напоминания о необходимости пополнения запасов.

Важной особенностью Medisafe является возможность настройки взаимодействия между препаратами: приложение предупреждает о потенциально опасных комбинациях лекарств, что особенно актуально для пациентов, принимающих несколько видов препаратов одновременно. Система также позволяет назначать ответственного за прием (например, родственника или сиделку), который будет получать уведомления в случае пропуска дозы. Такая функция значительно повышает безопасность пациентов с нарушениями памяти или сложными схемами лечения.

Зайдя в приложение, можно увидеть главную страницу с указанием даты и нашими расписаниями лекарств. Главный экран представлен на рисунке 1.9.

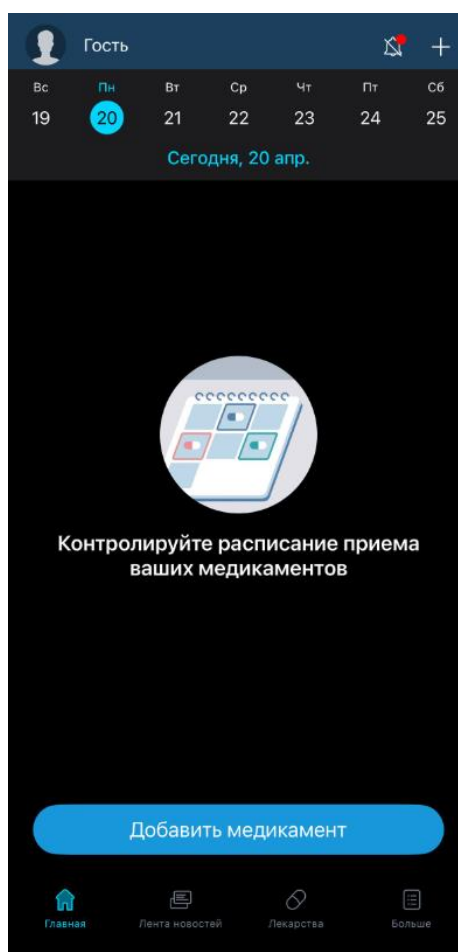


Рисунок 1.9 – Домашняя страница Medisafe

Добавление медикамента начинается с нажатия на кнопку на главной странице. После этого открываются последовательные экраны добавления лекарства. На первом из них мы должны ввести названия медикамента или выбрать его из поиска. Пример показан на рисунке 1.10.

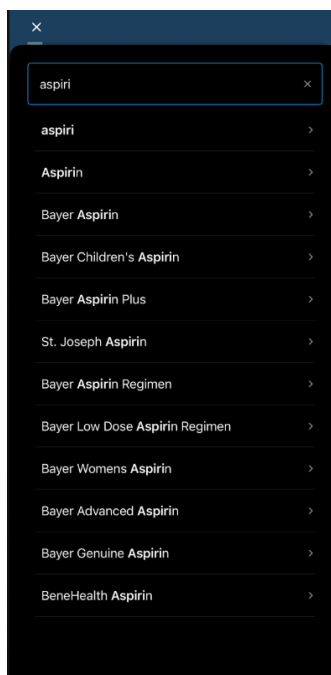


Рисунок 1.10 – Экран поиска препарата

После выбора лекарства пользователь переходит к следующему этапу. На этом этапе нужно выбрать форму выпуска. Если выбирать лекарство из существующих в базе данных, то будут определены все возможные формы выпуска для этого лекарства. В противном случае будут предложены все формы по умолчанию. Также пользователь может написать свою форму выпуска вручную. Экран выбора формы лекарства показан на рисунке 1.11.

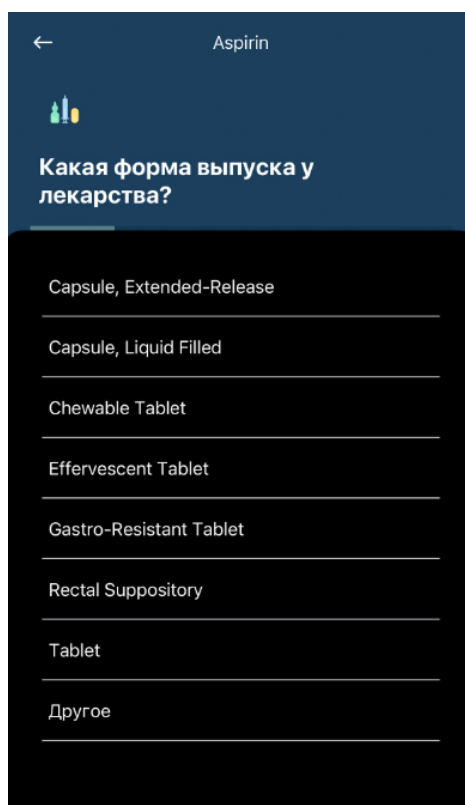


Рисунок 1.11 – Экран выбора формы лекарства

Следующим этапом будет выбор концентрации медикамента, как и при выборах ранее можно выбрать из существующих, а также написать пользователю самому концентрацию медикамента. Если эта опция не особо важна, то можно выбрать «Не знаю» из списка или «Пропустить» сверху слева в углу. Экран выбора концентрации медикамента показан на рисунке 1.12.

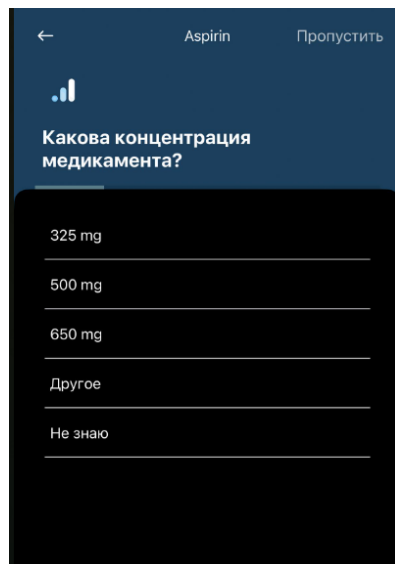


Рисунок 1.12 – Экран выбора концентрации

После этих выборов начинается выбор даты и времени. Пользователю предлагается указать, с какой периодичностью необходимо принимать данное лекарство: каждый день, через день, по определенным дням недели или с индивидуальным интервалом. Для каждого лекарства доступные периодичности могут различаться, поэтому то, что показано на экране, зависит от выбранного ранее препарата. Соответствующий экран выбора периодичности приема лекарств представлен на рисунке 1.13.

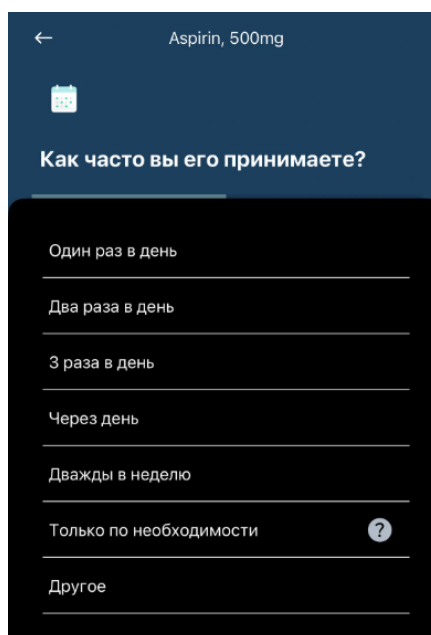


Рисунок 1.13 – Экран выбора частоты приема

После выбора периодичности пользователю показывается экран выбора времени, где можно указать конкретное время каждого приема. На рисунке 1.14 показан экран выбора времени.

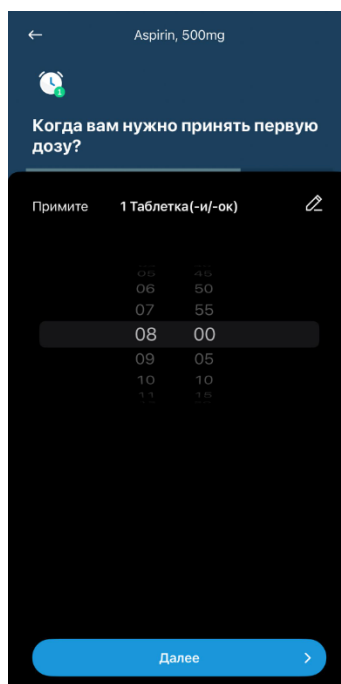


Рисунок 1.14 – Экран выбора время приема

После всех выполненных манипуляций наше лекарство появится на главном экране с расписанием. Его можно отметить, как принятое или пропущенное, а также изменить дату приема при необходимости. На рисунке 1.15 показан главный экран вместе с нашими расписаниями.

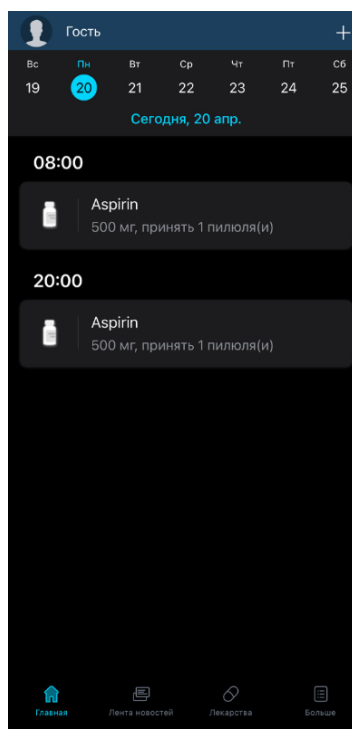


Рисунок 1.15 – Главный экран с расписаниями

Кроме главного экрана в приложении также представлены такие разделы как: «Лента новостей», «Лекарства», «Больше». В ленте новостей указываются доступные новости, связанные с лекарствами, которые пользователь употребляет. Во вкладке «Больше» указана вся дополнительная информация о приложении вместе с настройками. На экране лекарств видны все ваши лекарства, которые вы создавали ранее. На этой вкладке можно изменять старые, а также создавать новые. Экран лекарств представлен на рисунке 1.16.



Рисунок 1.16 – Экран с таблетками

Анализируя функциональность «Pill Reminder», можно выделить как достоинства, так и недостатки. К сильным сторонам приложения относится интуитивно понятный интерфейс, который позволяет пользователю быстро разобраться в функционале без длительного обучения. Процесс добавления нового препарата занимает минимум времени благодаря лаконичной форме ввода, где указываются название, дозировка, время приема и периодичность. Приложение поддерживает основные режимы напоминаний: ежедневно, через день, по выбранным дням недели, что покрывает потребности большинства пациентов. Система учета остатков автоматически отслеживает количество таблеток и при достижении минимального порога уведомляет пользователя о необходимости пополнить запас. Напоминания срабатывают точно в заданное время, а возможность подтвердить прием непосредственно из уведомления экономит время и упрощает взаимодействие с приложением. Также «Pill Reminder» предоставляет наглядную статистику соблюдения лечения в виде календаря и графиков, позволяя пользователю оценить свою приверженность терапии. В целом, «Pill Reminder» представляет собой удобное и доступное решение для контроля приема лекарств, ориентированное на тех, кто ценит простоту и надежность без избыточной функциональности.

1.1.3 Аналог «MyTherapy»

MyTherapy [3] – это популярное мобильное приложение для отслеживания приема лекарств, которое активно используется пациентами как с простыми, так и со сложными схемами лечения. Оно позволяет не только настраивать гибкие графики приема препаратов, но и отслеживать динамику показателей здоровья, таких как давление, уровень сахара, пульс и другие. Приложение поддерживает ведение дневника самочувствия, где пользователь может отмечать симптомы и оценивать свое состояние. Также «MyTherapy» предоставляет возможность формировать отчеты о соблюдении лечения и передавать их лечащему врачу. Кроме того, приложение имеет встроенную базу лекарств с подробной информацией о препаратах и их дозировках.

При первом входе в приложение, нас отправляет в раздел Лечение, который показан на рисунке 1.17.



Рисунок 1.17 – Экран лечения

На странице лечения нам сразу же предлагают добавить первое лекарство. При нажатии на кнопку пользователю показывают всплывающее окно выбора типа напоминания. Раздел «Лекарства» позволяет добавлять препараты в план лечения, настраивать напоминания и отслеживать их прием. В разделе «Измерения» пользователь может регистрировать показатели здоровья, такие как артериальное давление, вес или уровень сахара в крови. Раздел «Активности» предназначен для установки напоминаний о повседневных привычках – ходьбе, питье воды, растяжке и других. Раздел «Настроение и симптомы» дает возможность отслеживать эмоциональное состояние и фиксировать возникающие симптомы. Раздел «Отчеты» позволяет просматривать статистику соблюдения лечения за выбранный период и экспортировать данные для врача. Экран с выбором категорий представлен на рисунке 1.18.



Рисунок 1.18 – Экран выбора категории

После выбора первого пункта пользователю показывают экран добавления лекарства. На нем предлагается заполнить основные поля, необходимые для создания напоминания. Первое поле предназначено для ввода названия препарата. В данном примере пользователь вводит «парацетамол». Второе поле позволяет выбрать единицу измерения дозировки, например «таблетка(и)». Экран добавления нового лекарства представлен на рисунке 1.19.

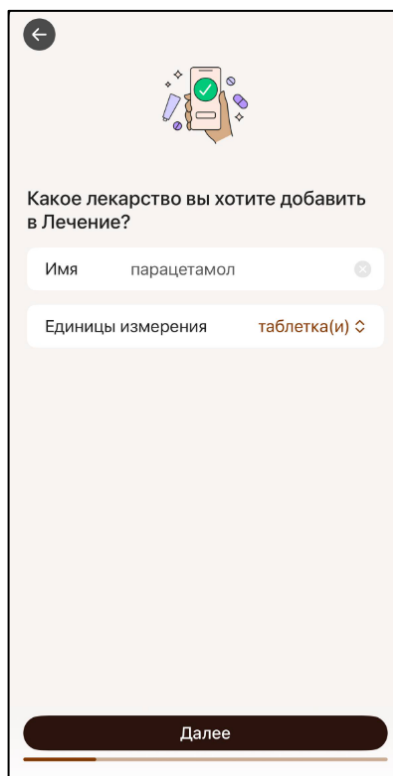


Рисунок 1.19 – Экран добавления имя лекарства

После ввода названия препарата и единиц измерения приложение переходит к настройке частоты приема. Пользователю предлагается выбрать, как часто принимать лекарство. Доступны варианты: «Один раз в день», «Два раза в день», «По мере необходимости», а также пункт «Мне нужно больше вариантов...», который открывает расширенные настройки. Для перехода к следующему шагу предусмотрена кнопка «Далее». Экран выбора частоты приема представлен на рисунке 1.20.

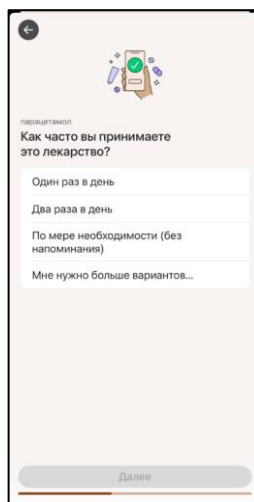


Рисунок 1.20 – Экран частоты выбора лекарства

На следующем экране пользователь выбирает время напоминания. С помощью поля «Час» устанавливается точное время приема, например 08:00. Ниже расположено поле «Дозировка», где указывается количество таблеток на прием. Также на экране присутствует переключатель «Разрешить важные предупреждения». При его активации пользователь будет получать напоминания даже в режиме «Без звука» или «Не беспокоить». Экран настройки времени и дозировки представлен на рисунке 1.21.

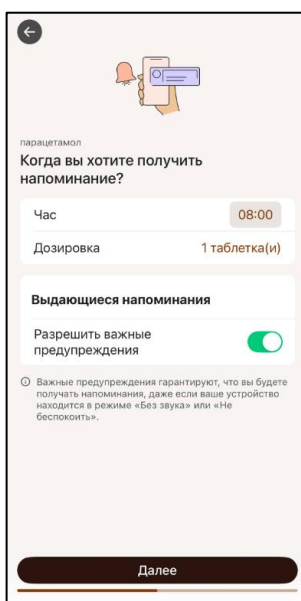


Рисунок 1.21 – Экран настройки времени

На заключительном экране пользователю предлагается настроить напоминания о пополнении запаса. Для этого нужно активировать переключатель «Получать напоминания». Затем указывается текущее количество препарата и пороговое значение «Напомни, когда осталось». После заполнения параметров пользователь нажимает кнопку «Сохранить». Экран настройки учета остатков представлен на рисунке 1.22.

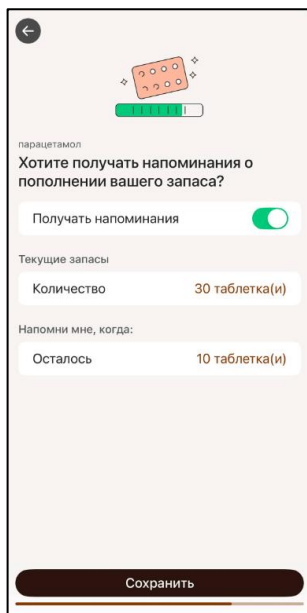


Рисунок 1.22 – Экран настройки пополнения запаса

После сохранения основных параметров приложение может запросить цель приема препарата. Пользователю предлагается указать заболевание через поле «Поиск болезни». Эта информация поможет персонализировать поддержку. На экране есть кнопки «Пропустить» и «Добавить». Экран указания цели приема представлен на рисунке 1.23.

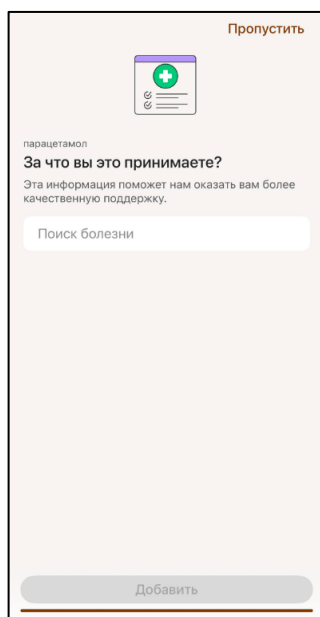


Рисунок 1.23 – Экран дополнительной информации

В конце у нас создается расписание пользователя. После этого главный экран приложения отображает список задач, запланированных на текущий день. В верхней части экрана указан заголовок «Сегодня». Ниже представлена карточка с предстоящим приемом лекарства: указано время (08:00), название препарата (парацетамол) и дозировка (1 таблетка(и)). В нижней части экрана расположена панель навигации, состоящая из четырех основных разделов: «Сегодня», «Статистика», «Поддержка» и «Лечение». Это позволяет пользователю быстро переключаться между просмотром дневного расписания, анализом прогресса, обращением за помощью и управлением списком лекарств. Главный экран приложения представлен на рисунке 1.24

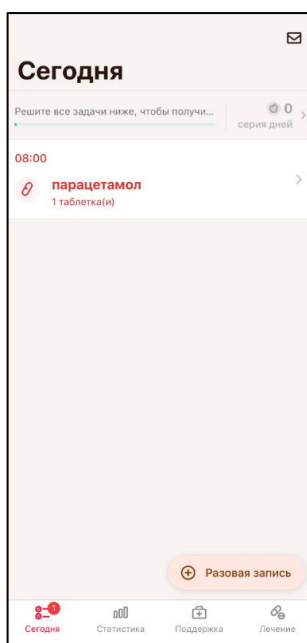


Рисунок 1.24 – Главный экран приложения

Анализируя функциональность myTherapy, можно выделить как достоинства, так и недостатки. К сильным сторонам приложения относится комплексный подход к контролю здоровья: пользователь может не только отслеживать прием лекарств, но и фиксировать различные показатели состояния организма. Процесс добавления препарата сопровождается возможностью указать не только дозировку и время приема, но и форму выпуска (таблетки, капсулы, инъекции и т.д.), что делает настройку более гибкой. Приложение поддерживает сложные схемы лечения, включая прием «по необходимости» и постепенное изменение дозировки. Система учета остатков позволяет контролировать запас лекарств и получать своевременные напоминания о пополнении. Кроме того, myTherapy предоставляет возможность отслеживать динамику показателей здоровья – артериального давления, уровня сахара, веса, пульса и других, а также вести дневник настроения и симптомов. Все собранные данные могут быть сформированы в виде отчета для передачи лечащему врачу. В целом, myTherapy представляет собой многофункциональное решение для пациентов с хроническими заболеваниями, которым требуется не только контроль приема лекарств, но и комплексное наблюдение за состоянием здоровья.

1.2 Постановка задач мобильного приложения

Целью данного дипломного проекта является создание мобильного приложения для контроля приема лекарственных средств, которое обеспечит пользователям удобный инструмент для своевременного приема назначенных препаратов, а также предоставит доступ к справочной информации о лекарствах. Для достижения этой цели необходимо решить ряд взаимосвязанных задач, каждая из которых охватывает определенную функциональную область. Ниже представлено подробное описание каждой задачи.

1.2.1 Реализация системы авторизации и аутентификации

Первой задачей является создание механизма регистрации и входа в приложение. Это позволит пользователю сохранять данные о своем расписании приема лекарств и получать к ним доступ после повторной установки приложения или смены устройства. Для решения этой задачи интегрируется Firebase Authentication, обеспечивающий аутентификацию по электронной почте и паролю. После успешного входа пользователь получает доступ к своему профилю, где хранятся все добавленные препараты и настройки напоминаний.

1.2.2 Создание расписания приема лекарств

Вторая задача заключается в разработке функционала добавления и управления лекарственными препаратами. При создании расписания пользователю предоставляется два варианта: выбрать уже существующий препарат из общей базы или добавить собственный.

Общая база содержит список лекарственных средств, доступных всем пользователям приложения. При выборе препарата из общего списка автоматически подставляются основные сведения о нем, такие как название и дозировка. Пользователь также может просмотреть инструкцию к препарату прямо в приложении. Однако дозировку и время приема пользователь может указать самостоятельно или оставить предложенные по умолчанию в любом случае, независимо от того, выбран препарат из общей базы или создается новый.

Если пользователь не нашел нужное лекарство в общей базе, он может создать собственный препарат. В этом случае необходимо указать название, дозировку и другую информацию по аналогии с готовыми лекарствами. Такой препарат можно редактировать позже. Добавленный пользователем препарат сохраняется в его личном профиле и виден только ему.

После заполнения всех полей приложение сохраняет информацию о препарате в облачной базе данных Cloud Firestore, привязывая ее к конкретному пользователю. Все данные синхронизируются между устройствами пользователя, что позволяет продолжить работу на другом телефоне без потери информации. На основе сохраненного расписания формируются записи о запланированных приемах, которые отображаются на главном экране. Пользователь может в любой момент отредактировать или удалить созданное им расписание.

1.2.3 Разработка системы уведомлений о приеме лекарств

Третья задача посвящена созданию системы напоминаний, которая будет своевременно оповещать пользователя о необходимости принять лекарство. Для этого используется библиотека `flutter_local_notifications`, позволяющая планировать уведомления в заданное время. Данная библиотека интегрируется с системными сервисами уведомлений Android, что обеспечивает надежную доставку напоминаний независимо от состояния приложения.

При создании или редактировании расписания приложение рассчитывает время следующего приема и передает его в планировщик уведомлений. Планировщик, в свою очередь, регистрирует задачу в операционной системе. Благодаря этому даже после полного закрытия приложения пользователем (свайпом из списка активных приложений) запланированное уведомление все равно сработает в нужный момент. Это достигается за счет того, что `flutter_local_notifications` использует механизмы Android, которые работают на уровне операционной системы и не зависят от запущенного приложения.

Если устройство выключено в момент запланированного приема, уведомление не сработает. Однако при включении телефона система проверяет пропущенные события и, в зависимости от настроек, может отправить уведомление о пропущенном приеме. Для более надежной работы приложение может при запуске проверять текущее расписание и при необходимости создавать пропущенные уведомления.

В случае, когда приложение работает в фоновом режиме или свернуто, уведомление все равно отображается на экране устройства. Пользователь может взаимодействовать с ним, не открывая приложение: отметить прием как выполненный или отложить напоминание. При нажатии на уведомление приложение открывается на соответствующем экране, позволяя подтвердить или скорректировать информацию о приеме.

Таким образом, разработанная система уведомлений гарантирует, что пользователь получит напоминание о приеме лекарства в большинстве ситуаций: при открытом приложении, свернутом, закрытом, а также при работающем устройстве в режиме энергосбережения. При выключенном устройстве напоминание будет доставлено после его включения.

1.2.4 Просмотр справочной информации о лекарствах

Четвертая задача направлена на предоставление пользователю доступа к справочной информации о лекарственных средствах. В приложении реализована возможность просмотра листов-вкладышей, то есть инструкций по медицинскому применению, для доступных препаратов.

Все данные о лекарствах, включая названия, описания, показания, противопоказания, способ применения, дозировки и побочные эффекты, собираются с внешних сайтов-источников (например, официальных государственных реестров лекарственных средств, фармакологических справочников или сайтов производителей) с помощью парсинга. Собранная информация струк-

турируется и сохраняется в облачной базе данных Cloud Firestore. Такой подход позволяет поддерживать актуальность данных и оперативно обновлять информацию при изменении инструкций.

Пользователь может выбрать интересующий его препарат из списка и ознакомиться с краткой версией инструкции непосредственно в приложении. Однако для получения полной и наиболее достоверной информации в интерфейсе предусмотрена ссылка на оригинальный источник (сайт, с которого были взяты данные). При нажатии на ссылку открывается веб-страница с полной инструкцией по медицинскому применению. Это гарантирует, что пользователь имеет доступ к самой свежей и официальной версии документа, а также может при необходимости ознакомиться с дополнительными сведениями, которые не вошли в краткое описание.

Таким образом, данная функция позволяет пользователю быстро получать основную информацию о принимаемых препаратах непосредственно в приложении или перейти к полной инструкции на сайте-источнике. Это повышает удобство использования приложения и помогает пользователю быть информированным о свойствах и правилах применения лекарственных средств.

1.2.5 Учет остатков лекарственных средств (аптечка)

Пятая задача заключается в реализации функционала учета остатков лекарственных средств. Пользователь может добавлять лекарства в аптечку, указывая начальное количество и единицу измерения (таблетка, капсула, мл и т.д.). При каждом отмеченном приеме количество автоматически уменьшается на дозировку, указанную для данного препарата. Если остаток падает до нуля или ниже установленного порога, система уведомляет пользователя о необходимости пополнения. Пользователь также может вручную пополнить запас или скорректировать количество. Для удобства на экране аптечки отображается визуальный индикатор состояния запаса.

1.2.6 Визуализация статистики и формирование отчетов

Шестая задача заключается в разработке экрана статистики, который позволяет пользователю анализировать соблюдение режима лечения. Пользователь может выбрать период (неделя, месяц, произвольный интервал) и получить наглядную информацию: общий процент выполненных приемов, количество принятых и пропущенных доз, динамику выполнения по дням в виде столбчатой диаграммы, а также детальную статистику по каждому лекарству. Экран также отображает среднюю приверженность лечению за выбранный период. Данные обновляются при каждом новом приеме или пропуске. Кроме того, пользователь может поделиться отчетом через мессенджеры или электронную почту сразу после его генерации. Для взаимодействия с лечащим врачом реализована функция экспорта отчета в формате PDF. Отчет включает все ключевые показатели, таблицы по дням и лекарствам, а также дату генерации, что делает его пригодным для распечатки или отправки по электронной почте.

1.2.7 Разработка интуитивно понятного интерфейса

Седьмая задача заключается в создании понятного интерфейса, который позволит пользователю легко ориентироваться в приложении. Для реализации интерфейса используется фреймворк Flutter, предоставляющий готовые виджеты Material Design. Основная навигация строится на меню бургере с несколькими вкладками: расписание на сегодня, список всех лекарств, справочная информация. Процесс добавления нового препарата реализован в виде последовательных экранов, где пользователь шаг за шагом указывает название, дозировку, время и периодичность приема. Все элементы управления выполняются крупными и контрастными для удобства использования.

1.3 Выводы по разделу

В первом разделе проведен анализ существующих мобильных приложений для контроля приема лекарств и сформулирован комплекс задач, необходимых для реализации полноценного продукта. Рассмотрены аналоги – Medisafe, myTherapy и Pill Reminder – представляющие разные подходы: от многофункциональных платформ до минималистичных решений. Каждое из приложений было проанализировано с точки зрения удобства, функциональности, простоты и надежности.

Medisafe предлагает гибкие настройки и экспорт отчетов, но страдает от перегруженности и рекламы. myTherapy обеспечивает комплексный контроль здоровья, однако излишне сложен для пользователей, которым нужны только напоминания. Pill Reminder отличается простотой, но не дает достаточно справочной информации. Таким образом, у каждого аналога есть сильные и слабые стороны, которые были учтены при проектировании.

На основе анализа сформулированы требования к собственной разработке. Ключевое направление – создание приложения, сочетающего простоту с полезными справочными функциями. В отличие от аналогов, разрабатываемое приложение предоставляет доступ к листам-вкладышам и ссылкам на покупку препаратов. Это позволяет пользователю получать всю необходимую информацию без перехода в сторонние сервисы.

Требуется добавление учета остатков (аптечка) с уведомлениями о пополнении. Реализование модуля статистики с динамикой выполнения приемов и экспортом отчетов в PDF. Поставленные задачи: авторизация, гибкое расписание (выбор из базы или добавление своего), надежные уведомления, справочная информация. Технологии: Flutter, Dart, Firebase Authentication, Cloud Firestore, flutter_local_notifications.

Таким образом, аналитический обзор аналогов позволил избежать типичных ошибок и сформировать техническое задание, ориентированное на реальные потребности пользователей. Поставленные задачи создают основу для разработки качественного, надежного и удобного мобильного приложения, которое поможет пользователям своевременно принимать лекарства и получать актуальную информацию о препаратах.

2 Проектирование мобильного приложения

В данной главе будут рассмотрены средства разработки приложения, выбранные технологии и архитектура приложения.

2.1 Обзор средств разработки

От средств разработки зависит большая часть успеха проекта, правильно выбранные платформы и фреймворки позволяют снизить вероятность не удачной разработки приложения, а также позволит ускорить время разработки.

2.1.1 Платформа Android

Для реализации данного мобильного приложения была выбрана платформа Android. Android [4] – это операционная система с открытым исходным кодом, основанная на ядре Linux и предназначенная для широкого спектра устройств: смартфонов, планшетов, носимых устройств, телевизоров и автомобильных мультимедийных систем. Платформа предоставляет разработчикам полный набор инструментов и API для создания высокопроизводительных, безопасных и удобных приложений, распространяемых через магазин Google Play и другие каналы.

Выбор платформы Android обусловлен ее популярностью и доступностью для широкого круга пользователей. Устройства на базе Android занимают значительную долю рынка мобильных устройств, что обеспечивает потенциально большую аудиторию для разрабатываемого приложения. Кроме того, Android предоставляет гибкие возможности для работы с уведомлениями, фоновыми задачами и хранением данных, что важно для приложения, связанного с контролем приема лекарств.

Несмотря на то, что само приложение разрабатывается с использованием кроссплатформенного фреймворка Flutter, конечная сборка ориентирована на операционную систему Android. Flutter компилируется в нативный код, что позволяет приложению работать непосредственно с Android SDK и использовать все возможности платформы: системные уведомления через flutter_local_notifications, аутентификацию и облачное хранение через Firebase сервисы, а также работу в фоновом режиме.

Таким образом, платформа Android предоставляет все необходимые возможности для реализации функционала разрабатываемого приложения, а использование Flutter позволяет эффективно работать с Android SDK, создавая современный и производительный интерфейс.

					ДП 02.00.ПЗ				
		ФИО	Подпись	Дата					
Разраб.		Осадчий П.А.			2 Проектирование мобильного приложения		Лит.	Лист	Листов
Пров.		Тимонович Г.Л.					У	1	22
							БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.							
Утв.		Блинова Е.А.							

2.1.2 Язык программирования Dart

Для реализации мобильного приложения выбран язык программирования Dart. Dart [5] – это современный статически типизированный язык, разработанный Google и используемый в качестве основного языка для фреймворка Flutter. Он сочетает привычный синтаксис, напоминающий Java и JavaScript, высокую производительность и поддержку как JIT-компиляции для быстрой разработки, так и AOT-компиляции для создания нативного кода.

Dart предоставляет ряд важных возможностей. Нулевая безопасность (null safety) исключает ошибки с нулевыми ссылками на этапе компиляции, что особенно важно для медицинского приложения. Асинхронное программирование через Future, Stream, async и await упрощает работу с сетевыми запросами к Firebase и уведомлениями. Благодаря AOT-компиляции Dart-код преобразуется в нативный машинный код для Android, обеспечивая высокую производительность, а JIT-компиляция в процессе разработки поддерживает горячую перезагрузку (hot reload). Dart тесно интегрирован с Flutter, который предоставляет богатый набор виджетов для создания интерфейса.

Таким образом, выбор Dart обусловлен его надежностью, производительностью и совместимостью с экосистемой Flutter и Firebase.

2.1.3 Среда разработки

В качестве среды разработки выбрана Android Studio [6] – официальная интегрированная среда (IDE) для платформы Android, предоставляемая компанией Google. Android Studio включает редактор исходного кода с поддержкой интеллектуального автодополнения, анализа кода и безопасного рефакторинга. Встроенный отладчик позволяет выполнять отладку на уровне исходного кода, а также предоставляет профилировщики производительности, памяти и энергопотребления.

Несмотря на то, что Android Studio позиционируется как среда для нативной Android-разработки, она также имеет полноценную поддержку Flutter и Dart благодаря официальному плагину от Google. Этот плагин добавляет возможность создания новых Flutter-проектов, запуска и отладки приложений на эмуляторах и реальных устройствах, а также включает инструмент горячей перезагрузки (hot reload), позволяющий мгновенно видеть изменения в интерфейсе без полной перекомпиляции приложения.

Среда содержит ряд инструментов, упрощающих разработку Flutter-приложений. Встроенный редактор кода поддерживает автодополнение виджетов Flutter и навигацию по коду. Палитра виджетов (Flutter Inspector) позволяет визуально исследовать иерархию элементов интерфейса в реальном времени, что особенно полезно при отладке сложных экранов. Также доступен просмотр состояния приложения и анализ производительности рендеринга.

Дополнительно Android Studio включает инструменты для работы с Firebase: плагин Firebase Assistant помогает быстро подключить сервисы Authentication и Cloud Firestore к проекту, автоматически добавляя необходи-

мые зависимости и конфигурационные файлы. Для анализа сетевого трафика используется Network Inspector, а для оценки влияния приложения на заряд батареи – Energy Profiler. Хотя Database Inspector предназначен для локальных баз данных SQLite (например, Room), в данном проекте он не используется, так как все данные хранятся в облачной базе Cloud Firestore.

Android Studio поддерживает расширение функциональности через плагины, что позволяет интегрировать системы контроля версий (Git, GitHub), инструменты статического анализа кода и средства автоматизированного тестирования. Встроенная интеграция с Git обеспечивает удобную работу с репозиториями. Выбор Android Studio обусловлен тем, что она является стандартом для разработки под Android, имеет официальную поддержку Flutter от Google и содержит все необходимые инструменты для создания, отладки, тестирования и профилирования приложений.

2.1.4 Фреймворк Flutter

Для построения пользовательского интерфейса используется фреймворк Flutter. Flutter [7] – это современный UI Toolkit от Google, позволяющий создавать нативные приложения для Android с использованием единой кодовой базы. В отличие от традиционного подхода с XML-разметкой, Flutter использует декларативный подход, где интерфейс описывается на языке Dart в виде виджетов, реагирующих на изменение состояния приложения.

Декларативная модель означает, что интерфейс описывается как функция от текущего состояния: при изменении данных Flutter автоматически перерисовывает только те компоненты, которые затронуты изменениями. Это снижает количество ошибок, связанных с ручным обновлением UI, и улучшает производительность. Фреймворк тесно интегрирован с языком Dart, что позволяет легко реализовать реактивное связывание данных.

Особое значение для данного проекта имеет адаптивность и доступность интерфейса. Flutter из коробки поддерживает адаптацию под различные размеры экранов и ориентации. Встроенная поддержка Material Design обеспечивает современный внешний вид и соответствие рекомендациям Google по проектированию интерфейсов для Android. Кроме того, функция горячей перезагрузки (hot reload) позволяет мгновенно видеть изменения в интерфейсе без полной перекомпиляции приложения, что значительно ускоряет процесс разработки.

2.1.5 Управление состоянием (Provider)

Для реализации устойчивой и масштабируемой архитектуры приложения используется паттерн управления состоянием Provider [8]. Provider является рекомендуемым решением для Flutter-приложений, так как он прост в использовании, эффективен и тесно интегрирован с экосистемой Flutter.

Provider выступает связующим звеном между пользовательским интерфейсом и бизнес-логикой. Он позволяет передавать данные между виджетами без необходимости явной передачи через конструкторы, что упрощает код и

делает его более поддерживаемым. Provider также обеспечивает реактивное обновление интерфейса при изменении данных: при обновлении модели все виджеты, которые подписаны на изменения, автоматически обновляются.

В разрабатываемом приложении Provider используется для управления состоянием на каждом экране (список препаратов, добавление лекарства, расписание). Это обеспечивает четкое разделение ответственности: интерфейс только отображает данные и отправляет действия, а бизнес-логика обрабатывается в отдельных классах, что упрощает тестирование и поддержку кода.

2.1.6 Облачная база данных Cloud Firestore

Для организации хранения данных и синхронизации между устройствами используется облачный сервис Cloud Firestore [9], входящий в состав платформы Firebase. Cloud Firestore представляет собой документо-ориентированную NoSQL базу данных, которая обеспечивает гибкость структуры данных и высокую скорость чтения и записи.

В отличие от традиционных реляционных баз данных, Cloud Firestore не требует предварительного определения схемы. Данные организованы в виде коллекций, каждая из которых содержит документы с полями различных типов. Для разрабатываемого приложения созданы коллекции для хранения информации о пользователях, лекарствах и расписаниях приема.

Каждый пользователь имеет доступ только к своим данным благодаря правилам безопасности, определяемым на стороне сервера. При добавлении препарата или отметке приема данные сохраняются в Cloud Firestore, что обеспечивает возможность восстановления при смене устройства.

2.1.7 Аутентификация Firebase Authentication

Для обеспечения безопасной регистрации и входа пользователей используется сервис Firebase Authentication [10]. Firebase Authentication предоставляет безопасный способ управления учетными записями пользователей.

Сервис поддерживает различные способы аутентификации. В разрабатываемом приложении реализована аутентификация по электронной почте и паролю, что позволяет пользователям создавать собственные профили и восстанавливать доступ при необходимости. Firebase Authentication берет на себя все аспекты безопасности: хранение паролей в зашифрованном виде, защиту от атак перебором, управление сессиями.

После успешной аутентификации приложение получает уникальный идентификатор пользователя (UID), который используется для привязки всех данных к конкретной учетной записи в Cloud Firestore. Это гарантирует, что каждый пользователь работает только со своими данными.

Процесс регистрации и сохранения данных организован следующим образом. При создании новой учетной записи пользователь вводит адрес электронной почты, пароль. После успешной регистрации в Firebase Authentication приложение автоматически создает соответствующий документ в коллекции

пользователей Cloud Firestore. В этом документе сохраняются следующие данные: уникальный идентификатор пользователя, адрес электронной почты, дата регистрации, а также пустые коллекции для последующего хранения лекарств, расписаний и количества остатков в аптечке.

При входе зарегистрированного пользователя приложение выполняет аутентификацию через Firebase Authentication. После подтверждения правильности введенных данных приложение получает UID и загружает из Cloud Firestore профиль пользователя вместе со всеми связанными данными – списком добавленных препаратов и настройками расписаний.

В процессе работы с приложением все действия пользователя (добавление нового лекарства, создание расписания, отметка о приеме) сохраняются в Cloud Firestore и привязываются к его UID. Структура базы данных организована таким образом, что каждый пользователь имеет собственные подколлекции, что исключает возможность несанкционированного доступа к чужим данным. Например, при добавлении препарата документ сохраняется в коллекцию `medications`, где одним из полей является `userId`, соответствующий уникальному идентификатор текущего пользователя. При запросе списка лекарств приложение загружает только те документы, у которых `userId` совпадает с уникальным идентификатором авторизованного пользователя.

Такой подход обеспечивает полную изоляцию данных между разными пользователями, позволяет восстанавливать данные после переустановки приложения, а также гарантирует, что каждый пользователь работает исключительно со своим набором лекарств и расписаний.

2.1.8 Система локальных уведомлений (`flutter_local_notifications`)

Своевременное и надежное напоминание о приеме лекарств является ключевой функцией разрабатываемого приложения. Для реализации уведомлений используется библиотека `flutter_local_notifications` [11], которая предоставляет удобный API для работы с системными уведомлениями на Android.

Библиотека `flutter_local_notifications` взаимодействует с системным сервисом `NotificationManager`, позволяя планировать и отображать локальные уведомления. Для точного срабатывания в заданное время используются механизмы `AlarmManager`, которые работают на уровне операционной системы и не зависят от того, запущено ли приложение в данный момент.

При добавлении нового препарата приложение рассчитывает время следующего приема и передает его в планировщик уведомлений. Планировщик регистрирует задачу в операционной системе. Благодаря этому, даже после закрытия приложения, уведомление все равно сработает в нужный момент.

При срабатывании уведомления пользователь видит системное сообщение с информацией о препарате и времени приема. При нажатии на уведомление приложение открывается на соответствующем экране, позволяя подтвердить прием. Уведомления работают даже в режиме энергосбережения.

Для восстановления уведомлений после перезагрузки устройства приложение при запуске проверяет текущее расписание и при необходимости пере-

планирует пропущенные напоминания. Это обеспечивает непрерывность работы системы независимо от состояния устройства.

2.1.9 Получение справочной информации о лекарствах

Для предоставления пользователю доступа к инструкциям по медицинскому применению в приложении реализован функционал получения справочной информации о лекарствах. Данные о препаратах (названия, описания, показания к применению, противопоказания, способ применения и дозы, побочные эффекты, особые указания) хранятся в базе данных.

Сбор и структурирование справочной информации осуществляется заранее, на этапе наполнения базы данных. Для этого используются официальные источники по типу государственных реестров лекарственных средств. Собранные данные проходят проверку на актуальность и корректность перед добавлением в приложение. Такой подход гарантирует, что пользователь получает достоверную информацию о каждом препарате без необходимости самостоятельно проверять ее в сторонних ресурсах.

При открытии приложения пользователь может перейти в раздел со списком доступных лекарственных средств. Список загружается из Cloud Firestore и отображается с поиском по названию и фильтрацией по категориям. Выбрав интересующий препарат, пользователь может ознакомиться с краткой версией инструкции, которая содержит основные сведения о применении, противопоказаниях и побочных эффектах.

Для получения полной и наиболее достоверной информации в интерфейсе предусмотрена ссылка на оригинальный источник. При нажатии открывается веб-браузер с полной инструкцией. Это гарантирует доступ к самой свежей и официальной версии документа.

Такой подход позволяет поддерживать актуальность данных без ручного обновления. Пользователю не нужно искать инструкции в интернете – вся необходимая информация собрана в одном месте и доступна в любой момент.

2.2 Архитектура программного продукта

Архитектура является основной частью при проектировании программного обеспечения, именно она, в большей части, определит какие нагрузки сможет выдерживать система, время безотказной работы. Диаграмма развертывания приложения показана в приложении А.

Приложение построено на архитектуре, сочетающей в себе разделение ответственности между слоями пользовательского интерфейса, бизнес-логики и данных. Такой подход обеспечивает четкое разделение ответственности, что позволяет легко тестировать компоненты и поддерживать код при расширении функциональности. В качестве языка разработки выбран Dart, который благодаря нулевой безопасности и поддержке асинхронного программирования повышает надежность и ускоряет разработку. Кроме того, использование Dart позволяет создавать производительный и безопасный код.

Для реализации пользовательского интерфейса используется фреймворк Flutter. Flutter позволяет описывать интерфейс в виде виджетов, реагирующих на изменение состояния, что упрощает создание адаптивных экранов, учитывающих различные размеры устройств и настройки доступности (крупный шрифт, высокая контрастность). Взаимодействие между интерфейсом и бизнес-логикой осуществляется через паттерн управления состоянием Provider, который хранит состояние экрана, обрабатывает пользовательские события и взаимодействует со слоем сервисов. Для передачи данных от провайдеров к виджетам используются встроенные механизмы Flutter (Consumer, Selector), обеспечивающие реактивную связь и обновление интерфейса только при изменении соответствующих данных.

Бизнес-логика приложения инкапсулирована в сервисах и репозиториях, которые содержат конкретные сценарии использования: добавление препарата, создание расписания приема, отметка о приеме, получение справочной информации о лекарствах и планирование уведомлений. Этот слой не зависит от фреймворков пользовательского интерфейса.

Слой данных реализован с использованием репозитория, которые абстрагируют источники данных. Для хранения данных используется облачная база данных Cloud Firestore, которая предоставляет гибкую документо-ориентированную структуру. В базе данных хранятся информация о пользователях, лекарственных препаратах, расписаниях приема, а также справочная информация о лекарствах.

Для аутентификации пользователей используются облачные сервисы Firebase. Firebase Authentication обеспечивает регистрацию и вход пользователей (по электронной почте и паролю), а также управление сессиями. Cloud Firestore выступает в роли основной базы данных, где хранятся все данные, привязанные к конкретному пользователю через его уникальный идентификатор.

Система уведомлений о приеме лекарств реализована на базе библиотеки flutter_local_notifications. При добавлении нового расписания приложение планирует локальные уведомления через системный NotificationManager. Благодаря использованию AlarmManager уведомления срабатывают даже при закрытом приложении. После перезагрузки устройства приложение при запуске восстанавливает запланированные напоминания на основе данных из Cloud Firestore.

Для получения справочной информации о лекарствах приложение обращается к Cloud Firestore, где хранятся собранные с внешних сайтов данные. При необходимости пользователь может перейти по ссылке на оригинальный источник для ознакомления с полной инструкцией.

Таким образом, архитектура, построенная на Flutter, Provider, Cloud Firestore и Firebase Authentication, обеспечивает высокую надежность, производительность и удобство сопровождения приложения. Пользователи могут эффективно управлять своими курсами лечения, получать своевременные напоминания, просматривать инструкции к лекарствам, пользуясь интуитивно понятным интерфейсом.

Дополнительно в приложении реализован виджет рабочего стола, который отображает список сегодняшних приемов лекарств на главном экране устройства. Виджет работает независимо от основного приложения: он самостоятельно считывает данные из SharedPreferences, куда приложение сохраняет актуальную информацию о расписаниях. Обновление виджета происходит автоматически каждую минуту через системный AlarmManager, а при изменении данных (добавление, редактирование или удаление расписания) приложение отправляет команду на принудительное обновление.

2.3 Описание работы мобильного приложения

При запуске мобильного приложения, построенного на основе Flutter, Provider и Firebase, система инициализирует подключение к облачным сервисам Firebase, настраивает модуль уведомлений и проверяет наличие сохраненной сессии пользователя. При первом запуске пользователь автоматически попадает в гостевой режим – приложение выполняет анонимную аутентификацию через Firebase Authentication, создавая временного пользователя. В гостевом режиме доступны основные функции: просмотр расписания приема лекарств, добавление расписания, отметка выполненных приемов и получение уведомлений. Однако синхронизация данных между устройствами, использование аптечки, просмотр статистики и экспорт отчетов в гостевом режиме ограничены. Для получения полного доступа ко всем функциям приложения пользователю предлагается зарегистрироваться или войти в существующий аккаунт. При нажатии на кнопку «Войти» в верхней панели приложения открывается экран авторизации. После успешной аутентификации (по email/паролю или через Google) Firebase Authentication выдает уникальный идентификатор пользователя (UID), который используется для привязки всех данных к конкретному пользователю в Cloud Firestore. При этом данные, добавленные в гостевом режиме, автоматически объединяются с данными зарегистрированного аккаунта, что позволяет пользователю не терять информацию при переходе от гостевого режима к полноценной учетной записи.

Интерфейс приложения, реализованный с помощью фреймворка Flutter, загружается на экране устройства. Flutter позволяет создавать адаптивные экраны, которые мгновенно реагируют на изменения состояния, обеспечивая плавный пользовательский опыт. Основной экран отображает список запланированных на сегодня препаратов с указанием названия, дозировки и времени приема. Навигация между экранами (список лекарств, добавление препарата, расписание) осуществляется с помощью встроенных средств Flutter Navigator.

Когда пользователь решает добавить новый препарат, он открывает экран создания, где заполняет форму. Пользователь вводит название препарата, указывает дозировку, время приема и периодичность (ежедневно, через день, по определенным дням недели). При этом пользователь может выбрать препарат из общей базы (существующие лекарства, доступные всем пользователям) или создать собственный препарат, указав все данные вручную. Созданный пользователем препарат будет виден только ему.

После ввода данных приложение (через Provider) собирает информацию в объект модели. Provider вызывает соответствующий сервис (например, MedicationService), который реализует бизнес-логику добавления препарата. Сервис, в свою очередь, обращается к Cloud Firestore и сохраняет данные о препарате в коллекцию medications, привязывая их к уникальному идентификатору текущего пользователя. При успешном сохранении данные синхронизируются с облаком, что позволяет при следующем входе на другом устройстве получить актуальный список.

Одновременно с сохранением данных запускается планировщик уведомлений. Система на основе добавленного расписания рассчитывает ближайшие даты и время приема и регистрирует их с помощью библиотеки flutter_local_notifications, которая взаимодействует с системным NotificationManager. Все запланированные напоминания привязаны к идентификатору препарата и могут быть восстановлены после перезагрузки устройства – приложение при запуске проверяет текущее расписание и перепланирует активные напоминания.

В момент наступления времени приема срабатывает системное уведомление. Пользователь видит сообщение с названием препарата, дозировкой и временем приема. При нажатии на уведомление приложение открывается на экране с соответствующим препаратом, где пользователь может открыть приложение и подтвердить прием. При отметке о приеме сервис обновляет статус в Cloud Firestore, фиксируя фактическое время приема.

В случае, если пользователь хочет отредактировать или удалить существующий препарат, процесс аналогичен: интерфейс отправляет изменения в Provider, сервис выполняет валидацию, обновляет или удаляет данные в Cloud Firestore, после чего планировщик уведомлений пересоздает расписание напоминаний с учетом изменений.

Для получения справочной информации о лекарствах пользователь может перейти в соответствующий раздел приложения. Там отображается список доступных препаратов с краткой инструкцией (показания, противопоказания, способ применения). При необходимости пользователь может нажать на ссылку, которая откроет полную инструкцию в браузере.

Кроме того, в приложении реализован раздел статистики, где пользователь может анализировать соблюдение режима лечения. Для этого доступен выбор периода (неделя, месяц или произвольный интервал), отображается общий процент выполненных приемов, количество принятых и пропущенных доз, а также столбчатая диаграмма выполнения по дням. Для каждого лекарства показывается индивидуальный прогресс-бар. При необходимости пользователь может экспортировать отчет в PDF и поделиться им с лечащим врачом.

Также в приложении присутствует раздел «Аптечка», предназначенный для учета остатков лекарственных средств. Пользователь может добавлять препараты в аптечку, указывая текущее количество и единицу измерения. При отметке выполненного приема остаток автоматически уменьшается на указанную дозировку. Если запас заканчивается или падает ниже заданного порога, система отправляет соответствующее уведомление.

Таким образом, предложенная архитектура приложения обеспечивает пользователям полный цикл управления медикаментозным лечением: от создания персонализированного расписания до получения своевременных напоминаний и контроля соблюдения режима приемов. Интеграция с Firebase Cloud Firestore гарантирует надежную синхронизацию данных между устройствами, а механизм автоматического объединения гостевых данных с зарегистрированным аккаунтом исключает потерю информации при переходе к полноценной учетной записи. Система локальных уведомлений на базе `flutter_local_notifications` обеспечивает своевременное информирование о необходимости приема даже при выключенном интернете, а встроенное кэширование Firestore сохраняет частичную работоспособность приложения при отсутствии сетевого подключения. Дополнительные возможности – аптечка для контроля остатков лекарств, статистика соблюдения режима с визуализацией прогресса, экспорт отчетов в PDF и справочная информация с инструкциями – превращают приложение в комплексный инструмент поддержки приверженности лечению, удобный как для пациентов, так и для врачей.

2.3.1 Описание диаграммы использования

Диаграмма использования разработана для визуального представления функционала мобильного приложения для контроля приема лекарств. На диаграмме представлена основная роль пользователя системы и соответствующие ему функции, что позволяет продемонстрировать, какие действия доступны пользователю и как эти действия взаимосвязаны.

Диаграмма использования включает основную роль пользователя, который взаимодействует с приложением для управления своим лечением. При первом запуске пользователь автоматически работает в гостевом режиме без регистрации, что позволяет ему создавать и управлять расписаниями приема лекарств, отмечать выполненные приемы, получать уведомления и просматривать справочную информацию. Все данные гостя также сохраняются в базе данных. Однако для использования расширенных возможностей – ведения аптечки с учетом остатков, просмотра статистики, экспорта отчетов в PDF, добавления собственных лекарств и синхронизации данных между устройствами – необходимо зарегистрироваться или войти в аккаунт. При регистрации все данные, созданные в гостевом режиме, объединяются с новым аккаунтом.

Основной функционал приложения представлен на диаграмме вариантов использования в приложении Б.

Регистрация и авторизация позволяют пользователю создать новую учетную запись или войти в существующую через электронную почту и пароль либо через Google-аккаунт. Аутентификация обрабатывается Firebase Authentication, который после успешного входа выдает UID пользователя. Также доступна функция восстановления пароля через электронную почту.

Управление расписанием приема лекарств доступно как в гостевом, так и в авторизованном режиме. Пользователь может создавать, редактировать, удалять и просматривать расписания на выбранную дату.

Отметка о приеме лекарства позволяет фиксировать выполнение назначения непосредственно из интерфейса или через нажатие на уведомление. При отметке фиксируется фактическое время. Для авторизованных пользователей при отметке автоматически уменьшается остаток в аптечке.

Получение уведомлений реализовано через библиотеку `flutter_local_notifications`. Система отправляет напоминания в заданное время даже при закрытом приложении. При нажатии на уведомление приложение открывается на экране подтверждения приема.

Просмотр справочной информации о лекарствах дает возможность ознакомиться с краткой инструкцией, перейти по ссылке на полную инструкцию на сайте-источнике или на сайт для покупки препарата. Функция доступна только авторизованным пользователям.

Управление лекарствами доступно только авторизованным пользователям. Они могут добавлять, редактировать и удалять собственные лекарства, которые не входят в общую базу. Гостевой режим позволяет выбирать лекарства только из общей базы.

Управление аптечкой также доступно только авторизованным пользователям. Они могут добавлять лекарства в аптечку с указанием количества, пополнять запас, редактировать или удалять записи. При низком остатке система отправляет уведомление.

Просмотр статистики приемов и экспорт отчетов доступны только авторизованным пользователям. Пользователь может анализировать процент выполненных приемов за выбранный период, просматривать диаграмму выполнения по неделям и другим промежуткам, а также экспортировать отчет в формате PDF для отправки лечащему врачу.

Синхронизация данных обеспечивается Cloud Firestore. При наличии аутентификации все данные пользователя хранятся в облаке и автоматически загружаются на других устройствах. При регистрации гостевые данные объединяются с созданным аккаунтом.

2.4 Проектирование базы данных мобильного приложения

После разработки диаграммы вариантов использования была спроектирована структура базы данных для реализации всех поставленных задач. В качестве системы управления базами данных для мобильного приложения выбран Cloud Firestore – документо-ориентированная NoSQL база данных, отличающаяся высокой скоростью чтения и записи, входящая в состав платформы Firebase. Данный выбор обусловлен необходимостью обеспечения синхронизации данных между устройствами, простотой интеграции с Firebase Authentication.

Основой инфраструктуры хранения данных является гибкая документо-ориентированная модель, которая отражает взаимосвязь между сущностями приложения. Вся информация, создаваемая пользователем, сохраняется в облачном хранилище Firestore и привязывается к уникальному идентификатору.

Проектирование базы данных велось с учетом следующих ключевых сущностей: пользователи, лекарственные препараты (общая база), расписания при

ема лекарств. Перечень и назначение всех разработанных в ходе проектирования коллекций отображен в таблице 2.1.

Таблица 2.1 – Информация о коллекциях базы данных Cloud Firestore

Наименование таблицы	Назначение
users	Содержит информацию о зарегистрированных пользователях
drugs	Содержит информацию о лекарственных препаратах (общая база)
medicines	Содержит лекарства, добавленные пользователем в свой план лечения
schedules	Содержит расписания приема лекарств для каждого пользователя
intakes	Содержит список выполненных расписаний
stocks	Содержит информацию об остатках лекарственных средств пользователя

Схема созданной БД для данного проекта представлена в приложении В. Далее будет описана каждая таблица базы данных.

Коллекция «users» хранит информацию о зарегистрированных пользователях приложения. Каждый документ в этой коллекции соответствует одному пользователю и создается автоматически после успешной регистрации через Firebase Authentication.

Описание структуры коллекции «users» представлено в таблице 2.2.

Таблица 2.2 – Структура коллекции «users»

Название столбца	Тип данных	Описание столбца
uid	string	Идентификатор пользователя UUID
displayName	string	Отображаемое имя пользователя
email	string	Адрес электронной почты пользователя
provider	string	Провайдер аутентификации (password, google и т.д.)
createdAt	timestamp	Дата и время регистрации пользователя
lastLoginAt	timestamp	Дата и время последнего входа
photoURL	string	Ссылка на фото профиля

Коллекция «users» обеспечивает привязку всех данных к конкретному пользователю. При добавлении лекарств, создании расписаний или отметке приемов в каждом документе указывается уникальный идентификатор пользователя (uid), что позволяет при запросе загружать только те данные, которые принадлежат текущему авторизованному пользователю. Это обеспечивает полную изоляцию данных между разными учетными записями.

Коллекция «drugs» содержит справочную информацию о лекарственных препаратах, доступную всем пользователям приложения. Каждый документ соответствует одному лекарственному средству. Структура документа коллекции «drugs» представлена в таблице 2.3.

Таблица 2.3 – Структура коллекции «drugs»

Название столбца	Тип	Описание столбца
name	string	Название лекарственного препарата
genericName	string	Международное непатентованное название
dosage	string	Дозировка препарата
dosageForm	string	Форма выпуска (таблетки, капли, мазь, раствор и т.д.)
manufacturer	string	Производитель препарата
country	string	Страна-производитель
more_info	map	Дополнительная информация (код АТХ, особенности применения)
more_info_url	string	Ссылка на полную инструкцию по медицинскому применению
source_url	string	Ссылка на источник данных (сайт, откуда была получена информация)
createdAt	timestamp	Дата и время добавления записи в базу данных
last_updated	timestamp	Дата и время последнего обновления информации

Коллекция «drugs» является общей для всех пользователей. При добавлении нового лекарства в свой план лечения пользователь может выбрать препарат из этой коллекции, после чего `uid` копируется в таблицу расписаний, что позволяет легко сослаться на заданное нами лекарство.

Коллекция «medicines» хранит лекарственные препараты. В отличие от коллекции «drugs», данные здесь относятся только к текущему пользователю и могут быть им отредактированы или удалены.

Описание структуры коллекции «medicines» представлено в таблице 2.4.

Таблица 2.4 – Структура коллекции «medicines»

Название столбца	Тип	Описание столбца
name	string	Название препарата
dosage	string	Дозировка
dosageForm	string	Форма выпуска
manufacturer	string	Производитель
country	string	Страна-производитель
genericName	string	Международное непатентованное название
createdAt	timestamp	Дата и время добавления препарата в план лечения

Если пользователь создает собственный препарат (которого нет в общей базе «drugs»), он заполняет все поля вручную. Такой препарат будет виден только ему. После чего он может его использовать для своих целей.

Коллекция «schedules» хранит расписания приема лекарств для каждого пользователя. Каждый документ соответствует одному препарату в плане лечения пользователя и содержит информацию о периодичности,

дозировке препарата и времени приема.

Описание структуры коллекции «schedules» представлено в таблице 2.5.

Таблица 2.5 – Структура коллекции «schedules»

Название столбца	Тип	Описание столбца
medicineId	string	Идентификатор препарата (ссылка на документ в коллекции medicines)
medicineName	string	Название препарата (дублируется для удобства запросов)
dosage	string	Дозировка препарата
startDate	timestamp	Дата начала приема лекарства
endDate	timestamp	Дата окончания приема лекарства (может отсутствовать – бессрочный прием)
reminderTimes	array	Массив времен приема (например, ["08:00", "14:00", "20:00"])
createdAt	timestamp	Дата и время создания расписания

Поле reminderTimes хранит массив строк с временами приема в формате «ЧЧ:ММ». Это позволяет задавать как один прием в день, так и несколько (например, три раза в день). Поля startDate и endDate определяют период, в течение которого пользователь должен принимать лекарство. Если endDate не указан, прием считается бессрочным.

Внутри каждого документа расписания (schedules) создается подколлекция «intakes», которая хранит историю отметок о приеме лекарства. Это позволяет отслеживать, когда пользователь принял препарат, а когда пропустил, и в дальнейшем анализировать соблюдение режима лечения. Каждая запись в подколлекции содержит дату, время отметки и статус. Данные синхронизируются автоматически.

Описание подколлекции «intakes» представлено в таблице 2.6.

Таблица 2.6 – Структура таблицы «intakes»

Название столбца	Тип	Описание столбца
intakeTime	timestamp	Запланированное время приема (из расписания)
recordedAt	timestamp	Фактическое время отметки (когда пользователь нажал кнопку «Принято»)
taken	boolean	Статус приема: true – принято, false – пропущено
isMissed	boolean	Статус пропуска приема: true – пропущено, false – отменен

Поле intakeTime соответствует запланированному времени приема из расписания. Поле recordedAt фиксирует момент, когда пользователь фактически отметил прием. Это позволяет различать случаи, когда прием был выполнен вовремя, с опозданием или пропущен. Поле taken служит основным индикатором выполнения приема.

Для учета остатков лекарственных средств в документах пользователя создается подколлекция `stocks`. Каждый документ в этой подколлекции соответствует одному лекарству, идентификатор документа совпадает с `medicineId`. Структура документа представлена в таблице 2.7.

Таблица 2.7 – Структура таблицы «stocks»

Название столбца	Тип	Описание столбца
<code>currentAmount</code>	<code>double</code>	Текущее количество лекарства
<code>medicineId</code>	<code>string</code>	Уникальный идентификатор лекарств
<code>unit</code>	<code>string</code>	Единица измерения (таблетка, капсула, мл и т.д.)

Поле `medicineId` служит для привязки записи об остатке к конкретному лекарству, поле `currentAmount` хранит актуальное количество, а поле `unit` определяет единицу измерения (таблетка, капсула, мл и т.д.). При отметке выполненного приема значение `currentAmount` автоматически уменьшается на величину дозировки, а при падении остатка ниже порога система отправляет пользователю уведомление о необходимости пополнения запаса.

Для понимания взаимодействия созданных коллекций была спроектирована схема иерархий. Описание связей представлено в таблице 2.8.

Таблица 2.8 – Связи между коллекциями базы данных

Название связи	Связанные коллекции	Описание связи
<code>UserMedicines</code>	<code>users → medicines</code>	Определяет, какие лекарства принадлежат конкретному пользователю. Связь реализована через вложенную подколлекцию <code>medicines</code> ,
<code>UserSchedules</code>	<code>users → schedules</code>	Определяет, какие расписания принадлежат конкретному пользователю. Связь реализована через вложенную подколлекцию <code>schedules</code> ,
<code>UserStock</code>	<code>users → stock</code>	Определяет, какие аптечки принадлежат конкретному пользователю. Связь реализована через вложенную подколлекцию <code>stock</code> ,
<code>ScheduleIntakes</code>	<code>schedules → intakes</code>	Определяет, какие записи о приемах относятся к конкретному расписанию. Связь реализована через вложенную подколлекцию <code>intakes</code>

Спроектированная структура базы данных Cloud Firestore полностью покрывает все функциональные требования приложения: хранение информации о пользователях (`users`), общая справочная база лекарственных препаратов (`drugs`), личные лекарства пользователя (`medicines`), гибкие расписания приема с поддержкой нескольких времен в день и периода действия (`schedules`), хранилище лекарств и уведомление об окончании (`stocks`), а также история приемов с отметками о выполнении (`intakes`).

Коллекция `users` обеспечивает привязку всех данных к конкретному пользователю через уникальный идентификатор `UID`. Коллекция `drugs` содержит справочную информацию, собранную с внешних сайтов, и доступна всем пользователям. Коллекция `medicines` хранит препараты, добавленные пользователем в свой план лечения.

Коллекция `schedules` позволяет создавать гибкие расписания с указанием периода приема и нескольких времен в день. Коллекция `intakes` фиксирует каждый прием с отметкой о выполнении и временем фактического приема. Коллекция в `stock` хранит информацию о аптечке для выбранных лекарств.

Такая структура обеспечивает эффективную синхронизацию данных между устройствами и позволяет пользователю восстанавливать свои данные после переустановки приложения. Документно-ориентированная модель NoSQL дает возможность в будущем расширять функциональность без кардинальной перестройки базы данных. Правила безопасности Firebase обеспечивают изоляцию данных между разными пользователями.

2.5 Проектирование классов приложения

Архитектура приложения построена на принципе разделения ответственности между пользовательским интерфейсом, бизнес-логикой и данными. Для четкого разделения ответственности и упрощения тестирования внутренняя структура системы организована по слоям.

2.5.1 Основные классы сущности

Классы-модели представляют собой объекты данных, которые используются для хранения информации и передачи ее между слоями приложения. В проекте определены следующие основные модели: `User` – хранит информацию о пользователе (`UID`, `email`, `displayName`, `photoURL`). `Medicine` – хранит справочную информацию о лекарственном препарате, полученную из базы данных. `Schedule` – хранит расписание приема лекарства (даты, время). `DailyStat` и `MedicineStat` – используются для агрегации статистических данных о приемах за день или по конкретному лекарству соответственно. `Stock` – моделирует запись об остатках лекарства в аптечке пользователя, включая количество и единицу измерения. Все перечисленные классы содержат только поля и конструкторы, не содержат бизнес-логики и не зависят от фреймворков. Такая структура упрощает тестирование и поддержку кода. При добавлении новых функций модели легко расширяются. Кроме того, они могут использоваться независимо от платформы.

2.5.2 Контракты доступа к данным

В приложении реализовано три основных сервисных класса, каждый из которых отвечает за свою область функциональности.

Сервис `FirebaseService` является основным и инкапсулирует всю логику

взаимодействия с Firebase Authentication и Cloud Firestore. Он отвечает за регистрацию и вход пользователей (по email/пароллю), выход из учетной записи, сброс пароля и подтверждение email. Также сервис управляет созданием и обновлением документов пользователя в коллекции users, предоставляет методы для работы с расписаниями (создание, чтение, обновление, удаление) и лекарствами (поиск в общей базе drugs, добавление/удаление личных лекарств в коллекции medicines, отметка приемов в подколлекции intakes), предоставляет методы для работы с подколлекцией stocks: получение списка остатков, обновление количества при отметке приема, пополнение запаса, удаление лекарства из аптечки. . Сервис наследуется от ChangeNotifier, что позволяет уведомлять интерфейс об изменениях данных.

Сервис NotificationService отвечает за планирование и отображение локальных уведомлений о приеме лекарств. Он построен на базе библиотеки flutter_local_notifications. Сервис инициализирует каналы уведомлений для Android, запрашивает разрешения, планирует уведомления на основе данных из расписания (с учетом дат начала и окончания приема), а также отменяет уведомления при удалении или изменении расписания. Уведомления содержат название препарата и дозировку, срабатывают точно в заданное время и работают даже при закрытом приложении.

Класс DefaultFirebaseOptions содержит конфигурационные параметры подключения к проекту Firebase. В нем указаны apiKey, authDomain, projectId, storageBucket, messagingSenderId и appId. Этот класс используется при инициализации Firebase в приложении для подключения к облачным сервисам.

2.5.3 Классы управления состоянием

В приложении используется библиотека provider для управления состоянием. Основным классом-провайдером является FirebaseService, который наследуется от ChangeNotifier и предоставляет интерфейсу данные о состоянии аутентификации и работы с Firebase.

В функции main() при инициализации приложения создается MultiProvider, в который передается ChangeNotifierProvider, создающий экземпляр FirebaseService. Этот провайдер становится доступным для всех виджетов приложения. Виджеты автоматически подписываются на изменения.

Такой подход обеспечивает реактивное обновление интерфейса при изменении состояния аутентификации без необходимости ручного управления. FirebaseService также отвечает за загрузку расписаний и лекарств пользователя. Все запросы к Cloud Firestore централизованы в этом сервисе. Это позволяет избежать дублирования кода и упрощает внесение изменений.

2.5.4 Классы для работы с уведомлениями

В приложении также реализован специализированный класс NotificationService, который инкапсулирует всю логику работы с локальными уведомлениями на платформе Android. Данный класс построен по паттерну синглтон,

что особенно важно и гарантирует существование единственного экземпляра сервиса на все время работы приложения.

При инициализации сервиса настраивается временная зона Europe/Moscow для корректного планирования уведомлений, а также конфигурируются параметры канала уведомлений, включая иконку приложения, приоритет, возможность вибрации и звукового сопровождения.

Класс NotificationService предназначен для автоматического планирования и отправки напоминаний о приеме лекарств. Он анализирует все активные расписания пользователя, определяет даты и время будущих приемов и создает для них системные уведомления. Сервис также отвечает за отмену уведомлений при удалении или изменении расписания, обеспечивая актуальность напоминаний. Кроме того, он поддерживает мгновенные уведомления для информирования пользователя о событиях, происходящих непосредственно в приложении, например об успешной отметке приема или о критическом снижении остатка лекарства. Все операции с уведомлениями выполняются асинхронно, а сервис корректно обрабатывает ситуации, когда запланированное время уже наступило.

2.5.5 Взаимодействие классов в системе

Взаимодействие компонентов в рамках спроектированной архитектуры характеризуется следующими типами отношений:

Зависимость (Dependency) – виджеты пользовательского интерфейса зависят от провайдеров. Провайдеры, в свою очередь, зависят от сервисов. Виджеты не обращаются напрямую к сервисам, что снижает связанность.

Композиция (Composition) – провайдеры содержат ссылки на сервисы, которые передаются через конструктор или с помощью dependency injection. Это позволяет легко заменять реализации сервисов при тестировании.

Ассоциация (Association) – модели данных связаны между собой логическими отношениями. Например, объект Schedule содержит medicineId, ссылающийся на объект Medicine, а объект Intake привязан к конкретному Schedule. Объект User связан со всеми Schedule и Medicine через поле uid.

Наследование (Inheritance) – класс FirebaseService наследуется от ChangeNotifier, что позволяет ему использовать механизм уведомления об изменениях. При изменении данных (добавление, удаление, обновление) вызывается метод notifyListeners(), который автоматически и мгновенно обновляет все подписанные виджеты.

Реализация (Implementation) – класс NotificationService реализует паттерн синглтон (одиночка) через приватный конструктор и статический метод getInstance(), что гарантирует существование только единственного экземпляра сервиса в приложении.

Предложенная организация кодовой базы обеспечивает высокие показатели модульности, тестируемости и масштабируемости программного продукта на всех этапах разработки. Внесение изменений, связанных с расширением функциональности или заменой источника данных, не требует существенной модификации существующих компонентов.

2.6 Проектирование основных алгоритмов

2.6.1 Алгоритм работы первого запуска

При первом запуске приложение инициализирует Firebase и модуль уведомлений, после чего проверяет наличие сохраненной сессии. Поскольку сессии нет, система автоматически выполняет анонимную аутентификацию, создавая гостевого пользователя с временным анонимным идентификатором без необходимости ввода учетных данных. Затем приложение запрашивает у пользователя разрешение на показ уведомлений для обеспечения работы напоминаний. После получения разрешения пользователь попадает на главный экран, где ему доступна ограниченная версия функционала: создание и просмотр расписаний, отметка приемов, получение уведомлений и справочная информация. Для доступа к полному функционалу – аптечке, статистике, экспорту отчетов и синхронизации данных – пользователю необходимо зарегистрироваться или войти в существующий аккаунт. При регистрации все данные, созданные в гостевом режиме, автоматически объединяются с новым аккаунтом. Таким образом, гостевой режим служит удобной точкой входа для знакомства с приложением без обязательной регистрации.

2.6.2 Алгоритм создания расписания приема

Процесс создания расписания начинается с выбора лекарства. Пользователь попадает на экран поиска, где может найти нужный препарат. Поиск осуществляется по полю name в коллекциях drugs (общая база) и medicines (личные лекарства пользователя). При вводе текста в строку поиска вызывается метод searchMedicines(), который отправляет запросы в обе коллекции и объединяет результаты. Пользователь может выбрать препарат из общей базы (доступен всем) или из своих личных лекарств. Для гостевого режима кнопка добавления нового лекарства скрыта, так как создание собственных препаратов доступно только авторизованным пользователям.

После выбора лекарства пользователь переходит к настройке расписания. На экране создания расписания необходимо указать дозировку (количество и единицу измерения), дату начала приема, дату окончания приема, а также одно или несколько времен приема в формате «ЧЧ:ММ». Даты выбираются с помощью встроенного календаря, время – с помощью системного таймпикера. Все поля формы проходят валидацию перед сохранением.

После заполнения всех полей приложение сохраняет расписание в Cloud Firestore в коллекцию schedules, которая находится внутри документа пользователя. При успешном сохранении вызывается планировщик уведомлений, который создает напоминания на все дни от startDate до endDate для каждого указанного времени. Для авторизованных пользователей после сохранения расписания отображается диалог с предложением добавить лекарство в аптечку для отслеживания остатков. Гостевым пользователям этот диалог не показывается, так как функционал аптечки доступен только после регистрации.

Схема алгоритма для роли пользователя представлена в приложении Г.

2.6.3 Алгоритм планирования уведомлений

Уведомления о приеме лекарств являются центральным элементом приложения. Алгоритм их планирования реализован в классе `NotificationService`. При создании или обновлении расписания вызывается метод `scheduleRemindersForSchedule()`. Алгоритм работы этого метода: Сначала отменяются все старые уведомления для данного расписания с помощью `cancelScheduleNotifications()`. Определяется текущая дата и время (`DateTime.now()`). Устанавливается стартовая дата – `startDate` из расписания. Запускается цикл по дням от `startDate` до `endDate` (включительно). Для каждого дня цикл проходит по всем временам из массива `reminderTimes`. Для каждой комбинации (день + время) создается объект `DateTime`. Если запланированное время еще не наступило (позже текущего момента), вызывается внутренний метод `_scheduleSingleNotification()`, который создает одно уведомление через `flutter_local_notifications`. Каждое уведомление получает уникальный идентификатор, сформированный на основе хэш-кода ID расписания и счетчика дней и времени. После завершения цикла все уведомления запланированы и будут срабатывать в заданное время. Уведомления содержат заголовок «Напоминание о приеме» и текст «Пора принять: {название препарата} ({дозировка})». Настройки канала: высокая важность, вибрация, звук, режим `inexactAllowWhileIdle` для работы в режиме энергосбережения. Для восстановления уведомлений после перезагрузки устройства приложение при запуске вызывает метод `scheduleAllReminders()`, который загружает все активные расписания пользователя и перепланирует уведомления заново.

2.6.4 Алгоритм получение справочной информации о лекарстве

Приложение предоставляет пользователю доступ к справочной информации о лекарственных препаратах. Данные хранятся в коллекции `drugs` в `Cloud Firestore`. При выборе пользователем конкретного препарата из списка приложение загружает документ из коллекции `drugs`. Документ содержит следующие поля: `name` – название препарата; `genericName` – международное непатентованное название; `dosage` – дозировка; `dosageForm` – форма выпуска; `manufacturer` – производитель; `country` – страна-производитель; `more_info_url` – ссылка на полную инструкцию по медицинскому применению; `source_url` – ссылка на источник данных (сайт, откуда была получена информация). На экране информации о лекарстве пользователю отображается краткая версия: название, дозировка, форма выпуска, производитель. Для получения полной информации предусмотрена кнопка «Полная инструкция», при нажатии на которую открывается веб-браузер по ссылке `more_info_url`. Также имеется кнопка «Где купить», при нажатии на которую открывается сайт-источник по ссылке `source_url`, где пользователь может найти аптеки с данным препаратом, сравнить цены и оформить заказ. Такой подход позволяет пользователю не только получить базовую информацию о лекарстве, но и при необходимости ознакомиться с полной инструкцией или найти место покупки.

2.6.5 Алгоритм учета остатков при отметке приема

При отметке пользователем выполненного приема сначала фиксируется сам факт приема: в подколлекцию `intakes` добавляется запись с временем приема и признаком `taken = true`. Затем вызывается метод `decrementStock()`, который уменьшает текущее количество лекарства в подколлекции `stocks` на значение дозировки из расписания. Поиск нужной записи в аптечке осуществляется по `medicineId` и единице измерения, что позволяет корректно списывать лекарства даже если одно и то же средство хранится в разных формах (например, таблетки и миллилитры). Если запись в аптечке не найдена, прием все равно фиксируется, но пользователь получает предупреждение о том, что лекарство отсутствует в аптечке. Если новое количество становится равным нулю или опускается ниже порогового значения (по умолчанию пять единиц), система отправляет локальное уведомление с предупреждением о необходимости пополнить запас. При достижении нуля отправляется уведомление о том, что лекарство закончилось. Пользователь может в любой момент вручную пополнить запас через экран «В наличии», а также редактировать или удалять записи об остатках. Гостевым пользователям функция списания остатков недоступна, так как аптечка работает только после регистрации.

2.6.6 Алгоритм формирования статистики и экспорта PDF

При открытии экрана статистики пользователь выбирает период для анализа: неделя, месяц или произвольный интервал с возможностью указания произвольных дат начала и окончания. Данный экран является центральным инструментом для оценки эффективности лечения и позволяет пользователю наглядно видеть свой прогресс. Система загружает из Cloud Firestore все расписания пользователя и историю приемов за выбранный период, после чего вычисляет три ключевых показателя: общее количество запланированных приемов, количество фактически выполненных приемов и процент соблюдения режима лечения. Для наглядности на экране строится столбчатая диаграмма, где каждый столбец соответствует одному дню: зеленая часть отображает количество выполненных приемов, а серая – пропущенных. Для каждого лекарства в отдельности отображается индивидуальный прогресс-бар, показывающий процент выполнения по данному препарату, что позволяет пользователю быстро определить, какие лекарства он принимает регулярно, а какие часто пропускает. При нажатии на кнопку экспорта система формирует PDF-документ, содержащий все статистические данные за выбранный период: общие показатели, таблицу с детализацией по дням, прогресс по каждому лекарству и дату генерации отчета. Сформированный файл автоматически отправляется через стандартный системный диалог «Поделиться», что позволяет пользователю сохранить отчет на устройстве или отправить его лечащему врачу через мессенджеры, электронную почту или другие доступные приложения. Это делает взаимодействие с врачом максимально простым и удобным, так как отчет уже содержит всю необходимую информацию в структури-

рованом виде. Функционал статистики и экспорта отчетов доступен только авторизованным пользователям, так как требует хранения и обработки большого объема данных, привязанных к учетной записи. Таким образом, статистика не только помогает пациенту контролировать себя, но и становится полноценным инструментом взаимодействия с лечащим врачом.

2.7 Выводы по разделу

В ходе проектирования мобильного приложения была разработана архитектура, отвечающая современным требованиям к надежности, масштабируемости и удобству сопровождения. Использование фреймворка Flutter и языка Dart позволило создать гибкую и производительную основу для приложения. Выбор технологий – Flutter, Provider, Cloud Firestore и Firebase Authentication – обоснован необходимостью кроссплатформенной разработки, реактивного интерфейса и безопасной синхронизации данных между устройствами пользователя. Такой подход упрощает тестирование и поддержку кода.

Разработанная структурная схема приложения наглядно демонстрирует взаимодействие основных компонентов: мобильного приложения на Flutter, облачных сервисов Firebase (Authentication и Firestore) и системных служб Android (NotificationManager). Центральное место в архитектуре занимает облачное хранилище Cloud Firestore, которое обеспечивает синхронизацию данных между устройствами и резервное копирование. Система уведомлений, построенная на библиотеке flutter_local_notifications, учитывает особенности энергосбережения и гарантирует восстановление напоминаний после перезагрузки, что критически важно для своевременного приема лекарств.

Проектирование базы данных выполнено с учетом всех функциональных требований: хранение информации о пользователях, общая справочная база лекарственных препаратов, личные лекарства пользователя, гибкие расписания приема, а также детальная история приемов. Структура базы данных включает коллекции users, drugs, medicines, schedules и подколлекцию intakes. Диаграмма вариантов использования отражает все ключевые сценарии взаимодействия пользователя с приложением. Важной особенностью является проверка подтверждения email при входе – пользователь с неподтвержденным адресом не может войти в приложение, что повышает безопасность.

Реализованная архитектура позволяет легко масштабировать приложение в будущем, добавлять новые функции и подключать дополнительные источники данных без переписывания существующего кода.

Дополнительно реализованы модуль учета остатков (аптечка) для контроля текущего запаса лекарств и модуль статистики с визуализацией выполнения приемов и экспортом отчетов в PDF. Пользователь может получать уведомления о низком остатке препарата, когда количество таблеток достигает заданного порога. Это помогает своевременно пополнить аптечку и не прерывать курс лечения. Разрабатываемое приложение предназначено для помощи пользователям в соблюдении назначенных схем лечения, повышения их приверженности терапии и улучшения качества приема лекарств.

3 Реализация мобильного приложения

Проект реализован как мобильное приложение под Android, основная цель которого заключается в формировании расписания приема лекарств, своевременном напоминании пользователю о приемах и фиксации факта выполнения действий. В отличие от классического веб-приложения, где принято выделять серверную и клиентскую части и обмениваться данными через REST API, в рассматриваемом проекте основная логика размещена на стороне мобильного клиента, а облачная инфраструктура применяется как средство авторизации и синхронизации. Подобная архитектура особенно уместна для задач напоминаний, потому что именно мобильная платформа предоставляет системные механизмы уведомлений и фоновое планирование, позволяющие информировать пользователя даже при отсутствии активного интерфейса приложения.

При проектировании и реализации приложения сделан акцент на работу с облачной базой данных Cloud Firestore, которая обеспечивает синхронизацию данных между устройствами пользователя. Благодаря встроенной поддержке кэширования Firestore приложение остается работоспособным даже при временном отсутствии интернет-соединения: ранее загруженные данные доступны для просмотра, а новые записи сохраняются в локальный кэш и синхронизируются с сервером при восстановлении соединения. В результате пользователь не зависит от стабильности сети: добавленное лекарство отображается в интерфейсе сразу после сохранения в облаке, а синхронизация выполняется автоматически.

Код приложения написан на языке Dart с использованием фреймворка Flutter. Пользовательский интерфейс реализован с помощью декларативного подхода Flutter, что позволяет описывать UI в виде виджетов, реагирующих на изменение состояния приложения. Для структурирования проекта применен принцип разделения ответственности между слоями представления, бизнес-логики и данных. Слой представления отвечает за отображение виджетов и обработку действий пользователя, слой бизнес-логики инкапсулирован в сервисах (FirebaseService, NotificationService), а слой данных реализует взаимодействие с Cloud Firestore. Для управления состоянием и внедрения зависимостей используется паттерн Provider, который обеспечивает обновление интерфейса при изменении данных.

3.1 Разработка клиентской части

Клиентская часть приложения разделена на несколько основных компонентов. Экраны отвечают за отображение интерфейса и сбор ввода от пользователя. Провайдеры управляют состоянием и передают данные между экранами и сервисами. В контексте Flutter-приложения роль обработчика

					ДП 03.00.ПЗ		
		ФИО	Подпись	Дата	3 Реализация мобильного приложения		
Разраб.		Осадчий П.А.					
Пров.		Тимонович Г.Л.					
Н. контр.		Подрез А.А.					
Утв.		Блинова Е.А.					
					Лит.	Лист	Листов
					У	1	13
					БГТУ 1-98 01 03, 2026		

пользовательских действий выполняют виджеты (экраны) и провайдеры, а роль доступа к данным выполняют сервисы и модели.

Виджеты формируют интерфейс, принимают действия пользователя и передают их в провайдеры. Провайдеры управляют состоянием экранов и иницируют бизнес-логику, обращаясь к сервисам. Сервисы, в свою очередь, обеспечивают сохранение и получение информации, взаимодействуя с Cloud Firestore и локальным кэшем.

В проекте провайдеры выполняют функцию «центра управления» экраном: они содержат состояние, которое отображается в UI, и методы, которые вызываются при действиях пользователя. При создании расписания провайдер получает данные формы, вызывает соответствующий метод сервиса для сохранения в Cloud Firestore и обновляет состояние. Такой подход делает интерфейс реактивным и обеспечивает, что экраны всегда отображают актуальные данные. Более наглядно структура показана на рисунке 3.1

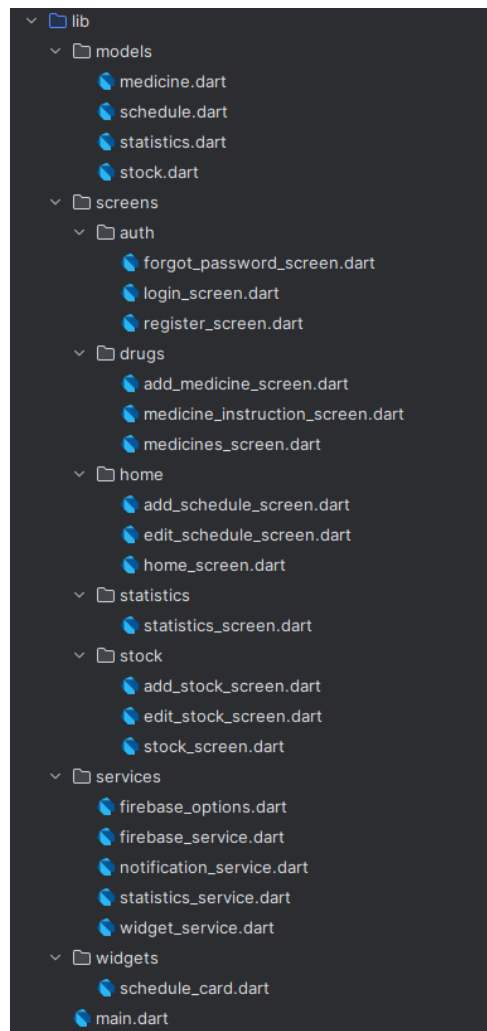


Рисунок 3.1 – Структура клиентской части проекта

Входной точкой в мобильное приложение является функция `main()`, которая выполняет инициализацию инфраструктуры. В ней происходит подключение Firebase, инициализация сервиса уведомлений и запуск приложения. Таким образом, функция `main()` является ключевым

компонентом, который подготавливает зависимости и сервисы до момента открытия первых экранов приложения.

В листинге 3.1 представлен фрагмент инициализации приложения.

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeDateFormatting('ru_RU', null);
  await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);
  await NotificationService().init();
  runApp(const MyApp());
}
```

Листинг 3.1 – Инициализация приложения

Данный фрагмент кода показывает, что при запуске приложения выполняются следующие действия: инициализация привязки Flutter к платформе Android, инициализация локализации дат для русского языка, инициализация Firebase с настройками проекта, инициализация сервиса локальных уведомлений, запуск корневого виджета, который создает провайдер `FirebaseService` для управления аутентификацией и старта приложения.

3.1.1 Реализация регистрации и авторизации пользователя

Для управления аутентификацией пользователей в приложении используется `Firebase Authentication`. Данный сервис обеспечивает безопасную регистрацию и аутентификацию пользователей по электронной почте и паролю, а также управление сессиями.

Для создания новой учетной записи в приложении используется метод `registerWithEmailAndPassword` класса `FirebaseService`. Данный метод принимает адрес электронной почты и пароль, после чего взаимодействует с `Firebase Authentication` для создания нового пользователя.

В листинге 3.2 представлен фрагмент кода, реализующий регистрацию.

```
Future<User?> registerWithEmailAndPassword(String email, String
password,) async { try {
  UserCredential result = await _auth.createUserWithEmailAndPassword(
    email: email.trim(),
    password: password,);
```

Листинг 3.2 – Фрагмент метода регистрации пользователя

Данный фрагмент кода показывает, что при регистрации выполняются следующие действия: создание нового пользователя в `Firebase Authentication` с указанными email и паролем, создание документа пользователя в коллекции `users` для хранения дополнительной информации (имя, дата регистрации, фото профиля), обработка возможных ошибок (email уже используется, слабый пароль, некорректный формат email) с выводом сообщений.

В дополнение к описанному функционалу, метод `registerWithEmailAndPassword` также реализует автоматическое объединения данных гостевого пользователя с только что созданным аккаунтом. Если пользователь до регистрации работал в гостевом режиме, система перед созданием нового аккаунта сохраняет все его расписания и историю приемов, а после успешной регистрации переносит эти данные в новую учетную запись. Такой подход обеспечивает бесшовный переход пользователя от гостевого режима к полноценной учетной записи без потери ранее внесенных данных, что повышает удобство использования приложения и стимулирует пользователей к регистрации.

Для входа зарегистрированного пользователя в приложение используется метод `signInWithEmailAndPassword`. Данный метод проверяет введенные учетные данные и, при успешной проверке, открывает доступ к приложению.

В листинге 3.3 представлен фрагмент кода, реализующий авторизацию.

```
Future<User?> signInWithEmailAndPassword(String email,
    String password,) async { try {
    UserCredential result = await
_auth.signInWithEmailAndPassword(
    email: email.trim(),
    password: password,
    );
    if (!result.user!.emailVerified) {throw Exception(
        'Подтвердите email перед входом. Проверьте свою
почту.', );}
    await _updateUserLastLogin(result.user!);
    return result.user;
  } on FirebaseAuthException catch (e) {
    throw _handleAuthError(e);
  }
}
```

Листинг 3.3 – Метод входа пользователя

Данный фрагмент кода показывает, что при авторизации выполняются следующие действия: проверка введенного email и пароля через `Firebase Authentication`, проверка статуса подтверждения email – если адрес не подтвержден, вход запрещается с соответствующим сообщением, обновление времени последнего входа в документе пользователя в `Cloud Firestore`, обработка ошибок аутентификации (неверный пароль, пользователь не найден, слишком много попыток) с выводом информативных сообщений.

3.1.2 Маршрутизация и навигация

Для управления состоянием в приложении используется паттерн `Provider`, который является рекомендуемым решением для Flutter-приложений. Основным классом-провайдером является `FirebaseService`, который наследуется от `ChangeNotifier` и предоставляет интерфейсу данные о состоянии аутентификации и работы с `Firebase`.

В листинге 3.4 представлена инициализация провайдера.

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {

    return MultiProvider(
      providers: [ChangeNotifierProvider(create: (_) => Fire-
baseService())],
      child: MaterialApp(
        title: 'Medicine Reminder',
        theme: ThemeData(primarySwatch: Colors.blue, useMate-
rial3: true),
        locale: const Locale('ru', 'RU'),
        localizationsDelegates: const [
          GlobalMaterialLocalizations.delegate,
          GlobalWidgetsLocalizations.delegate,
          GlobalCupertinoLocalizations.delegate,
        ],
        supportedLocales: const [Locale('ru', 'RU'), Lo-
cale('en', 'US')],
        home: const AuthWrapper(),
        debugShowCheckedModeBanner: false,
      ),
    );
  }
}
```

Листинг 3.4 – Инициализация провайдера FirebaseService

Данный фрагмент кода демонстрирует: использование MultiProvider для создания экземпляра FirebaseService, который становится доступным для всех дочерних виджетов приложения, настройку локализации приложения на русский язык с поддержкой английского языка, установку темы приложения, обеспечивающей единообразный внешний вид всех экранов, определение AuthWrapper в качестве домашнего экрана, который управляет навигацией в зависимости от статуса аутентификации.

Таким образом, корневой виджет MyApp обеспечивает централизованное управление состоянием через провайдер и настраивает основные параметры приложения: тему, локализацию и начальный экран.

Навигация между экранами в приложении реализована с использованием стандартного механизма Flutter – Navigator и MaterialPageRoute. Управление переходами осуществляется в зависимости от действий пользователя: нажатия на кнопки, выбор пунктов меню или изменение статуса аутентификации. При успешном входе выполняется автоматический переход на главный экран, а при выходе из аккаунта – на экран авторизации.

На главном экране навигация организована через боковое меню. В листинге 3.5 представлен фрагмент кода, демонстрирующий навигацию.

```

ListTile(
  leading: const Icon(Icons.medication, color: Colors.blue),
  title: const Text('Мои лекарства', style:
TextStyle(fontWeight: FontWeight.w500)),
  selected: true,
  selectedTileColor: Colors.blue.shade50,
  onTap: () => Navigator.pop(context),
),
ListTile(
  leading: const Icon(Icons.library_books, color: Colors.grey),
  title: const Text('Все лекарства'),
  onTap: () {
    Navigator.pop(context);
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const
MedicinesCatalogScreen()),
    );
  },
),

```

Листинг 3.5 – Навигация через боковое меню (Drawer)

Данный фрагмент кода демонстрирует реализацию бокового меню (Drawer) для навигации между основными экранами приложения.

Первый пункт меню «Мои лекарства» является текущим активным экраном. Для него установлены соответствующие визуальные атрибуты. При нажатии на него боковое меню закроется, показав активную страницу. Второй пункт меню «Все лекарства» предназначен для перехода к каталогу всех доступных лекарственных препаратов. При нажатии боковое меню закроется, а пользователю покажут соответствующую страницу.

Таким образом, реализованное боковое меню обеспечивает удобную навигацию между ключевыми разделами приложения: главным экраном «Мои лекарства», каталогом «Все лекарства», а также (в других пунктах меню) экранами «В наличии» и «Статистика».

3.2 Реализация доступа к данным

Для хранения данных приложения и синхронизации между устройствами используется облачная база данных Cloud Firestore, входящая в состав платформы Firebase. Подключение к Firebase выполняется на этапе инициализации приложения в функции `main()`.

После успешной инициализации коллекции и документы в Cloud Firestore создаются автоматически при первом обращении к ним через методы записи данных. Разработчику не требуется предварительно создавать схему базы данных, она формируется динамически по мере добавления данных приложением. Например, при регистрации нового пользователя автоматически создается документ в коллекции `users`, при добавлении лекарства.

Для конфигурации подключения используется класс `DefaultFirebase-`

Options, содержащий параметры проекта: API-ключ, домен аутентификации, идентификатор проекта и другие настройки.

Пример DefaultFirebaseOptions показан на листинге 3.6.

```
class DefaultFirebaseOptions {
    static FirebaseOptions get currentPlatform {
        return const FirebaseOptions(
            apiKey: YOUR_API_KEY,
            authDomain: YOUR_DOMAIN,
            projectId: medtracker-63e35,
            storageBucket: medtracker-63e35.firebaseiostorage.app,
            messagingSenderId: 736164815477,
            appId: 1:736164815477:web:21ba0915da2c7426e7ce3c,
        );
    }
}
```

Листинг 3.6 – Класс DefaultFirebaseOptions

Таким образом, подключение к Firebase выполняется однократно при запуске приложения, а база данных Cloud Firestore создается автоматически в облаке при первом обращении к ней.

3.2.1 Класс FirebaseService для работы с Firestore

Для инкапсуляции всей логики взаимодействия с Cloud Firestore и Firebase Authentication создан класс FirebaseService. Данный класс наследуется от ChangeNotifier, что позволяет уведомлять виджеты об изменениях данных и автоматически обновлять интерфейс.

В листинге 3.7 представлен фрагмент класса FirebaseService.

```
Stream<List<MedicineSchedule>> getSchedules() {
    if (currentUser == null) return Stream.value([]);
    return _firestore.collection('users').doc(currentUser!.uid).collection('schedules').snapshots().map((snapshot) {
        {
            return snapshot.docs.map((doc) {
                final data = doc.data();
                final Map<String, dynamic> jsonData = {...data};
                jsonData['id'] = doc.id;
                return MedicineSchedule.fromJson(jsonData);
            }).toList();
        }
    });
}
```

Листинг 3.7 – Методы класса FirebaseService

Данный фрагмент кода демонстрирует следующие возможности класса FirebaseService: метод getSchedules() возвращает Stream списка расписаний, что обеспечивает реактивное обновление интерфейса при изменении данных

в реальном времени. После выполнения операций записи вызывается `notifyListeners()` для автоматического обновления интерфейса.

Все методы класса `FirebaseService` используют идентификатор текущего аутентифицированного пользователя (`currentUser.uid`) для привязки данных к конкретной учетной записи, что обеспечивает полную изоляцию данных между разными пользователями.

3.3 Реализация взаимодействия между экранами

В приложении реализовано несколько ключевых экранов, каждый из которых отвечает за определенную функциональную область. Для всех экранов характерны общие принципы: использование провайдера `FirebaseService` для доступа к данным, реактивное обновление интерфейса с помощью `StreamBuilder` или `FutureBuilder`, обработка состояний загрузки и ошибок, а также возможность добавления, редактирования и удаления данных.

Основными экранами приложения являются: `HomeScreen` – главный экран с календарем и списком расписаний на выбранную дату, `MedicinesCatalogScreen` – экран со списком всех доступных лекарств (общая база и личные), `StockScreen` – экран для учета остатков лекарств (аптечка), `StatisticsScreen` – экран статистики с графиками и возможностью экспорта.

Кроме того, в приложении реализованы дополнительные экраны, обеспечивающие вспомогательную функциональность. Одним из таких экранов является экран `AddScheduleScreen`, который нужен для создания расписания.

Добавление расписания происходит поочередно, изначально выбирается препарат для расписания, а потом промежуток дат приема лекарств и время, которое нужно принимать каждый день. После создания расписания приложение предлагает добавить выбранное лекарство в аптечку для отслеживания количества оставшегося лекарства.

Фрагмент реализации перехода показан на листинге 3.8.

```
ElevatedButton(
  onPressed: () async {Navigator.pop(context);
    await Navigator.push(context, MaterialPageRoute( builder:
      (context) => AddToStockScreen(initialMedicine: medicine),),
    );
    completer.complete();
  },
  child: const Text('Добавить'),
),
```

Листинг 3.8 – Реализация перехода между экранами

В данном фрагменте сначала закрывается текущий экран, затем открывается экран добавления в аптечку с передачей данных о выбранном лекарстве через параметр `initialMedicine`. После завершения работы экрана вызывается `complete()`, сигнализирующий о завершении операции. Это позволяет обновить данные на предыдущем экране после возвращения.

3.4 Реализация работы уведомлений

Система уведомлений является ключевым функционалом приложения, обеспечивающим своевременное напоминание пользователю о необходимости приема лекарств. Для реализации уведомлений используется библиотека `flutter_local_notifications` версии 21.0.0, которая предоставляет удобный API для работы с системными уведомлениями на Android.

Сервис уведомлений реализован в виде класса-синглтона `NotificationService`, что гарантирует существование единственного экземпляра в приложении. Разработанная система уведомлений демонстрирует высокую надежность и способна доставлять напоминания независимо от состояния приложения и состояния устройства.

Если приложение находится на экране, уведомление все равно отображается в системной шторке, так как планирование осуществляется через `flutter_local_notifications`, который взаимодействует напрямую с системным `NotificationManager`. Пользователь видит уведомление и может на него нажать.

При свернутом или закрытом приложении. Когда пользователь сворачивает приложение или закрывает его свайпом, запланированные уведомления продолжают работать. Это достигается за счет того, что `flutter_local_notifications` использует механизмы `AlarmManager` на уровне операционной системы Android. Планировщик уведомлений регистрирует задачу в системе, которая не зависит от того, запущен ли процесс приложения в данный момент. В листинге 3.9 показан фрагмент планирования с использованием `zonedSchedule`, который сохраняет задачу в системе.

```
await _notifications.zonedSchedule(
  id: id,
  title: title,
  body: body,
  scheduledDate: tzScheduledTime,
  notificationDetails: details,
  androidScheduleMode:
    AndroidScheduleMode.inexactAllowWhileIdle,
);
```

Листинг 3.9 – Планирование уведомления

Параметр `androidScheduleMode: inexactAllowWhileIdle` гарантирует срабатывание уведомления даже при включенном режиме энергосбережения, хотя и с возможной небольшой задержкой.

Если устройство выключено в момент запланированного приема, уведомление не сработает. Это связано с тем, что операционная система Android не может выполнить запланированную задачу на выключенном устройстве. Однако после включения устройства приложение при запуске проверяет текущее расписание и при необходимости создает пропущенные уведомления. В листинге 3.10 показан фрагмент кода, который отвечает за проверку пропущенных приемов.

```

Future<void> _checkMissedIntakes() async {
  if (_isCheckingMissedIntakes || _isFirstCheckDone) return;
  _isCheckingMissedIntakes = true;
  try {
    final firebaseService =
Provider.of<FirebaseService>(context, listen: false);
    await firebaseService.checkAndAddMissedIntakes();
    _isFirstCheckDone = true;
  } finally {
    _isCheckingMissedIntakes = false;
  }
}

```

Листинг 3.10 – Проверка пропущенных приемов при запуске приложения

Таким образом, реализованная система обеспечивает максимально возможную надежность доставки напоминаний с учетом ограничений, накладываемых операционной системой Android.

3.5 Реализация виджета рабочего стола

Для повышения удобства использования приложения реализован виджет рабочего стола Android, который отображает список сегодняшних приемов лекарств без необходимости открывать приложение. Это позволяет пользователю быстро узнавать о предстоящих приемах в любой момент, просто взглянув на главный экран телефона. Виджет обновляется автоматически каждую минуту и адаптируется под размер, установленный пользователем.

Виджет рабочего стола функционирует на основе взаимодействия Flutter-приложения и нативного кода Android. Такая архитектура позволяет использовать преимущества обеих платформ: гибкость Flutter для управления данными и производительность нативного кода для отображения виджета. Вся логика построена вокруг SharedPreferences – системного хранилища, доступного как из Flutter, так и из нативного виджета. Приложение сохраняет в это хранилище актуальные данные о расписаниях, а виджет периодически их считывает и отображает. Процесс начинается с того, что при каждом изменении данных – добавлении нового расписания, отметке приема или удалении препарата – вызывается метод `updateWidgetData()` из сервиса `WidgetService`. Этот метод загружает из Cloud Firestore все активные расписания пользователя, фильтрует их, оставляя только те, у которых дата окончания еще не наступила, и преобразует полученные данные в JSON-формат. Затем полученная JSON-строка сохраняется в SharedPreferences с ключом `all_schedules`, после чего через пакет `home_widget` отправляется команда на обновление виджета. Виджет рабочего стола автоматически обновляется каждую минуту, даже если приложение не было открыто. Это гарантирует, что пользователь всегда видит актуальный список приемов на сегодня, не заходя в приложение.

Фрагмент кода, отвечающий за сохранение данных и установку их в виджете, представлен в листинге 3.11.

```

static Future<void> updateWidgetData(BuildContext context)
async {
    final activeSchedules = allSchedules.where((schedule) =>
        schedule.endDate.isAfter(now)).toList();
    final schedulesJson = activeSchedules.map((s) => ({
        'id': s.id, 'medicineName': s.medicineName, 'dosage':
s.dosage, 'startDate': s.startDate.millisecondsSinceEpoch,
'endDate': s.endDate.millisecondsSinceEpoch, 'reminderTimes':
s.reminderTimes,
    })).toList();
    final jsonString = jsonEncode(schedulesJson);
    await prefs.setString('all_schedules', jsonString);
    await HomeWidget.updateWidget(androidName: 'MedicineWidget');
}

```

Листинг 3.11 – Сохранение данных и обновление виджета

На стороне Android виджет реализован на языке Kotlin в классе `MedicineWidget`, который наследуется от `AppWidgetProvider`. При первом добавлении виджета на рабочий стол запускается фоновый процесс, который с помощью корутин каждую минуту выполняет обновление всех экземпляров виджета. При каждом обновлении виджет обращается к `SharedPreferences`, читает сохраненную JSON-строку по ключу `flutter.all_schedules`, после чего парсит ее и извлекает информацию о расписаниях. Затем вычисляются приемы на текущий день: для каждого расписания проверяется, входит ли сегодняшняя дата в интервал от `startDate` до `endDate`, и если входит, все времена приема из массива `reminderTimes` добавляются в список. Далее выполняется группировка лекарств по времени – если несколько препаратов нужно принять в одно и то же время, они объединяются в одну строку через запятую. Сформированный список строк отображается в виджете. Фрагмент кода, отвечающий за чтение данных и обновление интерфейса виджета, представлен в листинге 3.12.

```

private fun updateAppWidget(context: Context, appWidgetManager:
AppWidgetManager, appWidgetId: Int) {
    val views = RemoteViews(context.packageName, layoutId)
    if (groupedIntakes.isNotEmpty()) {
        views.setTextViewText(R.id.widget_line1,
groupedIntakes[0])
        if (groupedIntakes.size > 1)
views.setTextViewText(R.id.widget_line2, groupedIntakes[1])
    } else {views.setTextViewText(R.id.widget_empty, "Нет приемов
на сегодня")}
    appWidgetManager.updateAppWidget(appWidgetId, views)
}

```

Листинг 3.12 – Чтение данных и обновление интерфейса виджета

Виджет поддерживает два режима отображения в зависимости от размера, который пользователь задал на рабочем столе. При высоте менее 100 пикселей используется маленький макет, отображающий до двух строк с приемами, а также заголовок «Приемы». При высоте 100 пикселей и более за-

действуется большой макет, который показывает до четырех строк и содержит заголовок «Сегодняшние приемы». При изменении пользователем размера виджета система автоматически вызывает метод `onAppWidgetOptionsChanged`, который перерисовывает виджет с соответствующим макетом. Если на текущий день нет ни одного запланированного приема, виджет отображает сообщение «Нет приемов на сегодня». Таким образом, виджет работает полностью автономно, не требуя постоянного запуска основного приложения, и предоставляет пользователю информацию о приемах лекарств с рабочего стола.

3.6 Реализация статистики и экспорта отчетов

Для анализа соблюдения режима лечения в приложении реализован экран статистики, который отображает информацию о выполненных и пропущенных приемах лекарств за выбранный период. Пользователь может выбрать неделю, месяц или произвольный интервал дат. На основе выбранного периода система загружает из Cloud Firestore все расписания и историю приемов, после чего вычисляет необходимые показатели. Также пользователь может создать PDF-отчет и отправить его лечащему врачу

Фрагмент кода, отвечающий за формирование и сохранение PDF-отчета, представлен в листинге 3.13.

```
Future<void> _saveToDownloads() async {
    final fontData = await rootBundle.load('assets/fonts/Roboto-
Regular.ttf');
    final ttf = pw.Font.ttf(fontData);
    final pdf = pw.Document();
    pdf.addPage(pw.MultiPage(
        pageFormat: PdfPageFormat.a4,
        margin: pw.EdgeInsets.all(32),
        build: (context) => [
            pw.Header(level: 0, child: pw.Text('Отчет о приеме ле-
карств', style: pw.TextStyle(fontSize: 24, fontWeight: pw.Font-
Weight.bold, font: ttf))),
            .toStringAsFixed(1)}%', ttf),
            ],)... ..final dateStr =
DateFormat('yyyyMMdd_HH:mm:ss').format(DateTime.now());
    final fileName = 'medication_report_${dateStr}.pdf';
    final downloadDir =
Directory('/storage/emulated/0/Download');
    final file = File('${downloadDir.path}/${fileName}');
    await file.writeAsBytes(await pdf.save());
}
```

Листинг 3.13 – Формирование PDF-отчета

Экран статистики состоит из трех блоков. В верхней части отображаются три карточки с общей статистикой: процент выполненных приемов, общее количество запланированных приемов и количество пропущенных. Ниже располагается столбчатая диаграмма, показывающая динамику выполнения по

дням. При нажатии на столбец появляется всплывающая подсказка с детальной информацией. В нижней части отображается статистика по каждому лекарству в виде прогресс-бара.

Для визуализации используется библиотека `fl_chart`. При большом количестве дней данные автоматически группируются, чтобы количество столбцов на диаграмме не превышало десяти. Для экспорта данных реализованы две функции: сохранение отчета в формате PDF в папку «Загрузки» и отправка через диалог «Поделиться». При формировании PDF-документа в него включаются общая статистика, таблица по лекарствам, таблица по дням, а также период отчета и дата генерации. Для создания PDF используется библиотека `pdf`, позволяющая формировать документы с таблицами и текстами.

Таким образом, реализованный экран статистики предоставляет пользователю полную информацию о соблюдении режима лечения, позволяет анализировать динамику выполнения приемов и при необходимости передавать отчет лечащему врачу в удобном формате PDF.

3.7 Вывод по разделу

В третьем разделе дипломного проектирования описана реализация мобильного приложения для отслеживания приема лекарств. В ходе разработки были успешно реализованы все ключевые функциональные компоненты.

Клиентская часть построена на Flutter и Dart. Для управления состоянием применен паттерн `Provider`, обеспечивающий реактивное обновление интерфейса и разделение логики представления. Реализована система маршрутизации для навигации между экранами.

Для синхронизации данных используется облачная база `Cloud Firestore`. Все операции инкапсулированы в классе `FirebaseService`, который предоставляет методы для работы с расписаниями, лекарствами и историей приемов. Данные привязываются к конкретному пользователю.

Реализованы основные экраны: `HomeScreen` (календарь и расписания), `MedicinesCatalogScreen` (база лекарств с поиском), `StockScreen` (учет остатков) и `StatisticsScreen` (статистика с экспортом в PDF).

Система уведомлений на базе `flutter_local_notifications` обеспечивает своевременные напоминания. Это ключевая функция приложения, позволяющая пользователю не пропустить время приема лекарств. Уведомления работают при закрытом приложении и восстанавливаются после перезагрузки мобильного устройства.

Виджет рабочего стола на Kotlin с использованием `home_widget` отображает сегодняшние приемы лекарств, обновляется каждую минуту и группирует препараты по времени приема.

Таким образом, разработанное приложение представляет собой полноценное решение для контроля приема лекарств, объединяющее удобный интерфейс, надежную синхронизацию данных и интеллектуальную систему напоминаний. Все поставленные задачи выполнены, функциональные требования реализованы в полном объеме.

4 Тестирование мобильного приложения

Процесс тестирования позволяет убедиться в корректной работе всех функций приложения, выявить возможные ошибки и устранить их до передачи продукта пользователю. Для разработанного приложения контроля приема лекарств были проверены ключевые сценарии: аутентификации, авторизации, работа с лекарствами.

4.1 Тестирование авторизации и аутентификации

При первом запуске приложения пользователь попадает на экран входа. Для создания учетной записи необходимо заполнить поля электронной почты и пароля. Если какое-либо поле оставлено пустым, система отображает соответствующее предупреждение и не позволяет продолжить регистрацию.

Пароль должен содержать не менее 6 символов. При попытке установить короткий пароль, например «123», приложение выдает сообщение о том, что пароль слишком слабый. Адрес электронной почты проверяется на корректность: отсутствие символа «@» или домена приводит к сообщению об ошибке. Пример ошибки показан на рисунке 4.1

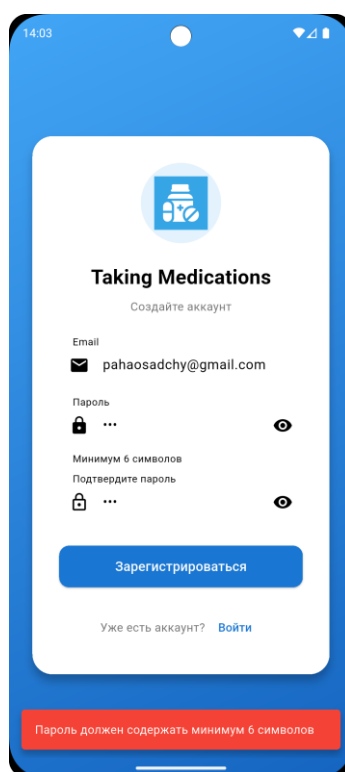


Рисунок 4.1 – Уведомления о ошибках при регистрации

					ДП 04.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.	Осадчий П.А.				4 Тестирование мобильного приложения	Лит.	Лист	Листов
Пров.	Тимонович Г.Л.					У	1	4
Н. контр.	Подрез А.А.					БГТУ 1-98 01 03, 2026		
Утверд.	Блинова Е.А.							

Те же самые проверки есть и на экране входа. Кроме того, при входе с неверным паролем или незарегистрированным email также отображается сообщение об ошибке. На рисунке 4.2 показано уведомление об ошибке при попытке входа в аккаунт через почту и пароль.

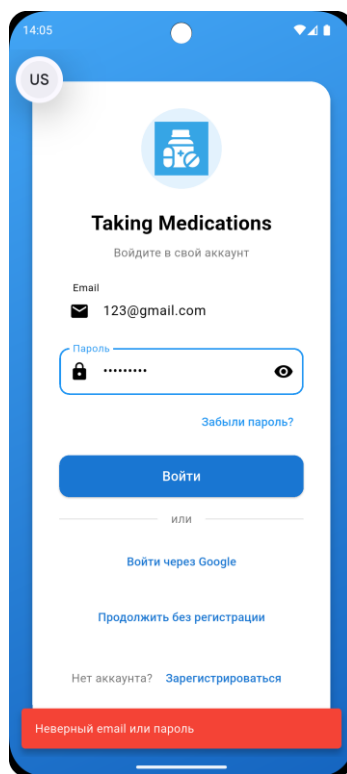


Рисунок 4.2 – Уведомление об ошибке при аутентификации

Таким образом, реализованная система аутентификации обеспечивает безопасность учетных записей за счет использования встроенных механизмов Firebase Authentication, включая хеширование паролей и управление сессиями. Приложение поддерживает два способа входа: по электронной почте и паролю или через Google-аккаунт, а при ошибках ввода выводит понятные локализованные сообщения на русском языке. Кроме того, реализована функция восстановления пароля и автоматическое объединение данных гостевого пользователя с аккаунтом при регистрации.

4.2 Тестирование добавления расписания

Основной функцией приложения является создание расписания приема лекарств. Процесс добавления включает выбор препарата (из общей базы или создание собственного), указание периода лечения и времени приема.

При вводе названия лекарства в строку поиска система обращается к Cloud Firestore и отображает подходящие варианты. Если нужного препарата нет в базе, пользователь может добавить собственный, указав соответствующие данные. Для того чтобы новое лекарство создавалось нужно указать хотя бы название, если название не указано, то пользователя уведомят, что без названия добавить лекарство нельзя, как показано на рисунке 4.3.

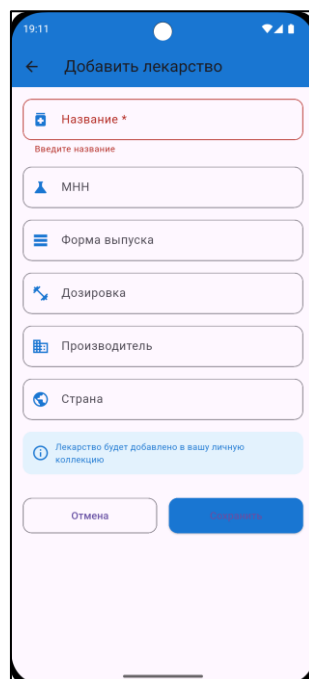


Рисунок 4.3 – Уведомление о пустом поле названия

При настройке периода приема необходимо указать дату начала и окончания. Если дата окончания раньше даты начала, система выводит сообщение об ошибке и не сохраняет расписание. При отсутствии даты начало или даты конца приема также отображается предупреждение. При добавлении расписания предусмотрено то, что нужно оставить не меньше одного выбранное времени, таким образом иконка удаления исчезает, не давая удалить последнее время приема. Ошибка, связанная с датой показана на рисунке 4.4.

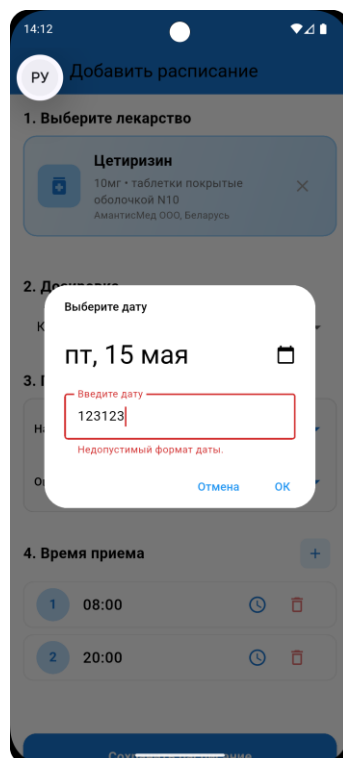


Рисунок 4.4 – Уведомления о неправильной дате

После успешного сохранения расписание появляется на главном экране. Для отметки выполненного приема пользователь нажимает на кнопку рядом с временем. При этом запись о приеме сохраняется в подколлекции `intakes`, а количество лекарства в аптечке автоматически уменьшается.

Тестирование подтвердило корректную работу всех этапов: поиск лекарств работает без задержек, валидация полей предотвращает создание некорректных записей, а отметка приема успешно фиксируется в базе данных.

4.3 Выводы по разделу

В ходе тестирования мобильного приложения были проверены все ключевые сценарии, обеспечивающие корректную работу системы. Тестирование проводилось последовательно по каждому из реализованных модулей, что позволило охватить весь функционал приложения. В разделе аутентификации успешно протестированы процессы регистрации и входа: приложение корректно обрабатывает пустые поля, выполняет валидацию пароля (длина не менее 6 символов), проверяет формат и уникальность адреса электронной почты, а также выводит понятные сообщения об ошибках при попытке использовать уже зарегистрированный email или неверные учетные данные. Все проверки проходят в штатном режиме, без сбоев и неожиданных поведений системы.

При тестировании добавления расписания подтверждена работоспособность всех этапов создания расписания приема лекарств. Каждый этап был проверен отдельно, что подтвердило корректность их взаимодействия. Особое внимание уделялось корректности сохранения данных в Cloud Firestore. Все записи успешно синхронизируются с облаком и становятся доступны на других устройствах пользователя. Поиск лекарств по названию выполняется без задержек, результаты отображаются корректно и полностью соответствуют поисковому запросу. Предусмотрена возможность добавления собственного лекарства при отсутствии нужного препарата в общей базе, при этом обязательное поле названия проходит валидацию. При настройке периода приема система проверяет, что дата окончания не может быть раньше даты начала, а также требует указания обеих дат. Времена приема настраиваются гибко: пользователь может добавить несколько интервалов, при этом последнее время невозможно удалить, что гарантирует наличие хотя бы одного времени приема. После успешного сохранения расписание корректно отображается на главном экране, а отметка выполненного приема фиксируется в подколлекции `intakes` и автоматически уменьшает количество соответствующего лекарства в аптечке. Все перечисленные функции работают без ошибок и задержек.

Таким образом, функциональность приложения соответствует заявленным требованиям, пользовательский интерфейс интуитивно понятен, а механизмы валидации предотвращают создание некорректных записей. В процессе тестирования не было выявлено критических проблем, влияющих на работу приложения. Выявленные в процессе тестирования мелкие недочеты были устранены, что позволяет считать приложение готовым к эксплуатации. Приложение может быть рекомендовано к использованию целевой аудиторией.

5 Руководство пользователя

5.1 Регистрация пользователя

Для начала работы с приложением пользователю необязательно проходить регистрацию – при первом запуске система автоматически создает гостевую учетную запись. Однако для доступа ко всем возможностям приложения, включая синхронизацию данных, ведение аптечки, просмотр статистики и экспорт отчетов, необходимо зарегистрироваться или войти в аккаунт.

Для регистрации необходимо на главном экране, нажать кнопку «Войти» в верхней панели, затем на экране авторизации нажать «Зарегистрироваться». Откроется форма регистрации, где следует указать адрес электронной почты, пароль длиной не менее 6 символов и подтверждение пароля. После заполнения полей нажмите кнопку «Зарегистрироваться». При успешной регистрации пользователь автоматически авторизуется в приложении, а ранее созданные в гостевом режиме данные будут перенесены в новый аккаунт.

Форма для регистрации предоставлена на рисунке 5.1.

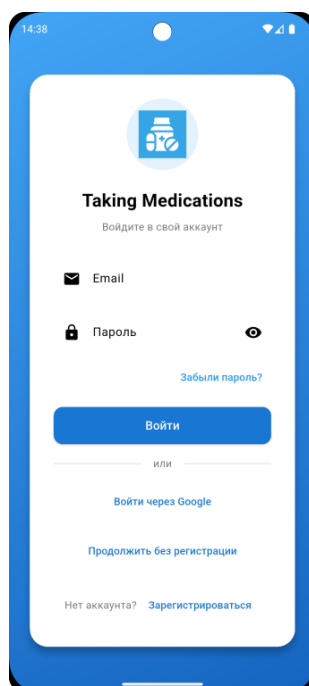


Рисунок 5.1 – Экран регистрации

При возникновении ошибок система отобразит соответствующее сообщение. Если email уже используется, необходимо указать другой адрес или восстановить пароль. При некорректном email следует проверить формат

					ДП 05.00.ПЗ					
		ФИО	Подпись	Дата						
Разраб.		Осадчий П.А.			5 Руководство пользователя			Лит.	Лист	Листов
Пров.		Тимонович Г.Л.						У	1	12
Н. контр.		Подрез А.А.						БГТУ 1-98 01 03, 2026		
Утверд.		Блинова Е.А.								

ввода. Если пароль слишком короткий, нужно ввести не менее 6 символов. После успешной регистрации пользователь может входить в приложение на любом устройстве, используя те же учетные данные, а все данные будут автоматически синхронизироваться через облачное хранилище.

5.2 Авторизация пользователя

Для входа в существующую учетную запись пользователю необходимо на главном экране приложения нажать на кнопку «Войти», расположенную в верхней панели. На открывшемся экране авторизации следует ввести адрес электронной почты и пароль, указанные при регистрации, после чего нажать на кнопку «Войти». При успешной авторизации пользователь попадает на главный экран, где ему становятся доступны все функции приложения, включая аптечку, статистику и экспорт отчетов. Если пользователь ранее работал в гостевом режиме на этом устройстве, его данные автоматически объединятся с данными аккаунта после входа.

Форма для авторизации предоставлена на рисунке 5.2.

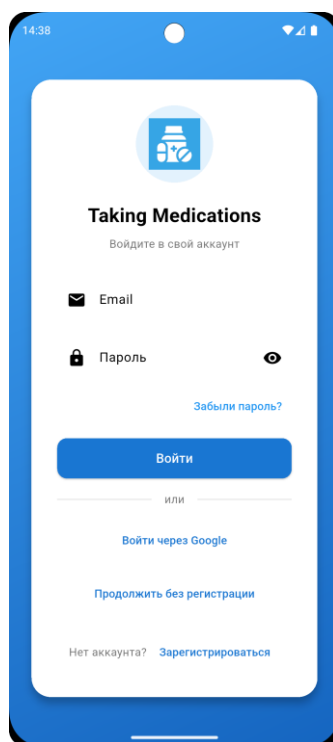


Рисунок 5.2 – Экран авторизации

При возникновении ошибок система отобразит соответствующее сообщение. Если введены неверный email или пароль, появится сообщение «Неверный email или пароль». В этом случае следует проверить правильность ввода данных или воспользоваться функцией восстановления пароля. Если пользователь не зарегистрирован, необходимо нажать на ссылку «Зарегистрироваться» и создать новую учетную запись. Также на экране авторизации доступна кнопка «Забыли пароль?», при нажатии на которую на указанный при регистрации email приходит письмо со ссылкой для сброса пароля. Кроме

того, предусмотрен вход через Google-аккаунт – для этого достаточно нажать на соответствующую кнопку и выбрать учетную запись Google.

5.3 Главная страница

После входа в приложение или автоматического перехода в гостевой режим пользователь попадает на главный экран, который является основным рабочим пространством приложения. Главная страница отображает расписание приема лекарств на текущий день и предоставляет быстрый доступ ко всем основным функциям приложения приема лекарств.

Главная страница предоставлена на рисунке 5.3.

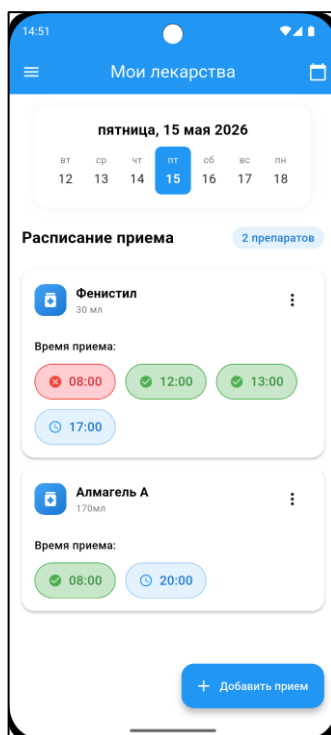


Рисунок 5.3 – Главная страница

В верхней части экрана расположен календарь для выбора даты. Пользователь может листать дни горизонтально или нажать на иконку календаря для выбора произвольной даты. Текущая дата выделена цветом, а над календарем крупным шрифтом отображается выбранная дата. Ниже представлен список лекарств, прием которых запланирован на выбранную дату.

Для каждого препарата отображается карточка с названием, дозировкой и временем приема. Каждое время представлено в виде кнопки, цвет которой указывает на статус: синий – не отмечен, зеленый – выполнен, красный – пропущен. При нажатии статус переключается. При отметке приема для авторизованных пользователей лекарство списывается из аптечки, а при снятии отметки – количество равно количеству дозировки возвращается обратно.

В правом верхнем углу карточки находится кнопка с тремя точками, открывающая контекстное меню с пунктами «Информация о лекарстве», «Редактировать» и «Удалить». Для гостевого режима пункт информации скрыт.

В левой части верхней панели для гостя отображается иконка входа, а для авторизованного пользователя – иконка меню для навигации по разделам «Мои лекарства», «Все лекарства», «В наличии» и «Статистика», а также кнопка выхода из аккаунта.

При нажатии на плавающую кнопку с иконкой плюса открывается экран создания расписания, представленный на рисунке 5.4.

15:00

← Добавить расписание

1. Выберите лекарство

Фенистил
1 мг · гель /г 30г N1
Хелен КХ САРЛ, Швейцария

2. Дозировка

Количество: 10 Единица измерения: МГ

3. Период приема

Начало: 15 мая 2026

Окончание: 22 мая 2026

4. Время приема

1 08:00

2 20:00

Сохранить расписание

Рисунок 5.4 – Экран добавления расписания

Пользователю необходимо последовательно выбрать лекарство, указать дозировку, задать период приема и добавить время приема. Для выбора лекарства в строке поиска вводится название препарата, после чего отображаются результаты из общей базы и личных лекарств. Авторизованные пользователи могут добавить собственный препарат, нажав кнопку «Добавить новое лекарство» при выборе лекарства. Дозировка задается количеством и единицей измерения. Период приема определяется датами начала и окончания, выбираемыми в календаре. Время приема можно добавить, изменить или удалить. После заполнения полей необходимо нажать «Сохранить расписание».

При успешном сохранении для авторизованных пользователей отображается диалог с предложением добавить лекарство в аптечку. Для гостя кнопка добавления лекарства и диалог аптечки скрыты.

Экран редактирования открывается при выборе пункта «Редактировать» в контекстном меню карточки лекарства. Интерфейс аналогичен экрану создания, но поля предварительно заполнены текущими значениями. После сохранения изменений старые уведомления отменяются и создаются новые.

Экран редактирования представлен на рисунке 5.5.

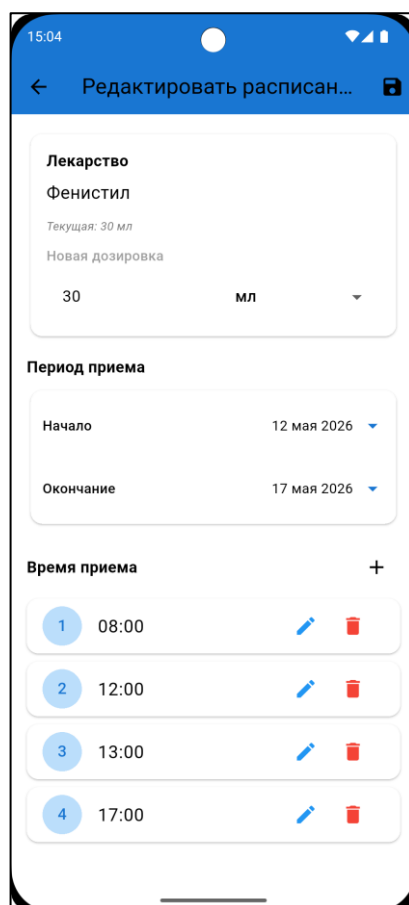


Рисунок 5.5 – Экран изменения расписания

При выборе пункта «Удалить» появляется диалоговое окно с подтверждением. Пользователь может подтвердить удаление или отменить его. При подтверждении расписание и все связанные с ним уведомления удаляются.

Также есть отдельный пункт, «Информация о лекарстве», который будет рассматриваться в следующем разделе.

5.4 Страница всех лекарств

Страница «Все лекарства» предназначена для просмотра справочной информации о лекарственных препаратах. Пользователь может открыть ее из бокового меню приложения. Экран предоставляет доступ к общей базе лекарств, собранной из открытых источников, а также к личным лекарствам, добавленным самим пользователем.

В верхней части экрана расположена строка поиска, позволяющая быстро найти нужный препарат по названию. Справа от строки поиска находится кнопка фильтра, при нажатии на которую открывается диалоговое окно выбора источника данных. Пользователь может выбрать один из трех вариантов: «Общая база» – отображаются только лекарства из общей коллекции drugs, доступные всем пользователям; «Личная база» – отображаются только лекарства, добавленные самим пользователем в коллекцию medicines; «Все» – отображается объединенный список из обеих баз.

Страница всех лекарств представлена на рисунке 5.6.

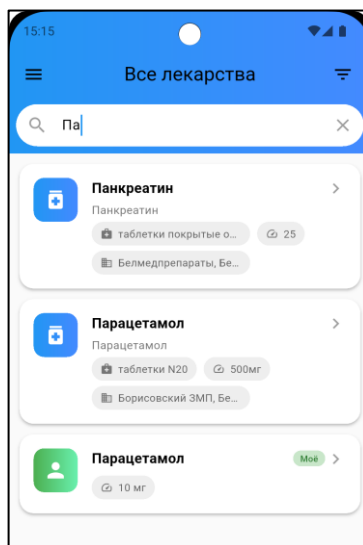


Рисунок 5.6 – Страница всех лекарств

Ниже строки поиска отображается список лекарств в виде карточек. Каждая карточка содержит название препарата, международное непатентованное название (при наличии), форму выпуска, дозировку и производителя. Личные лекарства помечаются зеленым чипом «Мое», а общие – синей иконкой. При нажатии на карточку открывается модальное окно с детальной информацией о препарате, включая форму выпуска, дозировку, производителя и страну происхождения. При нажатии на карточку всплывает модальное окно полной информации о лекарстве.

Модальное окно с информацией показаны на рисунке 5.7.

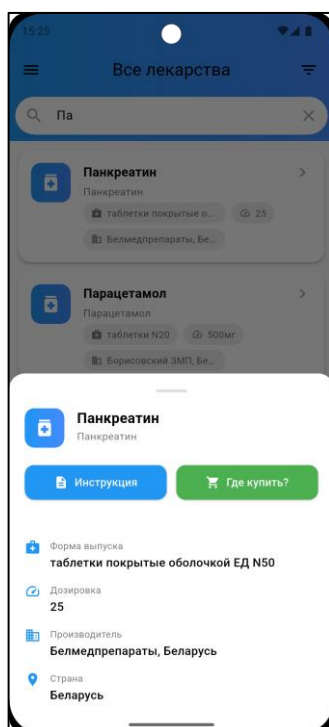


Рисунок 5.7 – Окно с информацией

В этом же окне для личных лекарств доступны кнопки редактирования и удаления. Также в окне деталей присутствуют кнопки «Инструкция» и «Где купить?». При нажатии на кнопку «Инструкция» открывается новый экран с подробной информацией о лекарстве, собранной из открытых источников, где каждый раздел можно развернуть для просмотра. При нажатии на кнопку «Где купить?» открывается внешний браузер со ссылкой на сайт-агрегатор аптек.

Экран с инструкцией представлен на рисунке 5.8.

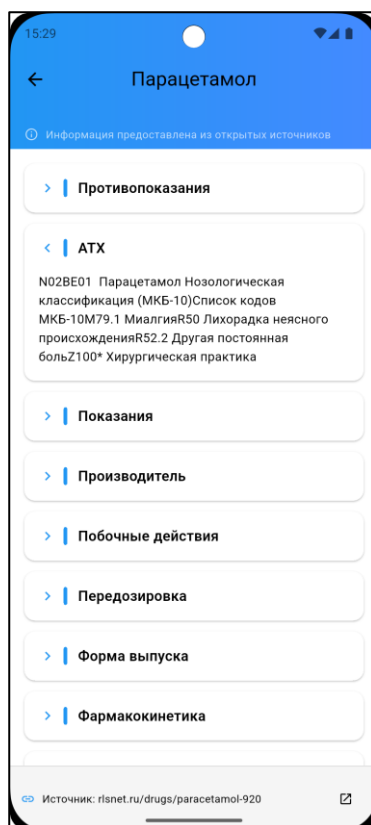


Рисунок 5.8 – Экран с инструкцией

Для авторизованных пользователей в окне деталей личного лекарства доступны кнопки редактирования и удаления. При нажатии на кнопку редактирования открывается диалоговое окно с формой, где можно изменить название, международное название, дозировку, форму выпуска, производителя и страну. При нажатии на кнопку удаления появляется диалоговое окно с подтверждением, после чего лекарство удаляется из личной базы. Для гостевого режима функции добавления, редактирования и удаления личных лекарств недоступны, поэтому кнопки редактирования и удаления скрыты.

5.5 Страница аптечки

Страница «В наличии» предназначена для учета остатков лекарственных средств. Пользователь может открыть ее из бокового меню приложения. Данный раздел доступен только авторизованным пользователям, так как требует привязки данных к учетной записи.

Страница аптечки представлена на рисунке 5.9.

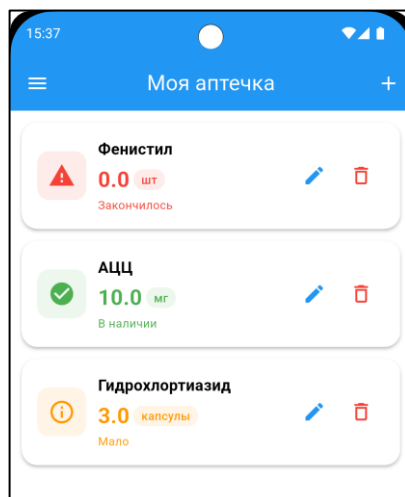


Рисунок 5.9 – Страница аптечки

В верхней части экрана расположена кнопка добавления лекарства в виде иконки с плюсом. При нажатии на нее открывается экран поиска, где пользователь может найти нужный препарат в общей или личной базе лекарств. После выбора лекарства необходимо указать количество и единицу измерения (мг, мл, шт, капсулы, капли или таблетки). При этом система проверяет, не добавлено ли уже это лекарство с такой же единицей измерения – если добавлено, появляется предупреждение, и кнопка сохранения блокируется, так как в одной единице измерения может быть только одна запись. Лекарство можно добавить в другом количестве, выбрав другую единицу измерения (например, если таблетки уже есть, можно добавить миллилитры).

Экран добавления лекарства в аптечку представлен на рисунке 5.10.

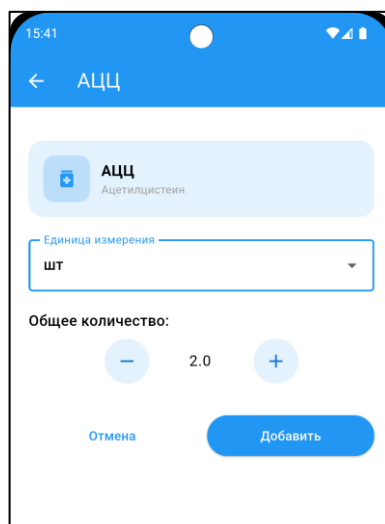


Рисунок 5.10 – Экран добавления в аптечку

Ниже экрана отображается список всех лекарств, добавленных в аптечку. Каждая карточка содержит название препарата, текущее количество, единицу измерения и индикатор статуса. Если количество равно нулю, статус

отображается красным цветом с надписью «Закончилось» и иконкой предупреждения. Если количество составляет от одной до пяти единиц, статус отображается оранжевым цветом с надписью «Мало». Если количество превышает пять единиц, статус отображается зеленым цветом с надписью «В наличии». В правой части карточки расположены две кнопки: карандаш для редактирования количества и мусорная корзина для удаления лекарства из аптечки.

При нажатии на кнопку редактирования открывается экран, где можно изменить количество лекарства с помощью кнопок увеличения и уменьшения или ввести значение вручную. Изменение количества может потребоваться при пополнении запаса или при обнаружении ошибки в учете.

Экран редактирования количества представлен на рисунке 5.11.

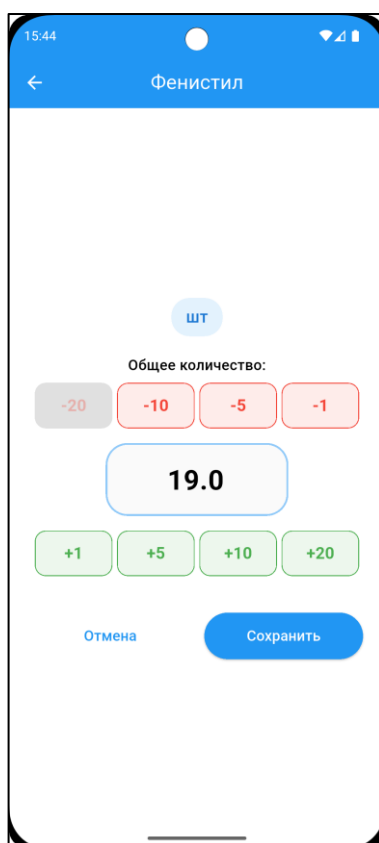


Рисунок 5.11 – Экран редактирования аптечки

При нажатии на кнопку удаления появляется диалоговое окно с подтверждением, после чего лекарство удаляется из аптечки. Удаление записи не влияет на расписания приема, так как аптечка ведется отдельно.

При отметке выполненного приема на главном экране количество соответствующего лекарства в аптечке автоматически уменьшается на величину дозировки из расписания. При снятии отметки количество возвращается обратно. Если после списания остаток становится равным нулю, при следующем открытии страницы карточка лекарства отобразится с красным статусом «Закончилось». Если остаток опускается ниже пяти единиц, лекарство помечается статусом «Мало», и пользователь получает соответствующее уведомление о необходимости пополнить запас.

5.6 Страница статистики

Страница статистики предназначена для анализа соблюдения режима лечения и доступна только авторизованным пользователям. Пользователь может открыть ее из бокового меню приложения. Экран позволяет выбрать период для анализа: неделя, месяц или произвольный интервал дат. При выборе произвольного периода открывается календарь для указания дат начала и окончания. При выборе периода система загружает из Cloud Firestore все расписания и историю приемов, после чего вычисляет необходимые показатели.

Страница статистики представлена на рисунке 5.12.

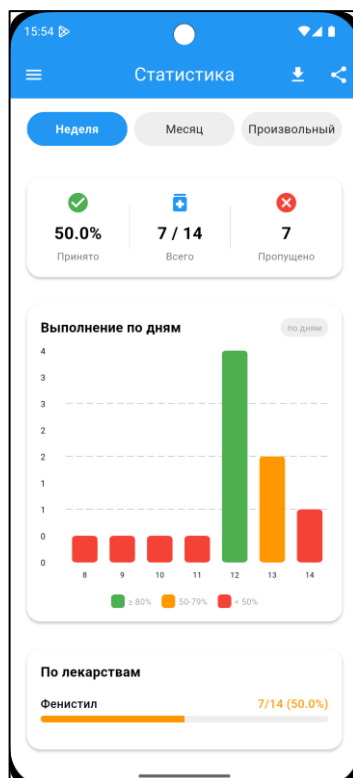


Рисунок 5.12 – Страница статистики

В центре экрана отображаются три карточки с общей статистикой: процент выполненных приемов, отношение принятых доз к общему количеству запланированных и количество пропущенных приемов. Ниже располагается столбчатая диаграмма, показывающая динамику выполнения приемов по дням. Каждый столбец диаграммы отображает количество принятых приемов относительно общего числа, запланированных на этот день. Цвет столбца зависит от процента выполнения: зеленый – 80% и выше, оранжевый – от 50% до 79%, красный – ниже 50%. При нажатии на столбец появляется всплывающая подсказка с детальной информацией: количество принятых и общего числа приемов, а также процент выполнения. Если выбранный период содержит большое количество дней, данные автоматически группируются для наглядности – например, при выборе месяца столбцы группируются по три дня, при выборе полугодия – по десять дней. Над диаграммой отображается индикатор группировки, поясняющий, сколько дней объединено в один столбец.

В нижней части экрана отображается статистика по каждому лекарству в виде списка с прогресс-барами. Для каждого препарата показывается название, количество принятых и запланированных приемов, процент выполнения, а также цветовая индикация прогресса.

В правом верхнем углу экрана расположены две кнопки для экспорта отчета: сохранение в формате PDF в папку «Загрузки» устройства и отправка отчета через стандартный диалог «Поделиться».

Пример сформированного PDF-отчета представлен на рисунке 5.13.

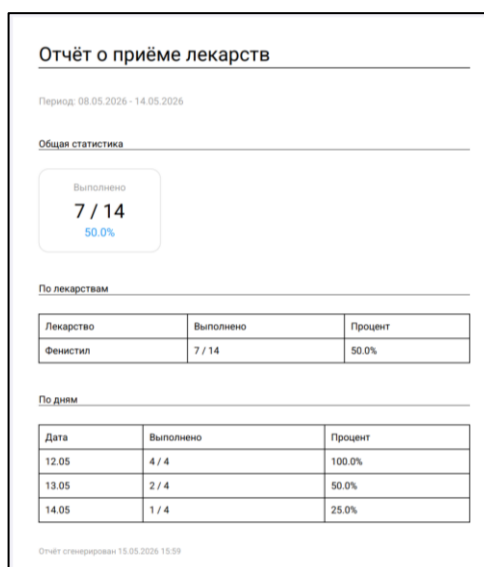


Рисунок 5.13 – PDF-отчет о приеме лекарств

Сформированный PDF-документ содержит общую статистику, таблицу по лекарствам, таблицу по дням, а также период отчета и дату генерации.

5.7 Виджет

Для удобства пользователя в приложении реализован виджет, отображающий на главном экране устройства список предстоящих приемов лекарств на текущий день. Виджет позволяет пользователю быстро узнавать о запланированных приемах без необходимости открывать само приложение.

Внешний вид виджета представлен на рисунке 5.14.

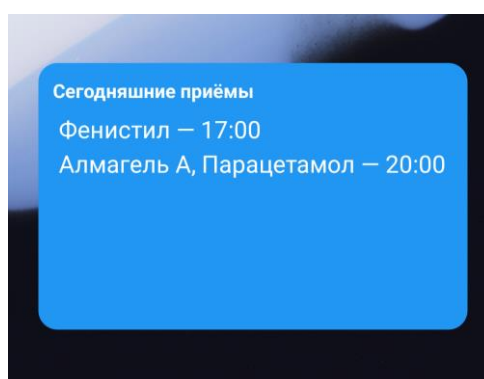


Рисунок 5.14 – Виджет

Виджет отображает список предстоящих приемов лекарств на текущий день. Для каждого приема показывается название препарата и время, в которое его необходимо принять. Если на одно время запланировано несколько лекарств, они группируются в одной строке через запятую. При отсутствии запланированных приемов на текущий день виджет отображает сообщение «Нет приемов». Виджет автоматически обновляется каждую минуту, поэтому информация всегда остается актуальной. При нажатии на виджет открывается основное приложение, где пользователь может отметить прием.

Виджет поддерживает два размера. В компактном режиме отображаются две строки с предстоящими приемами. В расширенном режиме отображаются до четырех строк, а заголовок меняется на «Сегодняшние приемы». При изменении размера виджета на рабочем столе он автоматически переключается между компактным и расширенным режимами. Для удаления виджета необходимо нажать и удерживать его, затем перетащить на появившуюся в верхней части экрана иконку «Удалить» или корзину. Виджет работает только при наличии данных в приложении – если пользователь не добавил ни одного расписания, виджет всегда будет показывать сообщение об отсутствии приемов.

5.8 Вывод по разделу

В данном разделе были рассмотрены все основные экраны и функции разработанного мобильного приложения для контроля приема лекарств. Подробно описан процесс регистрации и авторизации пользователей, включая автоматическое создание гостевой учетной записи при первом запуске и объединение данных при переходе к полноценному аккаунту. Представлена структура и функционал главного экрана с календарем и карточками расписаний, где пользователь может просматривать запланированные приемы и отмечать их выполнение с переключением статусов по циклу «не отмечено – принято – пропущено». Описан экран создания и редактирования расписаний, включающий выбор лекарства из общей или личной базы, указание дозировки, периода приема и времени. Рассмотрена страница «Все лекарства» со справочной информацией, возможностью поиска, фильтрации по источнику данных и просмотра детальной информации о каждом препарате. Представлена страница «В наличии» (аптечка), позволяющая отслеживать остатки лекарств с автоматическим списанием при отметке приема и уведомлениями о низком запасе. Описан экран статистики с визуализацией выполнения приемов в виде столбчатой диаграммы и прогресс-баров, а также функциями экспорта отчетов в PDF. Показан вспомогательный интерфейс в виде виджета на рабочем столе. Разработанный пользовательский интерфейс обеспечивает интуитивно понятную навигацию, наглядное отображение информации и удобное управление процессом лечения. Все описанные функции и экраны делают приложение удобным инструментом для контроля лечения как для новых пользователей, так и для тех, кто нуждается в расширенных возможностях мониторинга. Представленный интерфейс позволяет эффективно управлять приемом лекарств, отслеживать динамику и своевременно получать напоминания.

6 Технико-экономическое обоснование проекта

6.1 Общая характеристика разрабатываемого программного средства

Основной целью данного раздела является экономическое обоснование целесообразности разработки мобильного приложения для контроля приема лекарств, представленного в дипломном проектировании. Для выполнения поставленной цели будут определены экономические показатели, в том числе себестоимость продукта.

Объектом разработки является мобильное приложение приема лекарств, предназначенное для контроля приема лекарственных препаратов. Целью программного средства, реализуемого в дипломном проекте, является улучшение и систематизация процесса соблюдения режима лечения пациентами. Пользователь имеет возможность создавать расписания приема лекарств с указанием дозировки и времени, отмечать выполненные приемы с переключением статусов, получать своевременные напоминания через системные уведомления. Для авторизованных пользователей дополнительно доступны функции просмотра справочной информации о лекарственных препаратах, управления остатками лекарств в персональной аптечке, просмотра статистики соблюдения режима лечения за выбранный период, экспорта отчетов в формате PDF, а также синхронизации данных между несколькими устройствами. Разрабатываемое программное обеспечение отличается от существующих аналогов наличием встроенного гостевого режима, позволяющего начать использование без регистрации, функционалом учета остатков лекарств в аптечке с автоматическим списанием при отметке приема, а также возможностью создания собственных лекарств в личную базу.

Способ монетизации – продажа разработанного программного средства и имущественных прав на него заказчику.

В рамках данного раздела необходимо определить затраты, произведенные на всех стадиях разработки описанного программного средства.

6.2 Исходные данные для проведения расчетов

Созданная система программ не является принципиально новым программным средством, ввиду наличия большого количества аналогов на рынке мобильных приложений для контроля приема лекарств, таких как Medisafe, MyTherapy, «Пилюля» и другие. Разработанное приложение относится ко второй группе программ по степени сложности, поскольку включает

					ДП 06.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			6 Технико экономическое обоснование проекта	Лит.	Лист	Листов
Пров.		Тимонович Г.Л.				У	1	11
Консульт.		Соболевский А.С.				БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.						
Утверд.		Блинова Е.А.						

в себя клиент-серверное взаимодействие, интеграцию с облачной базой данных Firebase, систему локальных уведомлений, а также функции экспорта данных в PDF и синхронизацию между устройствами. Источниками исходных данных для расчетов выступают действующие законы и нормативно-правовые акты, а также данные, полученные в результате анализа рынка труда в сфере информационных технологий. Исходные данные для расчета стоимости разработки приведены в таблице 6.1.

Таблица 6.1 – Исходные данные для расчета

Наименование показателя	Единица измерения	Условные обозначения	Норматив
Численность разработчиков	чел.	$Ч_p$	1
Норматив дополнительной заработной платы	%	$Н_{д.з.}$	15
Ставка отчислений в Фонд социальной защиты населения	%	$Н_{ф.с.з.н.}$	34,6
Ставка отчислений по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний	%	$Н_{б.г.с.}$	0,6
Норматив прочих затрат	%	$Н_{п.з.}$	18
Норматив общепроизводственных и общехозяйственных расходов	%	$Н_{обп, обх}$	25
Норматив расходов на сопровождение и адаптацию	%	$Н_{р.с.а.}$	10
Норматив накладных расходов	%	$Н_{опб, опх}$	25

Указанные в таблице 6.1 нормативы соответствуют действующему законодательству Республики Беларусь и типовым значениям для организаций, осуществляющих разработку программного обеспечения. Коэффициент изменения скорости обработки информации $K_{ск} = 0,7$ отражает использование современных сред разработки (Android Studio, Flutter), которые повышают производительность труда программиста. Тарифная ставка первого разряда и тарифный коэффициент приняты на уровне, соответствующем должности инженера-программиста в организациях бюджетной сферы. Далее эти данные понадобятся для расчета всех необходимых параметров в процессе экономического анализа приложения.

6.3 Методика обоснования цены

В современных рыночных экономических условиях программное средство выступает преимущественно в виде продукции организаций, представляющей собой функционально завершенные и имеющие товарный вид продукты, реализуемые покупателям по рыночным отпускным ценам. Данный подход позволяет оценивать программное обеспечение не просто как набор кода, а как полноценный товар, способный приносить прибыль. Все завершенные разработки являются научно-технической продукцией. Разработанное мо-

бильное приложение для контроля приема лекарств полностью соответствует данным критериям, поскольку представляет собой готовый к использованию программный продукт, имеющий интуитивно понятный интерфейс, полную документацию и может быть передан заказчику для дальнейшего использования или коммерческого распространения.

Широкое применение вычислительных технологий требует постоянного обновления и совершенствования программных средств. Выбор эффективных проектов в этой сфере связан с их экономической оценкой и расчетом экономического эффекта, который может определяться как у разработчика, так и у пользователя. В случае данного дипломного проекта экономический эффект у разработчика выступает в виде чистой прибыли от прав на использование программного средства, остающейся в распоряжении организации-разработчика. У пользователя экономический эффект выражается в виде экономии трудовых, материальных и финансовых ресурсов, получаемой за счет:

- снижения трудоемкости контроля за соблюдением режима лечения;
- сокращения расходов на приобретение платных аналогов,
- улучшения показателей эффективности лечения в результате своевременного приема лекарственных препаратов.

Оценка стоимости мобильного приложения у разработчика предполагает определение затрат, что включает следующие статьи:

- заработная плата исполнителей (основная и дополнительная);
- отчисления в Фонд социальной защиты населения и по обязательному страхованию от несчастных случаев на производстве и заболеваний;
- расходы на материалы и комплектующие;
- расходы на специальное оборудование;
- расходы на оплату машинного времени;
- общепроизводственные и общехозяйственные расходы;
- расходы, связанные с сопровождением программного средства.

На основании рассчитанных затрат определяется полная себестоимость разработки и отпускная цена разработанного мобильного приложения.

6.4 Определение объема программного средства

В таблице 6.2 указаны в укрупненном виде все работы, выполненные для создания указанного в дипломной работе программного средства, и количество рабочих дней, потраченных для выполнения этих работ.

Таблица 6.2 – Содержание и объем функций на разрабатываемое ПС

Содержание работ	Исполнитель	Количество разработчиков	Затраты рабочего времени, дней
Согласование ТЗ с заказчиком	Проджект-менеджер	1	5
Анализ требований и проектирование архитектуры	Бизнес-аналитик	1	3

Продолжение таблицы 6.2

Создание дизайна интерфейса (UI/UX)	UI/UX-дизайнер	1	5
Разработка клиентской части мобильного приложения	Flutter-разработчик	1	20
Разработка модуля аутентификации и интеграция с Firebase	Flutter-разработчик	1	5
Разработка системы уведомлений и виджета	Flutter-разработчик	1	5
Тестирование и отладка приложения	Тестировщик	1	10
Координация и управление проектом	Проджект-менеджер	1	2
Всего			52

Как видно из таблицы, при разработке программного средства проджект-менеджер привлекался на 4 рабочих дня, бизнес-аналитик – на 3 рабочих дня, Flutter-разработчик – на 30 дней, UI/UX-дизайнер – на 5 дней, тестировщик – на 10 дней. Всего программное средство разрабатывалось 52 дня.

6.5 Расчет основной заработной платы

Для определения величины основной заработной платы было проведено исследование величин заработных плат для специалистов в сфере мобильной разработки на языке программирования Dart с использованием фреймворка Flutter. Анализ проводился на основе данных с сайтов по поиску работы и кадровых агентств за 2025–2026 годы. Средняя заработная плата для позиции Junior Flutter-разработчика составляет 2 200 руб., для позиции Junior UI/UX-дизайнера – 1 900 руб., для позиции Junior тестировщика – 1 700 руб., для позиции Junior бизнес-аналитика – 1 850 руб., для позиции Junior проджект-менеджера – 2 100 руб.

Количество рабочих дней, затраченных на разработку программного средства, рассчитано в таблице 6.2 и переводится в количество рабочих месяцев из расчета 21 рабочий день в месяце. Время работы исполнителя i -й должности рассчитывается по формуле 6.1:

$$T_{\text{раб } i} = \frac{T_{\text{дн } i}}{D_{\text{м}}}, \quad (6.1)$$

где $T_{\text{раб } i}$ – время работы исполнителя i -й должности, мес.;

$T_{\text{дн } i}$ – затраты рабочего времени исполнителя i -й должности, дней;

$D_{\text{м}}$ – среднее количество рабочих дней в месяце, дней.

Согласно данным таблицы 6.2 проджект-менеджер работал на протяжении 7 дней, что соответствует 0,33 месяца; бизнес-аналитик работал 3 дня, что соответствует 0,14 месяца; Flutter-разработчик работал на протяжении 30

дней, что соответствует 1,43 месяца; UI/UX-дизайнер работал 5 дней, что соответствует 0,24 месяца; тестировщик работал 10 дней, что соответствует 0,48 месяца. Основная заработная плата рассчитывается по формуле 6.2:

$$C_{озi} = T_{разi} \cdot C_{зпi} \cdot K_{разi} \quad (6.2)$$

где $C_{озi}$ – основная заработная плата, руб.;

$T_{разi}$ – время раработки, месяцев;

$C_{зпi}$ – средняя часовая заработная плата;

$K_{разi}$ – количество разработчиков, человек.

Рассчитаем основную заработную плату специалистов, привлекаемых для реализации программного средства:

$$C_{озПМ} = 0,19 \cdot 2\,100 \cdot 1 = 399,00 \text{ руб.},$$

$$C_{озБА} = 0,14 \cdot 1\,850 \cdot 1 = 259,00 \text{ руб.},$$

$$C_{озФР} = 1,43 \cdot 2\,200 \cdot 1 = 3\,146,00 \text{ руб.},$$

$$C_{озДиз} = 0,24 \cdot 1\,900 \cdot 1 = 456,00 \text{ руб.},$$

$$C_{озтест} = 0,48 \cdot 1\,700 \cdot 1 = 816,00 \text{ руб.}$$

Суммарная основная заработная плата всех исполнителей проекта определяется по формуле 6.3:

$$C_{оз} = \sum_{i=1}^n C_{озi} \quad (6.3)$$

$$C_{оз} = 399,00 + 259,00 + 3\,146,00 + 456,00 + 816,00 = 5\,076,00 \text{ руб.}$$

В итоге, основная заработная плата составит 5 076,00 руб.

В дальнейшем для других расчетов используется основная заработная плата, рассчитанная по указанной выше методике.

6.6 Расчет основной заработной платы

Дополнительная заработная плата включает выплаты, предусмотренные законодательством о труде Республики Беларусь (оплата отпусков, выполнение государственных обязанностей, выплаты за выслугу лет и другие компенсационные выплаты), и определяется по нормативу в процентах к основной заработной плате по формуле 6.4:

$$C_{дз} = \frac{C_{оз} \cdot H_{дз}}{100}, \quad (6.4)$$

где $C_{дз}$ – дополнительная заработная плата, руб.;

$C_{оз}$ – основная заработная плата, руб.;

$H_{дз}$ – норматив дополнительной заработной платы, %.

После подстановки основной заработной платы получается значение

$$C_{\text{дз}} = 5\,076,00 \cdot 15 / 100 = 761,40 \text{ руб.}$$

Таким образом дополнительная заработная плата специалиста по разработке приложений на позиции Junior составила 761,40 рублей.

6.7 Расчет отчислений на социальные цели

Отчисления в Фонд социальной защиты населения и по обязательному страхованию от несчастных случаев на производстве и профессиональных заболеваний в БРУСП «Белгосстрах» рассчитываются по формуле 6.5:

$$C_{\text{фсзн}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot N_{\text{фсзн}}}{100}, \quad (6.5)$$

где $C_{\text{фсзн}}$ – отчисления в Фонд социальной защиты населения, руб.;

$C_{\text{оз}}$ – основная заработная плата, руб.;

$C_{\text{дз}}$ – дополнительная заработная плата, руб.;

$N_{\text{фсзн}}$ – норматив отчислений в Фонд социальной защиты населения, %.

После подстановки фонда оплаты труда получается значение:

$$C_{\text{фсзн}} = (5\,076,00 + 761,40) \cdot 34 / 100 = 1\,984,72 \text{ руб.}$$

Отчисления по обязательному страхованию от несчастных случаев на производстве в БРУСП «Белгосстрах» вычисляются по формуле 6.6:

$$C_{\text{бгс}} = \frac{(C_{\text{оз}} + C_{\text{дз}}) \cdot N_{\text{бгс}}}{100}, \quad (6.6)$$

где $C_{\text{бгс}}$ – отчисления в БРУСП «Белгосстрах», руб.;

$N_{\text{бгс}}$ – ставка отчислений по обязательному страхованию, %.

По нормативу обязательного страхования получается значение:

$$C_{\text{бгс}} = (5\,076,00 + 761,40) \cdot 0,6 / 100 = 35,02 \text{ руб.}$$

Таким образом, общие отчисления в БРУСП «Белгосстрах» составили 35,02 руб., а в фонд социальной защиты населения – 1 984,72 руб.

6.8 Расчет расходов на специальное оборудование и платных услуг

Расходы на специальное оборудование и платные услуги, необходимые для разработки и отладки разрабатываемого модуля, не осуществлялись. Вся инфраструктура для разработки уже имелась в наличии, что позволило не привлекать сторонние организации и не приобретать дополнительное оборудование.

6.9 Расчет расходов на материалы

Сумма расходов на материалы C_M определяется из таблицы 6.3, в которой собраны реально понесенные затраты на разработку программного средства.

Таблица 6.3 – Материальные затраты на разработку программного средства

Вид материальных затрат	Фактические затраты, руб.
Бумага, 0,75 пачки	7,00
Картридж Canon, 0,5 шт.	12,00
Всего	19,00

Сумма расходов на материалы составляет 19,00 руб.

6.10 Расчет прочих прямых затрат

Прочие прямые затраты включают расходы на поиск и подготовку справочной, нормативной и технической информации, сопутствующие организационные расходы, связанные с разработкой программного средства. Они рассчитываются по нормативу к основной заработной плате по формуле 6.7:

$$C_{пз} = \frac{C_{оз} \cdot H_{пз}}{100}, \quad (6.7)$$

где $C_{пз}$ – прочие прямые затраты, бел. руб.;

$H_{пз}$ – норматив прочих прямых затрат, %.

После подстановки основной заработной платы получается значение

$$C_{пз} = 5\,076,00 \cdot 18 / 100 = 913,68 \text{ руб.}$$

Сумма прочих прямых затрат составит 913,68 руб.

6.11 Расчет общепроизводственных и общехозяйственных расходов

Общепроизводственные и общехозяйственные расходы отражают затраты организационного характера, связанные с обеспечением условий разработки: использованием рабочего места, связью, администрированием проекта и эксплуатацией общей инфраструктуры. Данные расходы являются косвенными и распределяются пропорционально основной заработной плате исполнителей.

Сумма расходов рассчитывается по формуле 6.8:

$$C_{обп, обх} = \frac{C_{оз} \cdot H_{обп, обх}}{100}, \quad (6.8)$$

где $C_{обп, обх}$ – общепроизводственные и общехозяйственные расходы, руб.;

$N_{\text{обп, обх}}$ – норматив общепроизводственных и общехозяйственных расходов, % от основной заработной платы.

После применения принятого норматива получается значение

$$C_{\text{обп, обх}} = 5\,076,00 \cdot 25 / 100 = 1\,269,00 \text{ руб.}$$

Размер вычисленных расходов составит 1 269,00 руб.

6.12 Расчет суммы расходов на разработку программного средства

Сумма расходов на разработку программного средства определяется как сумма основной и дополнительной заработной платы исполнителей, отчислений на социальные цели, расходов на материалы, расходов на специальное оборудование и платные услуги, прочих прямых затрат и общепроизводственных и общехозяйственных расходов по формуле 6.9:

$$C_p = C_{\text{оз}} + C_{\text{дз}} + C_{\text{фсзн}} + C_{\text{бгс}} + C_{\text{м}} + C_{\text{сопу}} + C_{\text{пз}} + C_{\text{обп, обх}}. \quad (6.9)$$

С учетом рассчитанных ранее значений сумма расходов составляет

$$C_p = 5\,076,00 + 761,40 + 1\,984,72 + 35,02 + 24,00 + 0 + 913,68 + 1\,269,00 = 10\,063,82 \text{ руб.}$$

Общая сумма расходов на разработку средства составит 10 063,82 руб.

6.13 Расчет расходов на сопровождение и адаптацию

Расходы на сопровождение и адаптацию программного средства включают его настройку после внедрения, первичную техническую поддержку, исправление выявленных ошибок и адаптацию под условия эксплуатации. Данные расходы необходимы для обеспечения стабильной работы программного продукта у заказчика. Они рассчитываются по нормативу от суммы расходов на разработку по формуле 6.10:

$$C_{\text{р.с.а}} = \frac{C_p \cdot N_{\text{р.с.а}}}{100}, \quad (6.10)$$

где $C_{\text{р.с.а}}$ – расходы на сопровождение и адаптацию, руб.;

$N_{\text{р.с.а}}$ – норматив расходов на сопровождение и адаптацию, %.

После подстановки себестоимости разработки получается значение

$$C_{\text{р.с.а}} = 10\,063,82 \cdot 10 / 100 = 1\,006,38 \text{ руб.}$$

Сумма расходов на сопровождение и адаптацию была вычислена на основе данных, рассчитанных ранее в данном разделе, и составила 1 006,38 руб.

Все проведенные выше расчеты необходимы для вычисления полной себестоимости проекта. Без расчета полной себестоимости невозможно обосновать цену программного средства и его конкурентоспособность на рынке. Следовательно, данный расчет является ключевым при оценке экономической эффективности разработки и ее окупаемости.

6.14 Расчет полной себестоимости программного средства

Полная себестоимость программного средства определяется как сумма расходов на разработку и расходов на сопровождение и адаптацию и рассчитывается по формуле 6.11:

$$C_{\pi} = C_p + C_{p.c.a.} \quad (6.11)$$

С учетом расходов на разработку и сопровождение получается:

$$C_{\pi} = 10\,063,82 + 1\,006,38 = 11\,070,20 \text{ руб.}$$

Полученная структура полной себестоимости далее обобщается в таблице затрат на создание и первичное сопровождение программного средства.

6.15 Определение цены, оценка эффективности

Для оценки отпускной цены разработки необходимо рассмотреть цены разработки приложений, обладающих аналогичным функционалом. Для этого воспользуемся помощью трех сайтов-агрегаторов услуг по разработке программного обеспечения.

Так как стоимость разработки проектов в приведенных ниже примерах рассчитывается не в белорусских рублях, то было необходимо выполнить перевод (по курсу Национального банка Республики Беларусь на 05.05.2026)

Первым ресурсом является сайт <https://estimatemyapp.com>. Результаты расчетов стоимости разработки аналогичного приложения на рисунке 6.1.

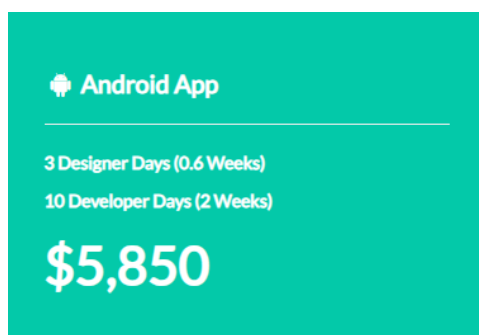


Рисунок 6.1 – Расчет стоимости сайта estimatemyapp.com

Как видно из результатов, примерная сумма похожего приложения обойдется в 5 850 долларов, что в пересчете на белорусские рубли составляет примерно 16 202,74 рублей.

Вторым ресурсом является сайт thebestapp.ru. Этот ресурс работает по аналогии с предыдущим, за исключением того, что итоговая цена будет представлена в российских рублях. В отличие от зарубежных аналогов, данный сервис ориентирован на реалии российского рынка разработки мобильных приложений. На рисунке 6.2 представлен результат работы сайта:

Скриншот веб-интерфейса сайта thebestapp.ru. В центре экрана крупным шрифтом выделена стоимость разработки приложения: **450 000 ₽**. Над этой суммой мелким шрифтом написано «Стоимость разработки вашего приложения». Справа от суммы находится синяя кнопка с белым текстом «ПОЛУЧИТЬ РАСЧЕТ СТОИМОСТИ». Ниже этой кнопки, также по центру, находится ссылка «ОЧИСТИТЬ ФОРМУ».

Рисунок 6.2 – Результат работы сайта thebestapp.ru

Как видно из результатов, примерная сумма похожего ресурса обойдется в 450 000 рублей, что в пересчете на белорусские рубли составляет примерно 17 182,35 рублей.

Последним ресурсом является сайт elapp.ru. Выбрав все необходимые компоненты системы, приблизительная стоимость составляет 521 600 российских рублей, что в пересчете составляет 19 916,25 белорусских рублей. Ниже на рисунке 6.3 приведен результат работы сайта

Скриншот веб-интерфейса сайта elapp.ru. В центре экрана крупным шрифтом выделена предварительная стоимость приложения или проекта: **521 600 руб**. Над этой суммой мелким шрифтом написано «ПРЕДВАРИТЕЛЬНАЯ СТОИМОСТЬ ВАШЕГО ПРИЛОЖЕНИЯ ИЛИ ПРОЕКТА:». Ниже суммы находится красная кнопка с белым текстом «Отправить заявку».

Рисунок 6.3 – Результат работы сайта elapp.ru

Как видно из результатов, отпускная цена на сайтах услуг по разработке программного обеспечения не сильно отличается друг от друга. Средняя цена разработки приложения с аналогичным функционалом рассчитывается как среднее арифметическое указанных выше цен:

$$Ц_{\text{ср}} = (16\,202,74 + 17\,182,35 + 19\,916,25) / 3 = 17\,767,11$$

Минимальная цена разработки приложения с аналогичными функциональными требованиями, составляет 17 767,11 руб.

Так как полная себестоимость разработки и поддержки программного средства составляет 11 070,20 руб., есть возможность установить цену, соответствующую средней цене.

Установим $C_{\text{с НДС}} = 17\,767,11$ руб.
 Цена ПС без НДС рассчитывается:

$$C_{\text{без НДС}} = 17\,767,11 * (100 / 120) = 14\,805,93 \text{ руб.}$$

Прибыль от реализации ПС:

$$\Pi_{\text{рп}} = 14\,805,93 - 11\,070,20 = 3\,735,73 \text{ руб.}$$

Рентабельность ПС:

$$P_{\text{ПС}} = (3\,735,73 / 11\,070,20) * 100 = 33,7\%$$

Таким образом, проведенные расчеты показывают, что разработка программного средства является экономически целесообразной.

6.16 Выводы по разделу

В таблице 6.4 приведены результаты расчетов экономических показателей, проведенных в данном разделе.

Таблица 6.4 – Расчет затрат на создание программного средства

Наименование показателя	Значение показателя
Время разработки, дней	52
Основная заработная плата	5 076,00
Дополнительная заработная плата	761,40
Отчисления в Фонд социальной защиты населения	1 984,72
Отчисления в БРУСП «Белгосстрах»	35,02
Расходы на материалы	24,00
Расходы на специальное оборудование и платные услуги	0
Прочие прямые затраты	913,68
Общепроизводственные и общехозяйственные расходы	1 269,00
Себестоимость разработки программного средства	10 063,82
Расходы на сопровождение и адаптацию	1 006,38
Полная себестоимость программного средства	11 070,20
Цена без НДС	14 805,93
Прибыль от реализации, руб.	3 735,73
Рентабельность ПС, %	33,7

Таким образом, выполненные экономические расчеты подтверждают, что разработка представленного программного продукта экономически обоснована, а предполагаемые затраты являются вполне обоснованными и окупятся в процессе его реализации.

7 Анализ информационной безопасности приложения

В процессе разработки мобильного приложения для отслеживания медикаментозного лечения одной из главных задач являлось обеспечение надежной защиты пользовательской информации. Поскольку система работает с чувствительными медицинскими сведениями: перечнем препаратов, режимом дозирования, календарем приемов, историей отметок и количеством остатков в персональной аптечке, вопросы безопасности выходят на первый план. Современные вызовы в области кибербезопасности, такие как несанкционированное проникновение, компрометация персональных данных, перехват интернет-трафика и дизассемблирование кода, требуют внедрения комплексной эшелонированной защиты, способной предотвратить потенциальные атаки и гарантировать целостность информации как непосредственно на мобильном устройстве, так и при ее передаче в облачную инфраструктуру. В настоящем разделе детально рассматриваются внедренные механизмы безопасности, среди которых аутентификация через Firebase, защита каналов связи, соблюдение норм работы с персональными данными, контролируемая доставка push-сообщений и процедуры резервного копирования.

7.1 Процедуры идентификации и разграничения доступа

Для верификации пользователей в приложении задействован сервис Firebase Authentication – облачное решение от компании Google, предоставляющее масштабируемые и защищенные инструменты проверки учетных записей. Firebase Authentication поддерживает несколько вариантов входа, включая классическую пару «электронная почта – пароль», а также авторизацию через аккаунт Google. Подобный выбор продиктован стремлением совместить простоту использования для конечного потребителя с серьезным уровнем безопасности. При оформлении нового пользователя вводимый пароль не покидает устройство в незашифрованном виде и не фиксируется локально –

Firebase производит хеширование по современному алгоритму bcrypt, генерируя индивидуальную соль для каждой учетной записи. Это гарантирует, что даже при возможной компрометации базы данных злоумышленник не сможет восстановить оригинальные пароли, поскольку хеш-функция действует в одну сторону, а наличие соли делает бессмысленным применение заранее вычисленных радужных таблиц.

Дополнительным плюсом использования Firebase Authentication выступает встроенный механизм противодействия атакам подбора (brute-force). Сервис автоматически фиксирует количество безуспешных попыток авторизации

					ДП 07.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			7 Анализ информационной безопасности приложения	Лит.	Лист	Листов
Пров.		Тимонович Г.Л.				У	1	5
						БГТУ 1-98 01 03, 2026		
Н. контр.		Подрез А.А.						
Утверд.		Блинова Е.А.						

с конкретного IP или для определенной учетной записи и при достижении порогового значения временно блокирует возможность входа, требуя увеличения паузы между попытками. Это делает автоматизированный перебор паролей крайне затруднительным. При подключении через Google система запрашивает лишь минимальный набор сведений – адрес электронной почты, имя и аватар, что соответствует принципу наименьших привилегий. Пользователь не передает системе лишних данных, а сам процесс верификации перепоручается надежному внешнему провайдеру.

В приложении также присутствует гостевой режим, дающий возможность начать использование без создания учетной записи. В этом случае все данные – графики приема, история отметок – хранятся в Cloud Firestore, но привязываются к временному анонимному идентификатору. Синхронизация между разными устройствами в таком режиме отсутствует. Если позже пользователь принимает решение зарегистрироваться, программа автоматически объединяет накопленную гостем информацию с новой учетной записью.

7.2 Обеспечение безопасности при обмене данными

Для синхронизации информации между несколькими устройствами владельца и для централизованного хранения в приложении задействована база данных Firebase Firestore – документно-ориентированное решение, входящее в экосистему Firebase. Весь трафик между мобильным клиентом и серверами Firestore автоматически шифруется с помощью протокола TLS (Transport Layer Security) версии 1.2 или новее. TLS гарантирует три фундаментальных свойства: конфиденциальность передаваемых сведений (защита от прослушивания), аутентификацию сервера (подтверждение, что соединение установлено именно с подлинным сервером Firebase, а не с фишинговым узлом), а также целостность сообщений (обнаружение любых попыток модификации данных в пути). Актуальные версии TLS используют алгоритмы на основе открытых ключей для безопасного согласования сеансового ключа, что исключает возможность расшифровки перехваченного трафика.

Firebase Firestore предлагает гибкую систему правил безопасности (Security Rules), определяющих условия доступа к документам в облаке. Правила описываются на специальном языке и проверяются на стороне сервера при каждом обращении. В разработанном приложении настроены правила, разрешающие доступ только авторизованным пользователям. Каждый человек может читать и изменять исключительно те документы, которые относятся к его учетной записи – идентификатор пользователя (UID) присутствует в пути к документам в коллекциях `users/{uid}/schedules`, `users/{uid}/medicines`, `users/{uid}/stocks` и аналогичных. Такой подход обеспечивает жесткую изоляцию информации: даже если злоумышленник гипотетически получит доступ к облачной инфраструктуре, он не сможет просмотреть чужие данные, поскольку правила безопасности откажут в разрешении.

Firebase также включает автоматические механизмы противодействия злоупотреблениям: система мониторинга выявляет аномальную активность

(например, необычно высокую частоту запросов с одного устройства) и может временно ограничить доступ. Встроенные алгоритмы ограничения частоты запросов (rate limiting) предотвращают DDoS-атаки и автоматизированный перебор параметров. Для дополнительной защиты в проекте используется аутентификация на основе токенов: каждый вызов к Firestore сопровождается JWT-токеном, выданным Firebase Authentication, который содержит информацию о пользователе и сроке действия. Сервер проверяет подпись токена, прежде чем применить правила безопасности.

7.3 Конфиденциальность

Приложение оперирует персональными данными – адресом электронной почты, именем пользователя, а также медицинской информацией (наименования лекарств, дозировки, расписания, история приемов, остатки в аптечке), которая во многих юрисдикциях относится к категории чувствительных. В связи с этим система должна соответствовать требованиям Общего регламента по защите данных (GDPR) Европейского союза, а также национальным законам о персональных данных. В соответствии с этими нормативными актами в приложение встроены инструменты, реализующие права субъектов данных. Пользователь в любой момент может удалить свою учетную запись через боковое меню. При активации этой функции программа удаляет все данные из облачного хранилища Firestore (документы в коллекциях `schedules`, `medicines`, `stocks` и других, связанные с данным пользователем), а также очищает локально сохраненную информацию на устройстве. Процесс удаления необратим, однако перед его запуском запрашивается подтверждение и выводится предупреждение о последствиях.

Сбор информации в приложении ограничен минимально необходимым для обеспечения заявленных функций – этот принцип называется минимизацией данных. Для корректной работы достаточно знать адрес электронной почты для входа и восстановления пароля, а также медицинские сведения, которые пользователь вводит добровольно при создании расписаний и добавлении лекарств в аптечку. Никакие дополнительные данные (например, точное местоположение, список контактов, показания сенсоров движения) не собираются и не запрашиваются. Все собираемые данные используются исключительно для работы самого мобильного приложения и не передаются третьим лицам. Пользователь полностью контролирует свою информацию и может в любой момент ее удалить.

Все данные, отправляемые в Firebase Firestore, хранятся на серверах, размещенных в регионе, выбранном при создании проекта. Firebase не передает информацию третьим сторонам без согласия пользователя, за исключением случаев, прямо предусмотренных законодательством. Для дополнительной защиты конфиденциальности реализована возможность формирования PDF-отчета для врача – пользователь отправляет этот файл самостоятельно, без передачи данных через серверы приложения. Такой подход исключает риск утечки медицинских данных при передаче.

7.4 Защита системы уведомлений

Функционал уведомлений является одной из ключевых возможностей приложения, обеспечивающей своевременные напоминания о необходимости принять лекарство. Для его реализации используется библиотека `flutter_local_notifications`, которая взаимодействует с системным Notification-Manager на Android. Уведомления функционируют в изолированной среде Android – каждое приложение работает в собственной песочнице, не имея доступа к уведомлениям других программ. Для предотвращения возможных атак, направленных на подмену или перехват уведомлений, применяются дополнительные меры. Все уведомления, создаваемые приложением, получают уникальный идентификатор, формируемый на основе хеша идентификатора расписания и порядкового номера приема. Это исключает возможность предсказания идентификатора и подмены уведомления другим приложением.

Сами уведомления не содержат избыточной конфиденциальной информации. В заголовке выводится стандартная фраза «Напоминание о приеме», а в теле сообщения – название препарата и дозировка (например, «Пора принять: Парацетамол (500 мг)»). Полная информация о курсе лечения, истории приемов, остатках в аптечке доступна только после открытия приложения. Это снижает риск утечки медицинских сведений в ситуациях, когда устройство разблокировано и уведомление видно на экране посторонним лицам. Пользователь также может в системных настройках отключить предварительный просмотр содержимого уведомлений.

Для восстановления уведомлений после перезагрузки устройства в приложении реализован механизм перепланирования. При каждом запуске программа загружает все активные расписания из Firestore и заново регистрирует уведомления на предстоящие даты. Никакая конфиденциальная информация не передается через Intent, а все внутренние идентификаторы генерируются заново в процессе восстановления. Запрос разрешения на показ уведомлений выполняется при первом запуске приложения, что соответствует требованиям Android 13 и более новых версий.

7.5 Резервное копирование и синхронизация данных

Потеря информации – одна из самых серьезных проблем для медицинского приложения, поскольку восстановление истории приемов и расписаний может оказаться невозможным. Для предотвращения утраты данных в проекте используется облачное хранилище Firebase Firestore. Все сведения пользователя (расписания, история приемов, личные лекарства, содержимое аптечки) сохраняются в облаке и привязываются к уникальному идентификатору владельца. При входе на новое устройство данные автоматически загружаются из облака, что обеспечивает их сохранность даже при потере или поломке телефона. В конфигурации Firestore включено автоматическое резервное копирование всех документов, позволяющее восстановить информацию в случае сбоя облачной инфраструктуры.

Приложение также предоставляет возможность экспорта статистических данных в формате PDF, позволяя пользователю сохранить отчет о приеме лекарств на устройстве. Для обратного импорта данных из PDF-отчета используется ручной ввод, поскольку формат PDF не предназначен для автоматического восстановления в текстовый формат.

При синхронизации между несколькими устройствами используется стратегия «последняя запись побеждает», основанная на временных метках. Каждый документ в Firestore содержит поле `updatedAt`, которое обновляется при каждом изменении. При возникновении конфликта (например, пользователь офлайн изменил запись на одном устройстве, а на другом тоже внес изменения до синхронизации) побеждает та версия, у которой значение временной метки больше. Это гарантирует, что ни одно изменение не будет потеряно.

7.6 Выводы по разделу

В рамках обеспечения информационной безопасности мобильного приложения были внедрены комплексные меры, гарантирующие сохранность и конфиденциальность пользовательских данных. Для верификации пользователей задействован сервис Firebase Authentication с хешированием паролей по алгоритму `bcrypt` с индивидуальной солью, а также встроенной защитой от атак подбора (`brute-force`). Приложение поддерживает вход по электронной почте и паролю, авторизацию через Google-аккаунт, а также гостевой режим с возможностью последующего объединения данных.

Все сетевые взаимодействия между клиентом и серверами Firestore защищены протоколом TLS, исключающим перехват и модификацию информации. Правила безопасности Firestore (Security Rules) реализуют строгую изоляцию данных: каждый пользователь имеет доступ только к своим документам на основе уникального идентификатора `UID`. Пользователь может удалить учетную запись с полным стиранием всех связанных данных из облачного хранилища и локального устройства. Сбор информации ограничен минимально необходимым набором (принцип минимизации данных), никакие дополнительные сведения не запрашиваются и не передаются третьим лицам.

Уведомления не содержат избыточной медицинской информации, используют уникальные идентификаторы для предотвращения подмены и восстанавливаются после перезагрузки устройства через механизм перепланирования. Резервное копирование и синхронизация реализованы через Firestore с автоматической загрузкой данных при входе на новое устройство и стратегией разрешения конфликтов «последняя запись побеждает» на основе временных меток, что минимизирует риски потери информации.

Спроектированная система безопасности охватывает все критические направления – от аутентификации пользователей до резервного копирования и защиты уведомлений. Перспективы развития включают шифрование локального хранилища на устройстве, внедрение двухфакторной аутентификации, а также добавление биометрического входа для более оперативного и безопасного доступа к чувствительным медицинским данным.

Заключение

В результате выполнения дипломного проекта разработано мобильное приложение для отслеживания приема лекарств, предназначенное для помощи пользователям в соблюдении назначенных схем лечения.

В ходе работы был проведен анализ существующих аналогов – Medisafe, myTherapy и Pill Reminder, что позволило сформулировать требования к собственной разработке и избежать типичных ошибок. На основе этого анализа спроектирована и реализована архитектура приложения на базе фреймворка Flutter и языка Dart с использованием паттерна управления состоянием Provider, облачной базы данных Cloud Firestore и сервиса аутентификации и авторизации Firebase Authentication.

Разработанное приложение обеспечивает регистрацию и вход пользователей (включая гостевой режим с последующим объединением данных), гибкое создание расписаний приема лекарств, систему локальных уведомлений, работающих при закрытом приложении, доступ к справочной информации о лекарствах, учет остатков в аптечке, а также экран статистики с визуализацией выполнения приемов и экспортом отчетов в PDF. Кроме того, реализован виджет рабочего стола Android для отображения сегодняшних приемов без открытия мобильного приложения.

Проведенное тестирование подтвердило корректную работу всех ключевых сценариев, а разработанное руководство пользователя позволяет быстро освоить функционал приложения. В разделе анализа информационной безопасности рассмотрены и внедрены механизмы защиты пользовательских данных: аутентификация через Firebase Authentication, шифрование трафика протоколом TLS, правила безопасности Firestore, минимизация сбора персональных данных и резервное копирование.

Приложение прошло полный цикл тестирования на реальных сценариях использования. Все выявленные в процессе тестирования недочеты были устранены, а функциональность полностью соответствует требованиям, предъявленным на этапе проектирования. Разработанный программный продукт может быть рекомендован для использования пациентами, нуждающимися в регулярном приеме лекарственных средств, а также для совместной работы с лечащим врачом посредством экспортируемых отчетов.

Таким образом, разработанное приложение представляет собой полноценное, готовое к эксплуатации решение для контроля приема лекарств. Перспективы дальнейшего развития включают портирование на iOS, добавление биометрического входа, шифрование локального хранилища и интеграцию с электронными медицинскими картами.

					ДП 00.00.ПЗ			
		ФИО	Подпись	Дата				
Разраб.		Осадчий П.А.			Заключение			
Пров.		Тимонович Г.Л.						
Н. контр.		Подрез А.А.						
Утверд.		Блинова Е.А.						
						Лит.	Лист	Листов
						У	1	1
						БГТУ 1-98 01 03, 2026		

Список использованных источников

1. Pill Reminder : [сайт]. – {Лимасол}, {2017–2026}. – URL: {<https://play.google.com/store/apps/details?id=com.pillreminder.app>} (дата обращения: 18.01.2026).
2. Medisafe : [сайт]. – {Бостон}, {2012–2026}. – URL: {<https://www.medisafe.com>} (дата обращения: 18.01.2026).
3. myTherapy : [сайт]. – {Берлин}, {2013–2026}. – URL: {<https://www.mytherapyapp.com>} (дата обращения: 18.01.2026).
4. Android : [сайт]. – {Маунтин-Вью}, {2007–2026}. – URL: {<https://www.android.com>} (дата обращения: 20.01.2026).
5. Google LLC. Dart language overview : [сайт]. – {Маунтин-Вью}, {2011–2026}. – URL: {<https://dart.dev/overview>} (дата обращения: 20.01.2026).
6. Android Studio : [сайт]. – {Маунтин-Вью}, {2013–2026}. – URL: {<https://developer.android.com/studio>} (дата обращения: 21.01.2026).
7. Google LLC. Flutter documentation : [сайт]. – {Маунтин-Вью}, {2017–2026}. – URL: {<https://docs.flutter.dev>} (дата обращения: 25.01.2026).
8. Pacher, M. Provider package documentation : [сайт]. – {Маунтин-Вью}, {2018–2026}. – URL: {<https://pub.dev/packages/provider>} (дата обращения: 31.01.2026).
9. Google LLC. Cloud Firestore Documentation : [сайт]. – {Маунтин-Вью}, {2017–2026}. – URL: {<https://firebase.google.com/docs/firestore>} (дата обращения: 15.02.2026).
10. Google LLC. Firebase Authentication for Flutter : [сайт]. – {Маунтин-Вью}, {2016–2026}. – URL: {<https://firebase.google.com/docs/auth/flutter>} (дата обращения: 20.02.2025).
11. flutter_local_notifications / Flutter package : [сайт]. – {Миссури}, {2017–2026}. – URL: {https://pub.dev/packages/flutter_local_notifications} (дата обращения: 24.02.2025)

					ДП 00.00.ПЗ				
		ФИО	Подпись	Дата					
Разраб.		Осадчий П.А.			Список использованных источников	Лит.	Лист	Листов	
Пров.		Тимонович Г.Л.				У	1	1	
						БГТУ 1-98 01 03, 2026			
Н. контр.		Подрез А.А.							
Утверд.		Блинова Е.А.							

ПРИЛОЖЕНИЕ А
Диаграмма развертывания

ПРИЛОЖЕНИЕ Б
Диаграмма вариантов использования

ПРИЛОЖЕНИЕ В
Логическая схема базы данных

ПРИЛОЖЕНИЕ Г

Блок-схема алгоритма создания расписания прием

ПРИЛОЖЕНИЕ Д
Таблица экономических показателей

ПРИЛОЖЕНИЕ Е

Скриншот работы программы